

In [1]: `%matplotlib widget`

```
import collections
import json
import math
import os
from os import path

import numpy as np
from matplotlib import pyplot as plt

# TODO: Add burstiness analysis from receiver pcap, flow level

FONTSIZE = 15
LINESIZE = 3

if __name__ == "__main__":
    RUN = True
else:
    RUN = False
```

In [2]: `if RUN:`

```
    EXP_DIR = "/home/pragna/work/chris_ns3/ns-3-dev-git/scratch/traces/DCTCF
    EXP = path.basename(EXP_DIR)
    GRAPH_DIR = path.join("/home/pragna/work/DC_bursts/Analysis-scripts/imc2
    if not path.isdir(GRAPH_DIR):
        os.makedirs(GRAPH_DIR)
```

In [3]: `def show(fig):`

```
    plt.tight_layout()
    # Change the toolbar position
    fig.canvas.toolbar_position = "left"
    # If true then scrolling while the mouse is over the canvas will not
    # move the entire notebook
    fig.canvas.capture_scroll = True
    fig.show()

def save(graph_dir, prefix=None, suffix=None, extent=None):
    """Save the entire figure."""
    assert prefix is not None or suffix is not None
    both_defined = prefix is not None and suffix is not None
    out_flp = path.join(
        graph_dir,
        (" if prefix is None else prefix)
        + (" " if both_defined else "")
        + (" if suffix is None else suffix),
    )
    plt.tight_layout()
    plt.savefig(out_flp + ".pdf", bbox_inches="tight" if extent is None else
    plt.savefig(
        out_flp + ".png", dpi=300, bbox_inches="tight" if extent is None els
    )
```

```

def save_axes(figure, axes, graph_dir, prefix=None, suffix=None):
    """Save a single axes, instead of the entire figure."""
    extent = axes.get_window_extent().transformed(figure.dpi_scale_trans.inverted())
    save(graph_dir, prefix, suffix, extent.expanded(1.3, 1.2))

def get_axes(rows=1, width=8, height=3, cols=1):
    with plt.ioff():
        fig, axes = plt.subplots(figsize=(width, height * rows), nrows=rows,
                                     ncols=cols)

        if rows == 1:
            axes = [axes]
        elif cols == 1:
            axes = axes.flatten()
        return fig, axes

def get_aligned_xs(old_start_sec, old_end_sec, interp_delta):
    # Create a new xs ndarray, ranging from >= old_start_sec to <= old_end_sec
    # at intervals of 1 / interp_delta.

    # Round the start *up* to the nearest multiple of 1 / interp_delta.
    # math.ceil(start / (1 / interp_delta)) * (1 / interp_delta)
    new_start_sec = math.ceil(old_start_sec * interp_delta) / interp_delta

    # Round the end *down* to the nearest multiple of 1 / interp_delta.
    # math.floor(end / (1 / interp_delta)) * (1 / interp_delta)
    new_end_sec = math.floor(old_end_sec * interp_delta) / interp_delta

    # If the old start and end are so close together that they do not overlap
    # aligned interval, then we cannot do anything.
    if new_start_sec > new_end_sec:
        return np.empty(0)

    return np.array(
        [
            x / interp_delta
            for x in range(
                math.ceil(new_start_sec * interp_delta),
                math.floor(new_end_sec * interp_delta) + 1,
            )
        ]
    )

def filter_samples(samples, start, end):
    return [sample for sample in samples if start <= sample[0] <= end]

def separate_samples_into_bursts(
    samples,
    burst_times,
    flow_times=None,
    filter_on_flow_times=False,
    bookend=True,

```

```

):
    num_bursts = len(burst_times)
    bursts = []

    if filter_on_flow_times:
        assert flow_times is not None
    else:
        assert flow_times is None
        flow_times = [(None, None, None, None)] * num_bursts

    for burst_idx, (
        (burst_start, burst_end),
        (flow_start, _, flow_end, _),
    ) in enumerate(zip(burst_times, flow_times)):
        burst = []
        for sample in samples:
            if burst_start <= sample[0] <= burst_end and (
                not filter_on_flow_times or flow_start <= sample[0] <= flow_end
            ):
                # This sample is part of the current burst.
                burst.append(sample)

        # Insert a sample at precisely the start and end time for this burst
        # if possible.
        if bookend:
            start, end = (
                (flow_start, flow_end)
                if filter_on_flow_times
                else (burst_start, burst_end)
            )
            if burst_idx > 0:
                # Make sure that the burst has a sample at the start time
                # Two case: Either we have no samples for this burst, so we
                # the last value from the previous burst, or we do have samples
                # this burst but not at the start time, so we also take the
                # value from the previous burst. In both cases, make sure there
                # a previous burst.
                if (not burst and bursts[-1]) or (
                    burst and burst[0][0] != start and bursts[-1]
                ):
                    burst.insert(0, (start, *bursts[-1][-1][1:]))
                # Every burst should now have at least one sample: start.
                # Note: This will fail if we have no data for the first burst.

            if burst:
                # Make sure that the burst has a sample at the end time
                if burst[-1][0] != end:
                    burst.append((end, *burst[-1][1:]))
                # Every burst should now have at least two samples: start and end.
                assert len(burst) >= 2, (burst, start, end)

        bursts.append(burst)
    # Make sure we have the expected number of bursts
    assert len(bursts) == num_bursts

    # Adjust the x values of the samples

```

```

bursts_ = []
for (start, end), burst in zip(burst_times, bursts):
    burst_ = []
    for sample in burst:
        burst_.append((sample[0] - start, *sample[1:]))
    bursts_.append(burst_)
return bursts_

```

```

In [4]: def parse_times_line(line):
# Format: <start time seconds> <end time seconds>
parts = line.strip().split(" ")
assert len(parts) == 2
return [float(sec) for sec in parts]

def get_burst_times(exp_dir):
with open(
    path.join(exp_dir, "logs", "burst_times.log"), "r", encoding="utf-8"
) as fil:
    return [parse_times_line(line) for line in fil if line.strip()[0] != "#"]

def get_config_json(exp_dir):
with open(path.join(exp_dir, "config.json"), "r", encoding="utf-8") as f:
    return json.load(f)

if RUN:
    BURST_TIMES = get_burst_times(EXP_DIR)
    # BURST_TIMES = [(start, (start + 0.03) if (end - start) > 0.03 else end) for start, end in BURST_TIMES]
    # BURST_TIMES = BURST_TIMES[:3]
    NUM_BURSTS = len(BURST_TIMES)
    CONFIG = get_config_json(EXP_DIR)
    # assert NUM_BURSTS == CONFIG["numBursts"]

    # ideal_sec = CONFIG["bytesPerSender"] * CONFIG["numSenders"] / (
    ideal_sec = CONFIG["bytesPerBurstSender"] * CONFIG["maxIncastScale"] / (
        CONFIG["smallLinkBandwidthMbps"] * 1e6 / 8
    ) + (6 * CONFIG["delayPerLinkUs"] / 1e6)
    print(
        "Burst times:",
        f"Ideal: {ideal_sec * 1e3:.4f} ms",
        *[
            (
                f"{burst_idx + 1}: [{start} -> {end}] - "
                f"{(end - start) * 1e3:.4f} ms - "
                f"{(end - start) / ideal_sec * 100:.2f} %"
            )
            for burst_idx, (start, end) in enumerate(BURST_TIMES)
        ],
        sep="\n",
    )

```

Burst times:

Ideal: 0.8036 ms

```
1: [2.623 -> 2.623844577] - 0.8446 ms - 105.10 %
2: [3.623844577 -> 3.625430162] - 1.5856 ms - 197.31 %
3: [4.625430162 -> 4.62692779] - 1.4976 ms - 186.36 %
4: [5.62692779 -> 5.62847114] - 1.5434 ms - 192.05 %
5: [6.62847114 -> 6.629709704] - 1.2386 ms - 154.13 %
6: [7.629709704 -> 7.630903309] - 1.1936 ms - 148.53 %
7: [8.630903309 -> 8.632423077] - 1.5198 ms - 189.12 %
8: [9.632423077 -> 9.633849906] - 1.4268 ms - 177.55 %
9: [10.633849906 -> 10.635489479] - 1.6396 ms - 204.03 %
10: [11.635489479 -> 11.636774133] - 1.2847 ms - 159.86 %
11: [12.636774133 -> 12.638288941] - 1.5148 ms - 188.50 %
12: [13.638288941 -> 13.639430613] - 1.1417 ms - 142.07 %
13: [14.639430613 -> 14.640844054] - 1.4134 ms - 175.89 %
```

```
In [5]: def parse_length_line(line):
# Format: <timestamp seconds> <num packets> <backlog time microseconds>
parts = line.strip().split(" ")
assert len(parts) == 2
time_sec, packets = parts
time_sec = float(time_sec)
packets = int(packets)
# backlog_us = float(backlog_us)
return time_sec, packets # , backlog_us

def parse_mark_line(line):
# Format <timestamp seconds>
parts = line.strip().split(" ")
assert len(parts) == 1
return (float(parts[0]), None)

def parse_drop_line(line):
# Format: <timestamp seconds> <drop type>
parts = line.strip().split(" ")
assert len(parts) == 2
time_sec, drop_type = parts
time_sec = float(time_sec)
drop_type = int(drop_type)
return time_sec, drop_type

def get_lengths_by_burst(exp_dir, queue_prefix, burst_times):
length_samples = []
with open(
    path.join(exp_dir, "logs", f"{queue_prefix}_depth.log"), "r", encoding="utf-8"
) as fil:
    length_samples = [
        parse_length_line(line) for line in fil if line.strip()[0] != "#"
    ]
    return separate_samples_into_bursts(length_samples, burst_times)

def get_marks_by_burst(exp_dir, queue_prefix, burst_times):
```

```

mark_samples = []
with open(
    path.join(exp_dir, "logs", f"{queue_prefix}_mark.log"), "r", encoding="utf-8"
) as fil:
    mark_samples = [parse_mark_line(line) for line in fil if line.strip()]
return separate_samples_into_bursts(mark_samples, burst_times, bookend=False)

def get_drops_by_burst(exp_dir, queue_prefix, burst_times):
    drop_samples = []
    with open(
        path.join(exp_dir, "logs", f"{queue_prefix}_drop.log"), "r", encoding="utf-8"
    ) as fil:
        drop_samples = [parse_drop_line(line) for line in fil if line.strip()]
    return separate_samples_into_bursts(drop_samples, burst_times, bookend=False)

def get_queue_metrics_by_burst(exp_dir, queue_name, burst_times):
    queue_prefix = (
        "incast_queue"
        if queue_name == "Incast Queue"
        else ("uplink_queue" if queue_name == "Uplink Queue" else None)
    )
    assert queue_prefix is not None
    return {
        "lengths": get_lengths_by_burst(exp_dir, queue_prefix, burst_times),
        "marks": get_marks_by_burst(exp_dir, queue_prefix, burst_times),
        "drops": get_drops_by_burst(exp_dir, queue_prefix, burst_times),
    }

def graph_queue(
    queue_name,
    lengths_by_burst,
    marks_by_burst,
    drops_by_burst,
    marking_threshold_packets,
    capacity_packets,
    graph_dir,
    prefix,
):
    for burst_idx, burst in enumerate(lengths_by_burst):
        # if burst_idx != len(lengths_by_burst) - 1:
        #     continue
        fig, axes = get_axes()
        ax = axes[0]

        # Plot length
        length_xs, length_ys = zip(*burst)
        length_xs = np.asarray(length_xs)
        length_xs = length_xs * 1e3
        ax.plot(
            length_xs,
            length_ys,
            label="queue length",
            drawstyle="steps-post",

```

```

        linewidth=LINESIZE,
        alpha=0.8,
    )
    max_x = length_xs[-1]
    max_y = max(length_ys)

    # If there are marks, plot them..
    if burst_idx < len(marks_by_burst) and marks_by_burst[burst_idx]:
        mark_xs, _ = zip(*marks_by_burst[burst_idx])
        mark_xs = np.asarray(mark_xs)
        mark_xs = mark_xs * 1e3
        mark_ys = [marking_threshold_packets] * len(mark_xs)
        ax.plot(
            mark_xs,
            mark_ys,
            "x",
            color="orange",
            label="ECN marks",
            linewidth=LINESIZE,
            alpha=0.8,
        )
        max_x = max(max_x, mark_xs[-1])

    # If there are drops, plot them..
    if burst_idx < len(drops_by_burst) and drops_by_burst[burst_idx]:
        drop_xs, _ = zip(*drops_by_burst[burst_idx])
        drop_xs = np.asarray(drop_xs)
        drop_xs = drop_xs * 1e3
        drop_ys = [capacity_packets] * len(drop_xs)
        ax.plot(
            drop_xs,
            drop_ys,
            "x",
            color="red",
            label="drops",
            linewidth=LINESIZE,
            alpha=0.8,
        )
        max_x = max(max_x, drop_xs[-1])

    # Draw a line at the marking threshold
    ax.plot(
        [0, max_x],
        [marking_threshold_packets] * 2,
        label="ECN threshold",
        color="orange",
        linestyle="dashed",
        linewidth=LINESIZE,
        alpha=0.8,
    )

    # For readability, only draw a line at the capacity if the max y is
    if max_y > capacity_packets / 2:
        # Draw a line at the queue capacity
        ax.plot(
            [0, max_x],

```

```

        [capacity_packets] * 2,
        label="queue capacity",
        color="red",
        linestyle="dotted",
        linewidth=LINESIZE,
        alpha=0.8,
    )
    max_y = capacity_packets
else:
    max_y = capacity_packets / 2

# ax.set_title(f"{queue_name} Length: Burst {burst_idx + 1} of {num_
ax.set_xlabel("time (ms)", fontsize=FONTSIZE)
ax.set_ylabel("packets", fontsize=FONTSIZE)
# ax.tick_params(axis='y', labelcolor=blue)
ax.tick_params(axis="x", labelsize=FONTSIZE)
ax.tick_params(axis="y", labelsize=FONTSIZE)
ax.set_xlim(left=-0.01 * max_x, right=1.01 * max_x)
ax.set_ylim(bottom=-0.01 * max_y, top=1.1 * max_y)
ax.legend(fontsize=FONTSIZE, loc="upper right")

with open(
    path.join(
        graph_dir,
        prefix
        + "-"
        + "_".join(queue_name.split(" ")).lower()
        + f"-burst{burst_idx}-length.dat",
    ),
    "w",
    encoding="utf-8",
) as fil:
    fil.write("time length\n")
    fil.write("\n".join(f"{x} {y}" for x, y in burst))

show(fig)
save(
    graph_dir,
    prefix,
    suffix="_".join(queue_name.split(" ")).lower() + "_" + str(burst
)

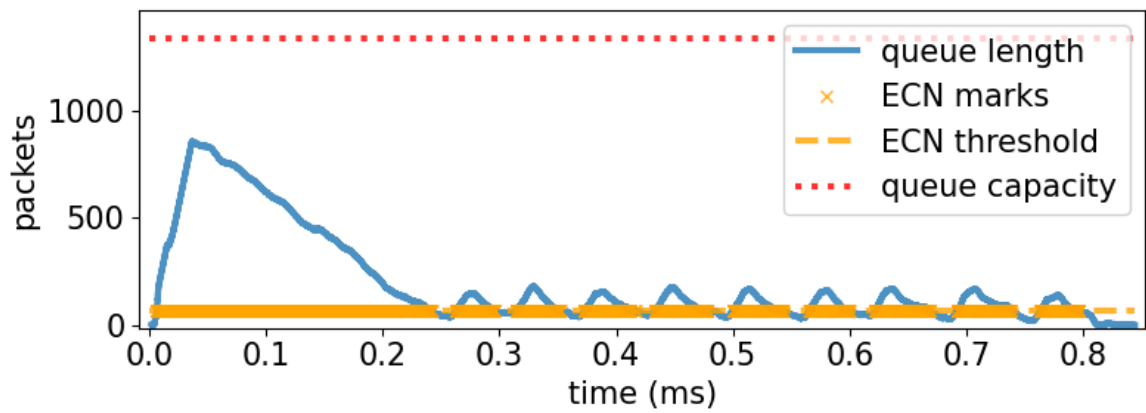
if RUN:
    MARKING_THRESHOLD = CONFIG["queueMinThresholdPackets"]
    QUEUE_CAPACITY = CONFIG["queueSizePackets"]
    INCAST_Q_METRICS = get_queue_metrics_by_burst(EXP_DIR, "Incast Queue", E
    graph_queue(
        "Incast Queue",
        INCAST_Q_METRICS["lengths"],
        INCAST_Q_METRICS["marks"],
        INCAST_Q_METRICS["drops"],
        MARKING_THRESHOLD,
        QUEUE_CAPACITY,
        GRAPH_DIR,

```

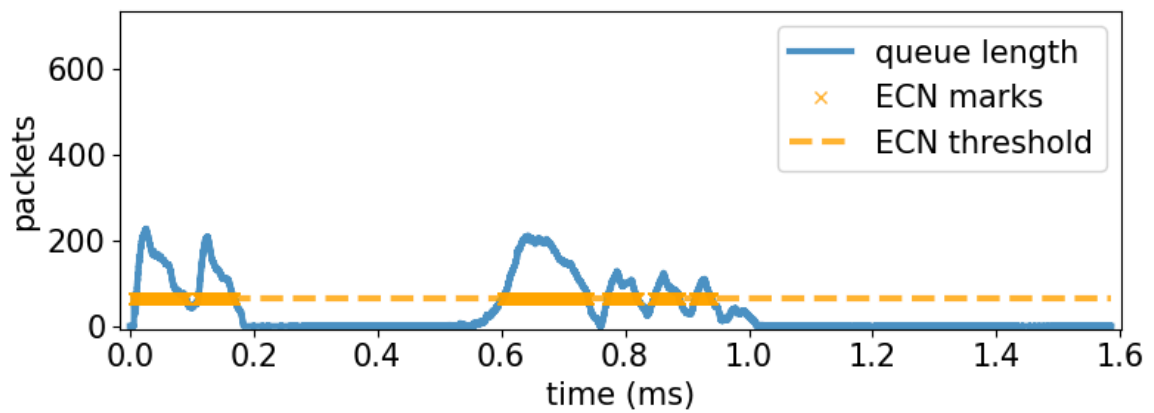
EXP,

)

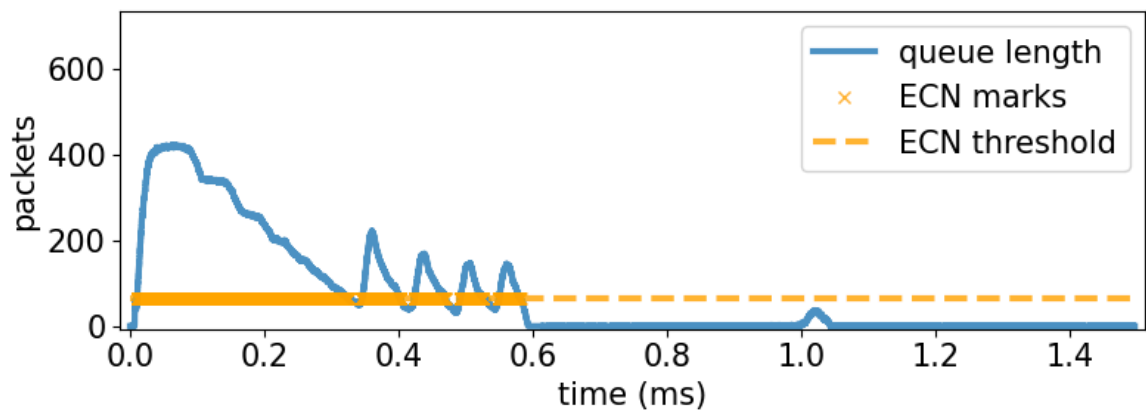
Figure



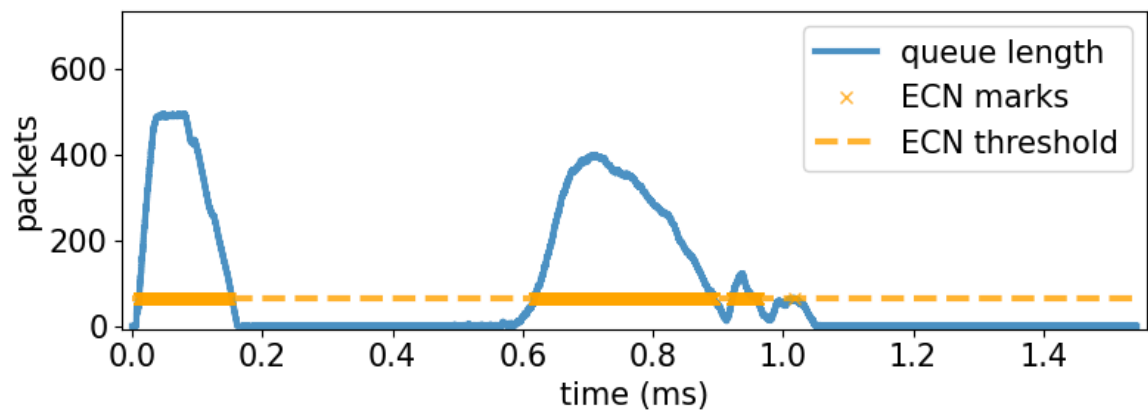
Figure



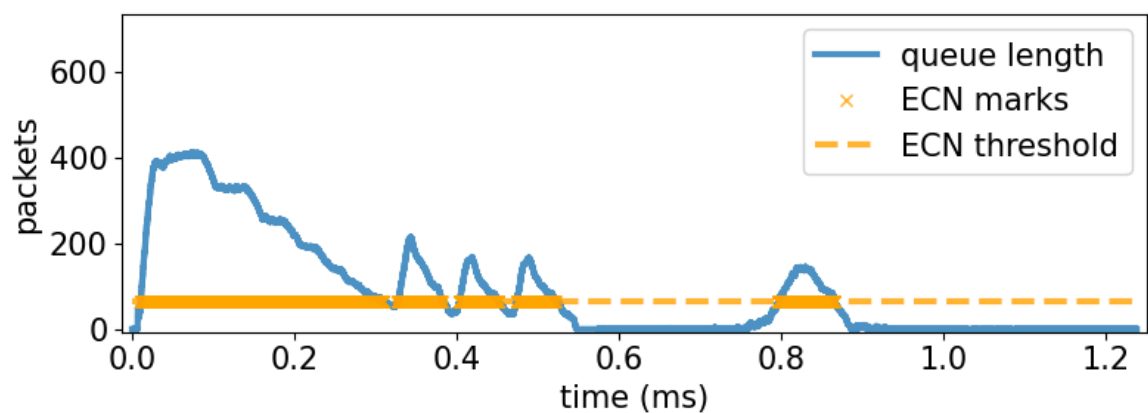
Figure



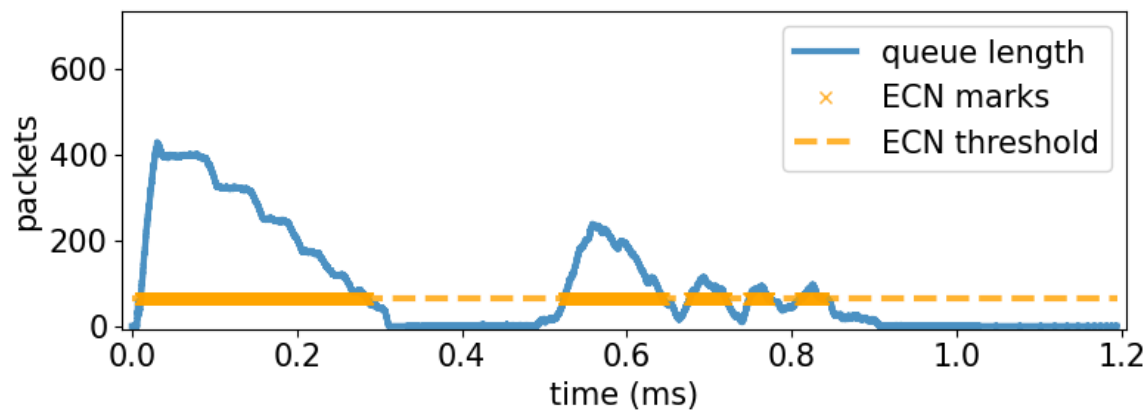
Figure



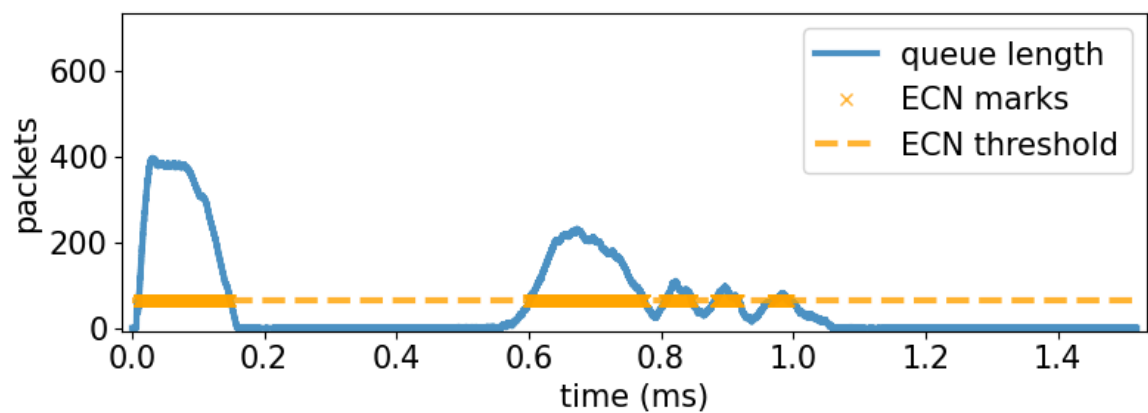
Figure



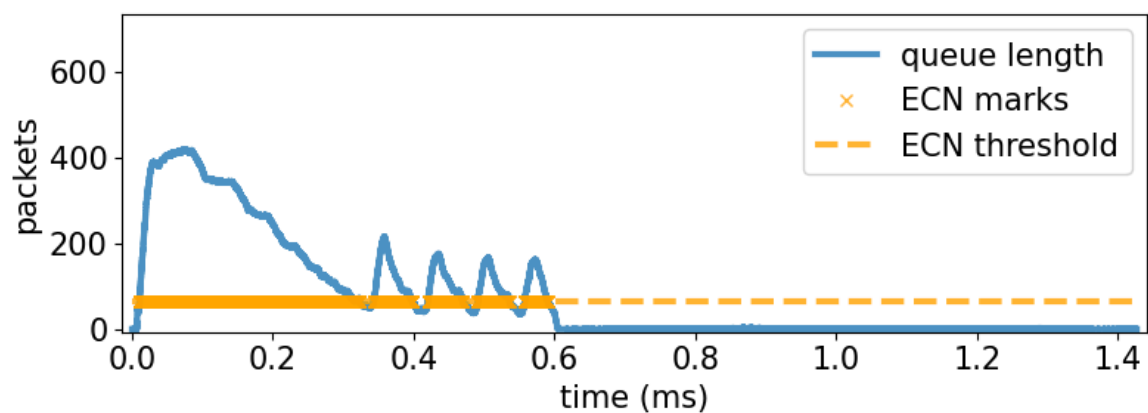
Figure



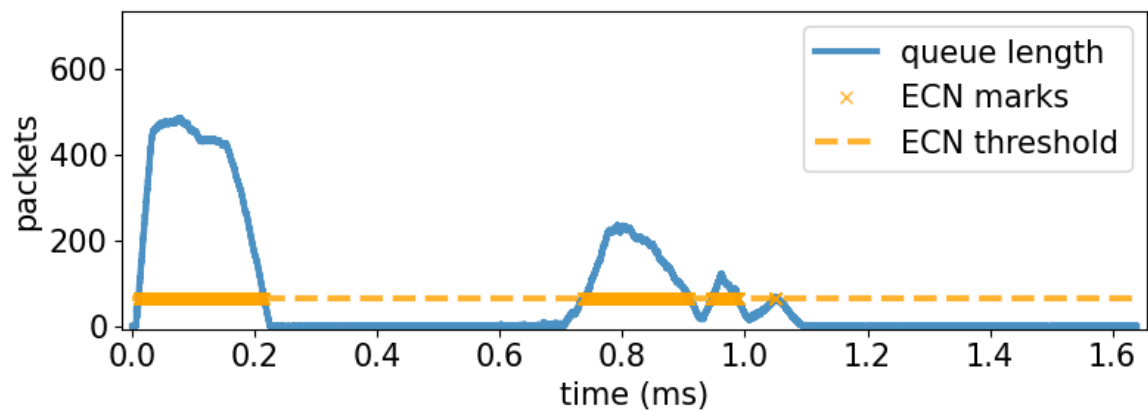
Figure



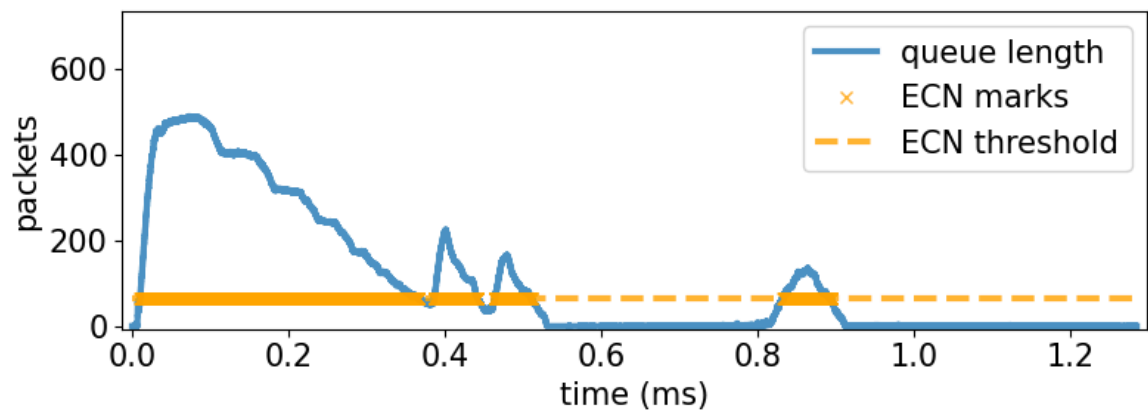
Figure



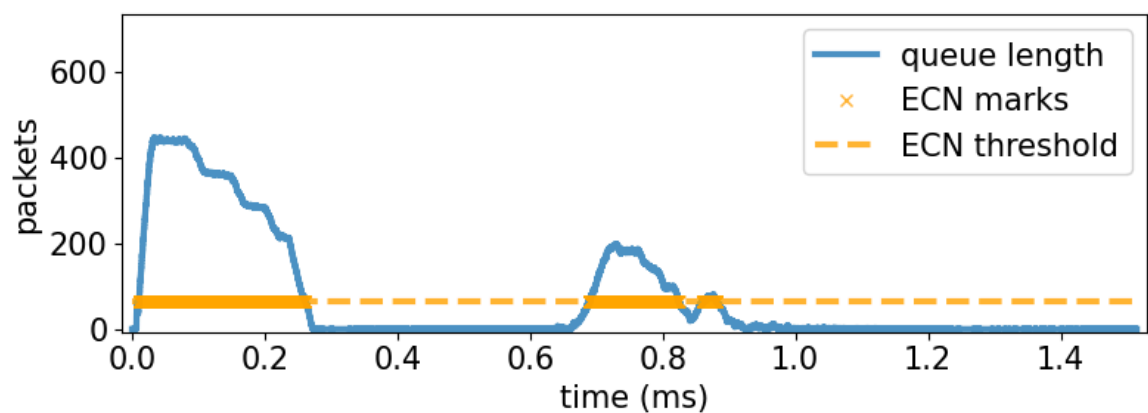
Figure



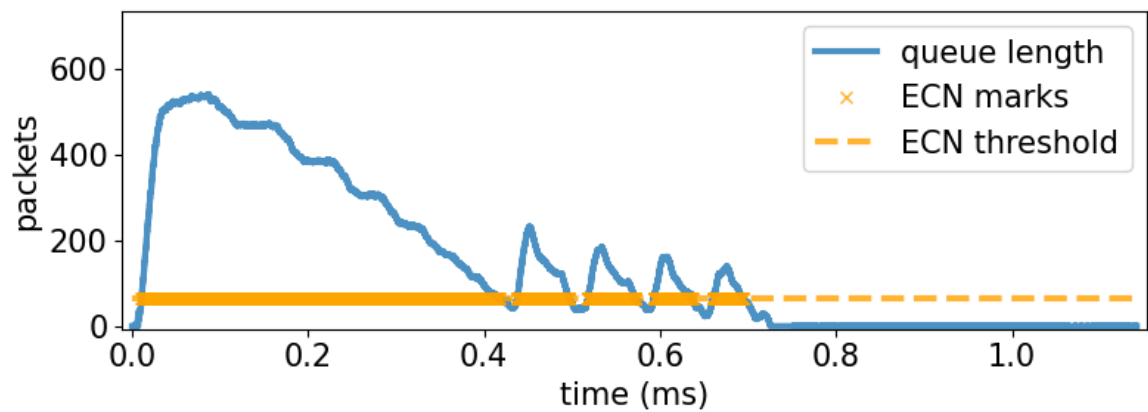
Figure



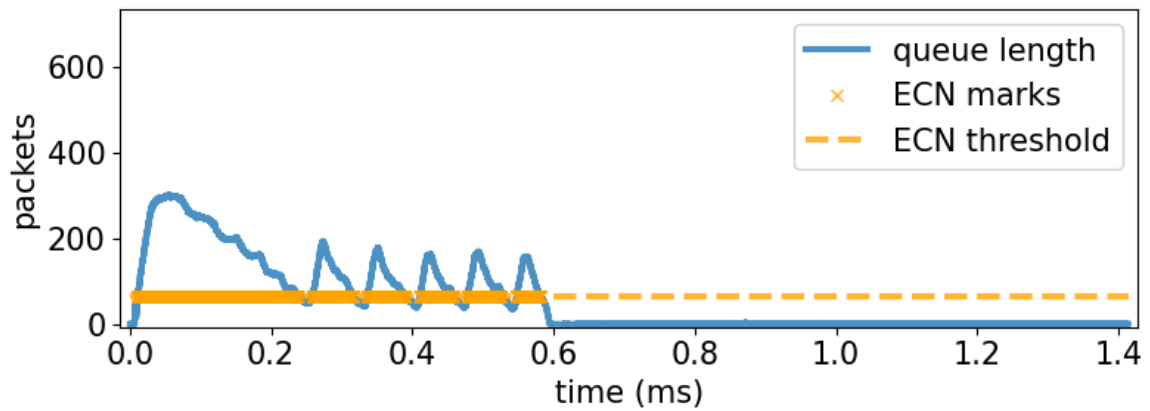
Figure



Figure



Figure



```
In [6]: def calculate_time_at_or_above_threshold_helper(lengths, thresh, start_sec,
# Identify crossover points and above regions points by filtering burst_
above_regions = []
last_length = None
last_cross_up = None
for x, length in lengths:
    if length < thresh:
        if last_cross_up is not None:
            above_regions.append((last_cross_up, x))
            last_cross_up = None
        elif length >= thresh:
            if last_length is None or last_length < thresh:
                last_cross_up = x
            last_length = length
    if last_cross_up is not None:
        above_regions.append((last_cross_up, end_sec))

above_sec = sum(
    region_end_sec - region_start_sec
    for region_start_sec, region_end_sec in above_regions
)
total_sec = end_sec - start_sec
return above_sec, total_sec, above_sec / total_sec * 100

def calculate_time_at_or_above_threshold(lengths_by_burst, burst_times, thresh):
    return [
        calculate_time_at_or_above_threshold_helper(lengths, thresh, start_sec, end_sec)
        for burst_idx, (lengths, (start_sec, end_sec)) in enumerate(
            zip(lengths_by_burst, burst_times)
        )
    ]

def print_q_above_thresh(lengths_by_burst, burst_times, thresh, label):
    num_bursts = len(burst_times)
    for burst_idx, (above_sec, _, perc) in enumerate(
        calculate_time_at_or_above_threshold(lengths_by_burst, burst_times,
    ):
        print(
```

```

        f"Burst {burst_idx + 1} of {num_bursts} "
        f"- Time above {label}: {above_sec * 1e3:.2f} ms ({perc:.2f}%)"
    )

if RUN:
    print_q_above_thresh(
        INCAST_Q_METRICS["lengths"], BURST_TIMES, MARKING_THRESHOLD, "marking"
    )

```

```

Burst 1 of 13 - Time above marking threshold: 0.60 ms (71.22%)
Burst 2 of 13 - Time above marking threshold: 0.39 ms (24.70%)
Burst 3 of 13 - Time above marking threshold: 0.49 ms (32.65%)
Burst 4 of 13 - Time above marking threshold: 0.46 ms (29.70%)
Burst 5 of 13 - Time above marking threshold: 0.52 ms (41.93%)
Burst 6 of 13 - Time above marking threshold: 0.49 ms (41.06%)
Burst 7 of 13 - Time above marking threshold: 0.41 ms (26.91%)
Burst 8 of 13 - Time above marking threshold: 0.50 ms (34.99%)
Burst 9 of 13 - Time above marking threshold: 0.42 ms (25.66%)
Burst 10 of 13 - Time above marking threshold: 0.52 ms (40.60%)
Burst 11 of 13 - Time above marking threshold: 0.41 ms (26.87%)
Burst 12 of 13 - Time above marking threshold: 0.60 ms (52.41%)
Burst 13 of 13 - Time above marking threshold: 0.47 ms (33.14%)

```

```

In [7]: if RUN:
        print_q_above_thresh(INCAST_Q_METRICS["lengths"], BURST_TIMES, 1, "empty")

```

```

Burst 1 of 13 - Time above empty: 0.82 ms (97.24%)
Burst 2 of 13 - Time above empty: 0.65 ms (41.01%)
Burst 3 of 13 - Time above empty: 0.65 ms (43.27%)
Burst 4 of 13 - Time above empty: 0.65 ms (42.12%)
Burst 5 of 13 - Time above empty: 0.72 ms (57.81%)
Burst 6 of 13 - Time above empty: 0.74 ms (61.74%)
Burst 7 of 13 - Time above empty: 0.68 ms (44.51%)
Burst 8 of 13 - Time above empty: 0.62 ms (43.11%)
Burst 9 of 13 - Time above empty: 0.65 ms (39.53%)
Burst 10 of 13 - Time above empty: 0.66 ms (51.19%)
Burst 11 of 13 - Time above empty: 0.63 ms (41.48%)
Burst 12 of 13 - Time above empty: 0.72 ms (63.39%)
Burst 13 of 13 - Time above empty: 0.60 ms (42.51%)

```

```

In [8]: if RUN:
        print_q_above_thresh(
            INCAST_Q_METRICS["lengths"], BURST_TIMES, QUEUE_CAPACITY * 0.9, "90% full"
        )

```

Burst 1 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 2 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 3 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 4 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 5 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 6 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 7 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 8 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 9 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 10 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 11 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 12 of 13 - Time above 90% capacity: 0.00 ms (0.00%)
 Burst 13 of 13 - Time above 90% capacity: 0.00 ms (0.00%)

```
In [9]: def parse_flow_times(flow_times_json):
    burst_to_sender_to_flow_times = [
        {
            times["id"]: (times["start"], times["firstPacket"], times["end"])
            for ip, times in flows.items()
        }
        for burst, flows in sorted(flow_times_json.items(), key=lambda p: ip)
    ]
    sender_to_flow_times_by_burst = {}
    for sender in burst_to_sender_to_flow_times[0].keys():
        sender_flow_times_by_burst = []
        for burst_idx in range(len(burst_to_sender_to_flow_times)):
            sender_flow_times_by_burst.append(
                burst_to_sender_to_flow_times[burst_idx][sender]
            )
        sender_to_flow_times_by_burst[sender] = sender_flow_times_by_burst
    return sender_to_flow_times_by_burst

def get_sender_to_flow_times_by_burst(exp_dir):
    with open(
        path.join(exp_dir, "logs", "flow_times.json"), "r", encoding="utf-8"
    ) as fil:
        return parse_flow_times(json.load(fil))

def get_active_flows_by_burst(sender_to_flow_times_by_burst, num_bursts):
    active_flows_by_burst = []
    for burst_idx in range(num_bursts):
        times = [
            flow_times_by_burst[burst_idx]
            for flow_times_by_burst in sender_to_flow_times_by_burst.values()
        ]
        starts, _, ends, _ = zip(*times)
        serialized = [(start, 1) for start in starts] + [(end, -1) for end in ends]
        serialized = sorted(serialized, key=lambda p: p[0])
        active = [serialized[0]]
        for time, action in serialized[1:]:
            active.append((time, active[-1][1] + action))
        active_flows_by_burst.append(active)
    return active_flows_by_burst
```

```

def graph_active_flows(active_flows_by_burst, num_bursts, graph_dir, prefix)
    for burst_idx in range(num_bursts):
        fig, axes = get_axes()
        ax = axes[0]

        xs, ys = zip(*active_flows_by_burst[burst_idx])
        ax.plot(xs, ys, drawstyle="steps-post", alpha=0.8)

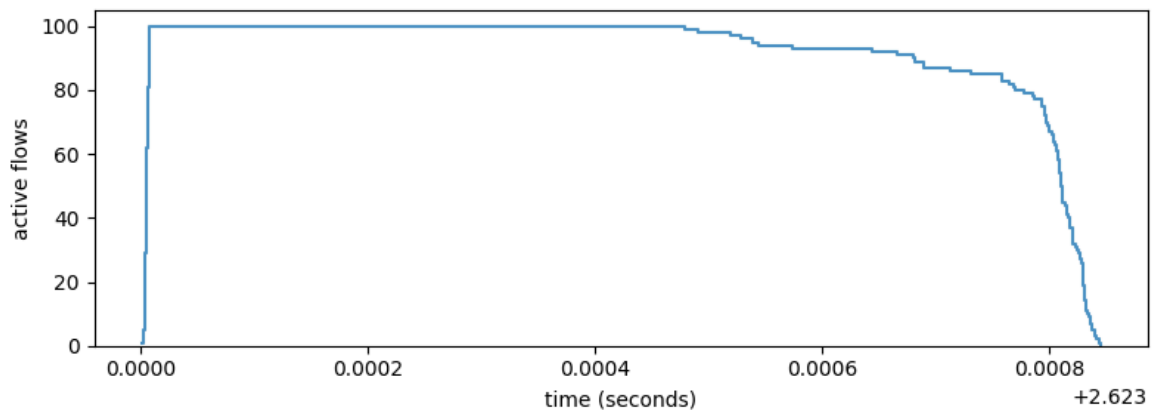
        # ax.set_title(
        #     f"Active flows over time: Burst {burst_idx + 1} of {num_bursts}
        # )
        ax.set_xlabel("time (seconds)")
        ax.set_ylabel("active flows")
        ax.set_ylim(bottom=0)

        show(fig)
        save(graph_dir, prefix, suffix=f"active_flows_{burst_idx}")

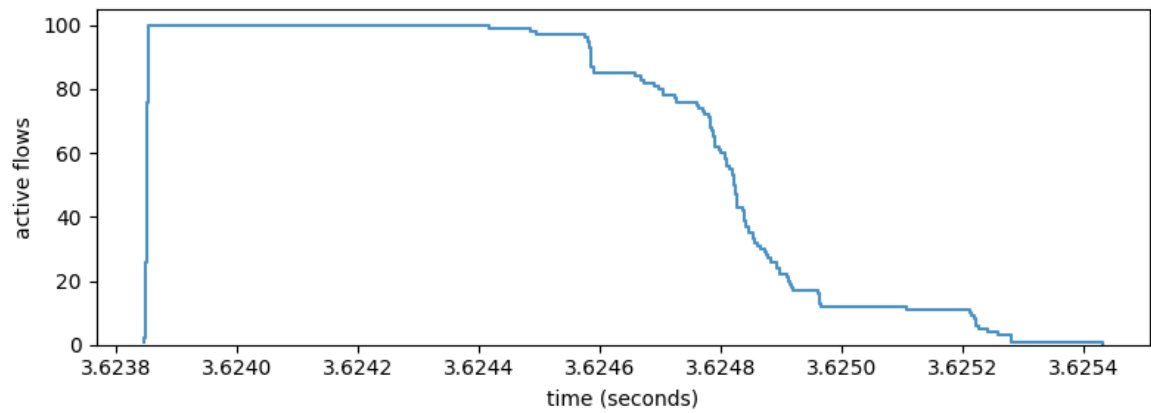
if RUN:
    SENDER_TO_FLOW_TIMES_BY_BURST = get_sender_to_flow_times_by_burst(EXP_DIR)
    ACTIVE_flowS_BY_BURST = get_active_flows_by_burst(
        SENDER_TO_FLOW_TIMES_BY_BURST, NUM_BURSTS
    )
    graph_active_flows(ACTIVE_flowS_BY_BURST, NUM_BURSTS, GRAPH_DIR, EXP)

```

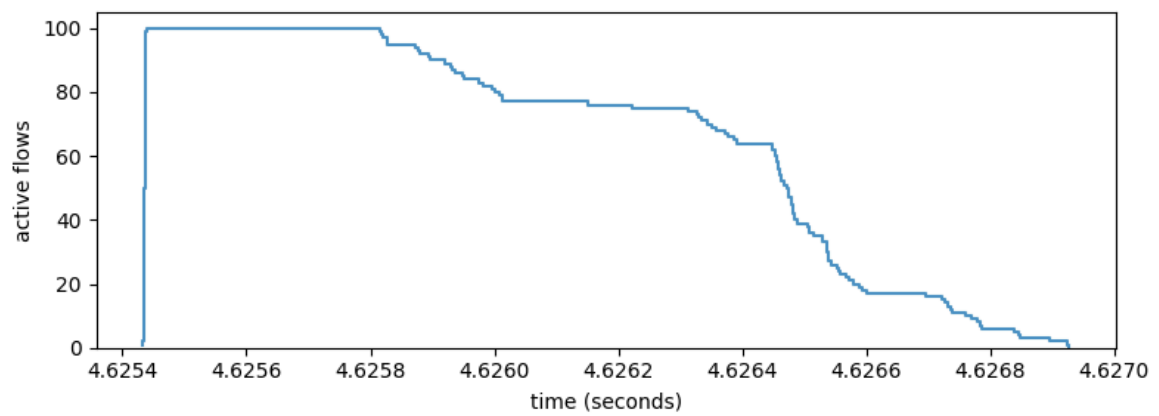
Figure



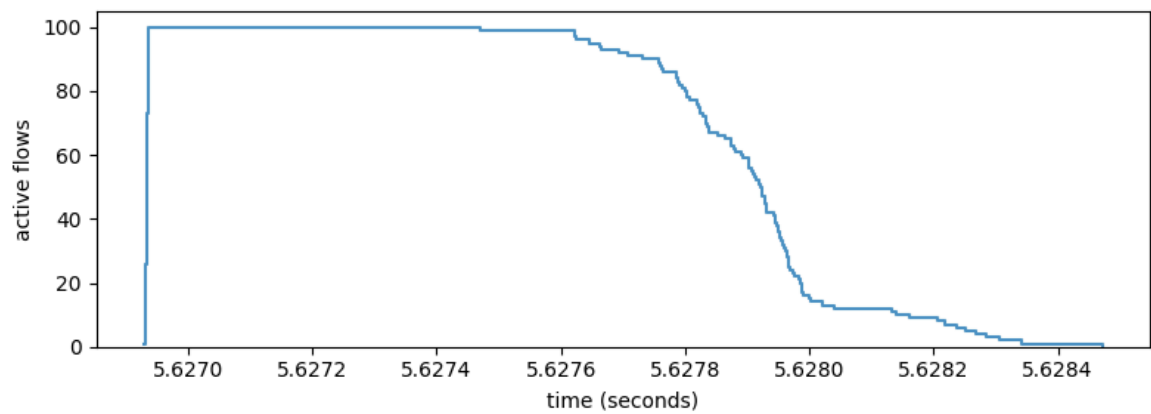
Figure



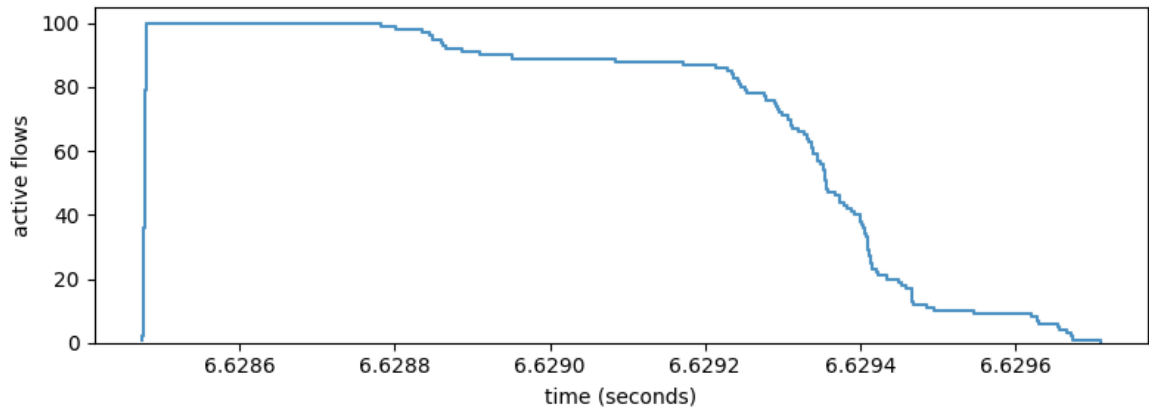
Figure



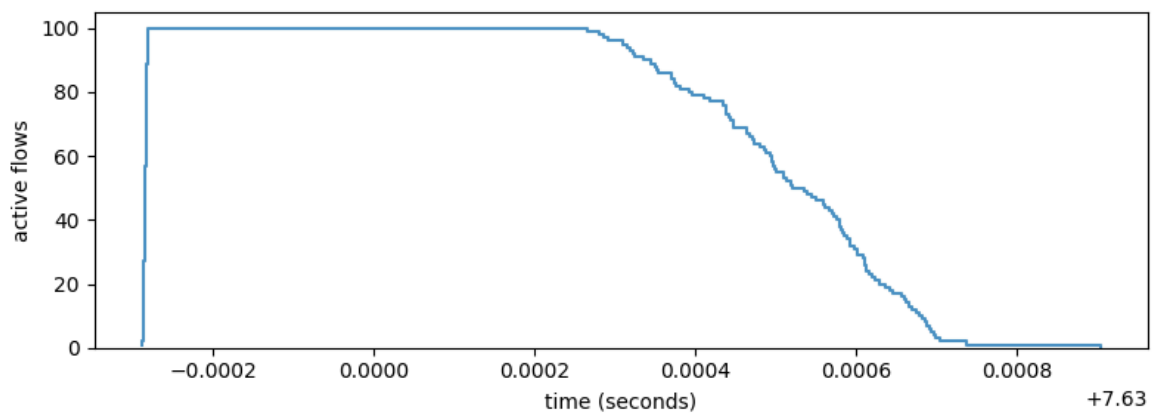
Figure



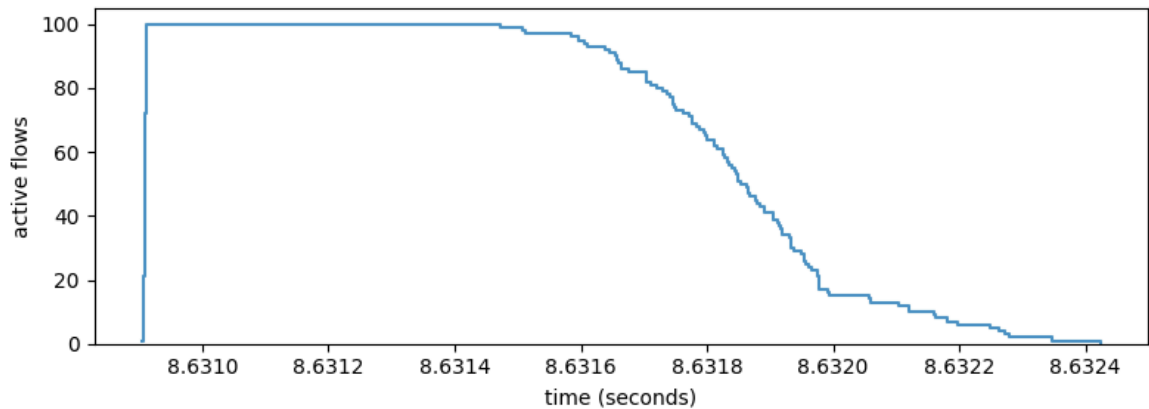
Figure



Figure



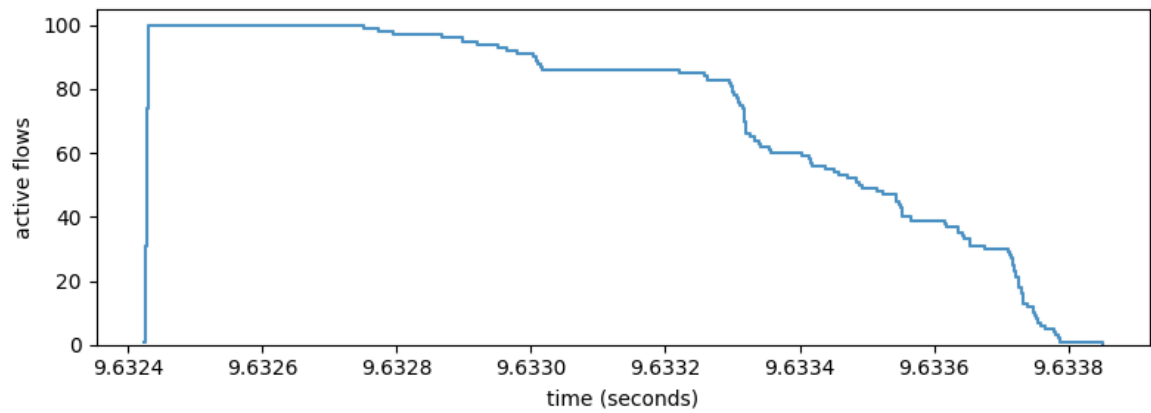
Figure



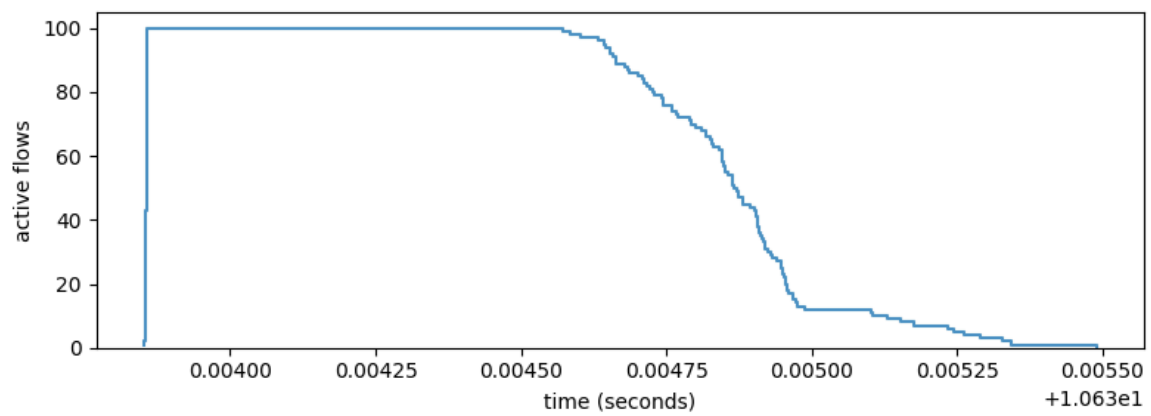
/tmp/ipykernel_3922147/1083710670.py:36: RuntimeWarning: More than 20 figures have been opened. Figures created through the pyplot interface (`matplotlib.pyplot.figure`) are retained until explicitly closed and may consume too much memory. (To control this warning, see the rcParam `figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.

```
fig, axes = plt.subplots(figsize=(width, height * rows), nrows=rows, ncols=cols)
```

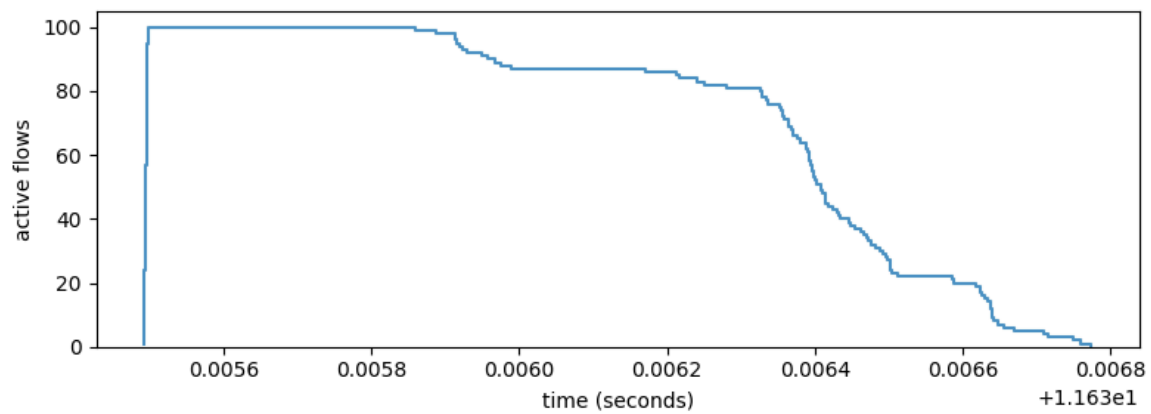
Figure



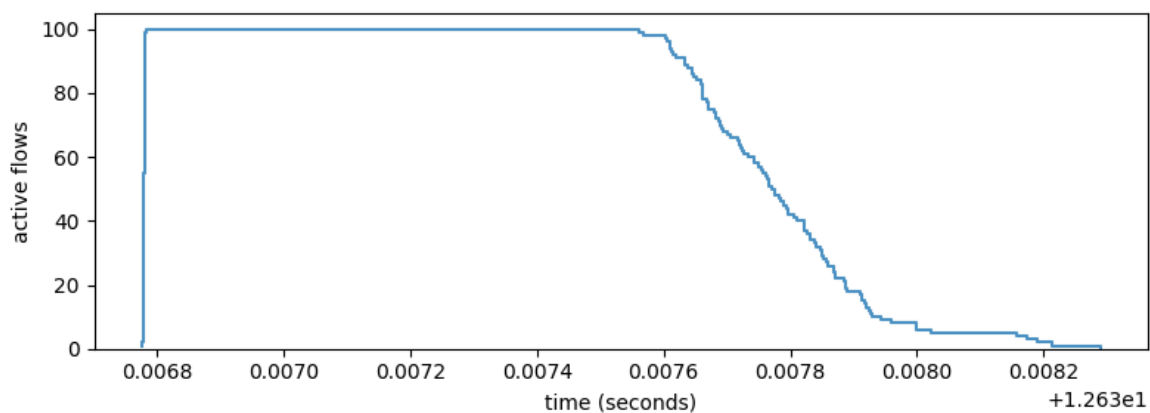
Figure



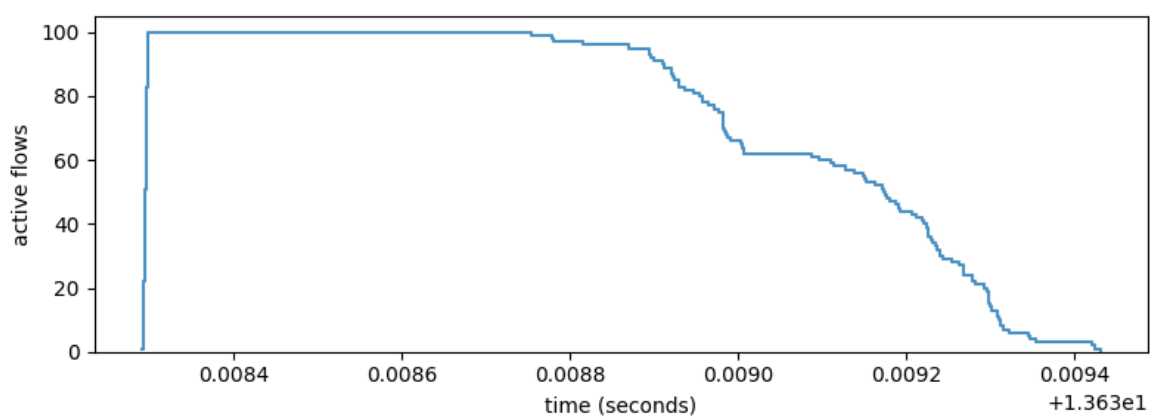
Figure



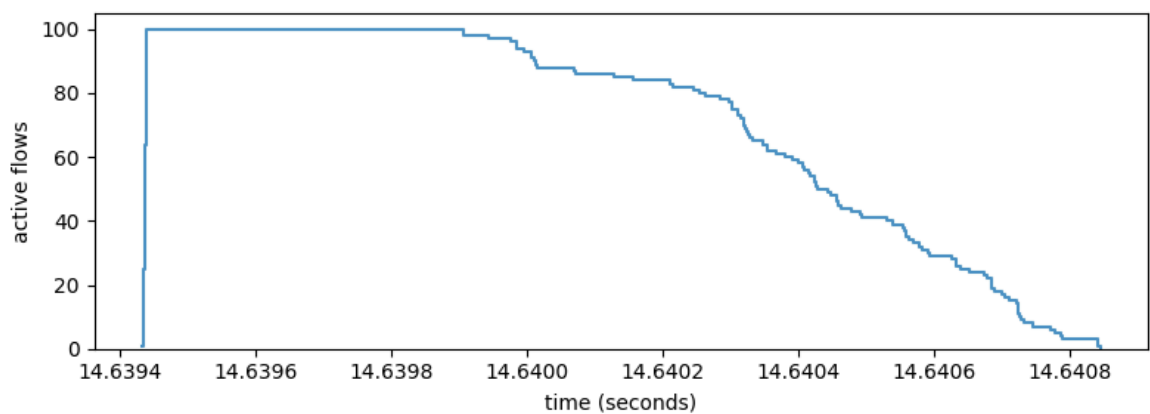
Figure



Figure



Figure



```
In [10]: def graph_cdf_of_flow_duration(
sender_to_flow_times_by_burst, num_bursts, graph_dir, prefix
):
    for burst_idx in range(num_bursts):
        # if burst_idx != num_bursts - 1:
        #     continue

        fig, axes = get_axes(width=3)
        ax = axes[0]
```

```

times = [
    flow_times_by_burst[burst_idx]
    for flow_times_by_burst in sender_to_flow_times_by_burst.values()
]
durations = [(end - start) * 1e3 for start, _, end, _ in times]
print(f"min: {min(durations)}, max: {max(durations)}")

count, bins_count = np.histogram(durations, bins=len(durations))
ax.plot(
    bins_count[1:], np.cumsum(count / sum(count)), linewidth=LINESIZE
)

# ax.set_title(f"CDF of flow duration: Burst {burst_idx + 1} of {num
ax.set_xlabel("FCT (ms)", fontsize=FONTSIZE)
ax.set_ylabel("CDF", fontsize=FONTSIZE)
ax.tick_params(axis="x", labelsize=FONTSIZE)
ax.tick_params(axis="y", labelsize=FONTSIZE)
ax.set_xticks([5, 10, 15], [5, 10, 15])
ax.set_xlim(left=0)
ax.set_ylim(bottom=0, top=1)

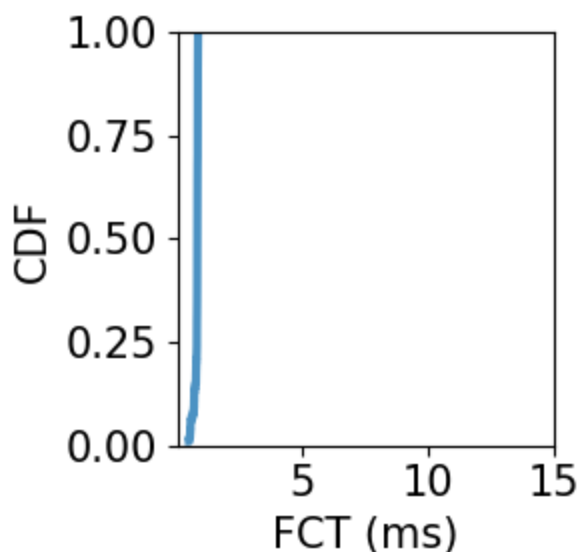
show(fig)
save(graph_dir, prefix, suffix=f"flow_duration_cdf_{burst_idx}")

if RUN:
    graph_cdf_of_flow_duration(
        SENDER_TO_FLOW_TIMES_BY_BURST, NUM_BURSTS, GRAPH_DIR, EXP
    )

```

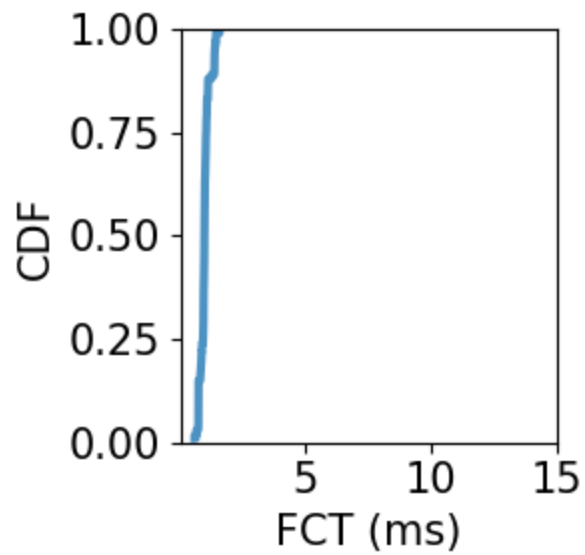
min: 0.4762189999998334, max: 0.83827200000001953

Figure



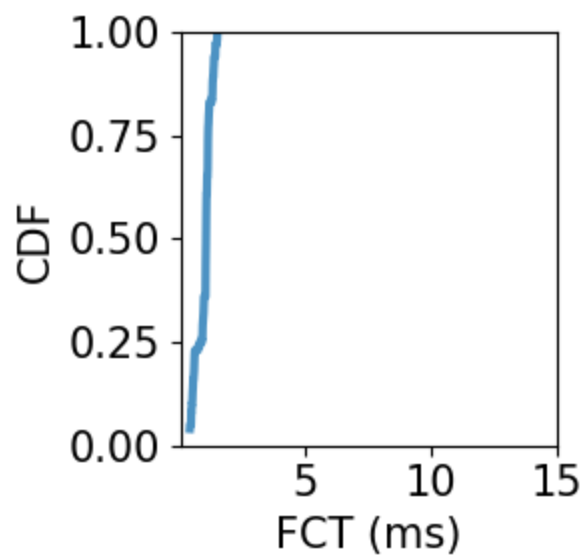
min: 0.5661439999999907, max: 1.5795999999999033

Figure



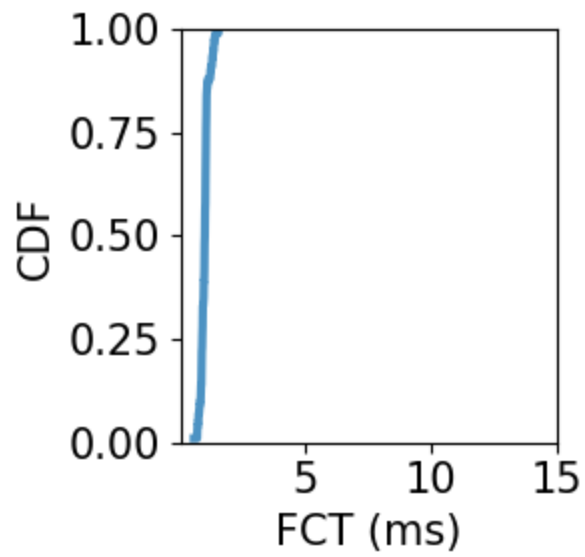
min: 0.3793889999998967, max: 1.4896129999995011

Figure



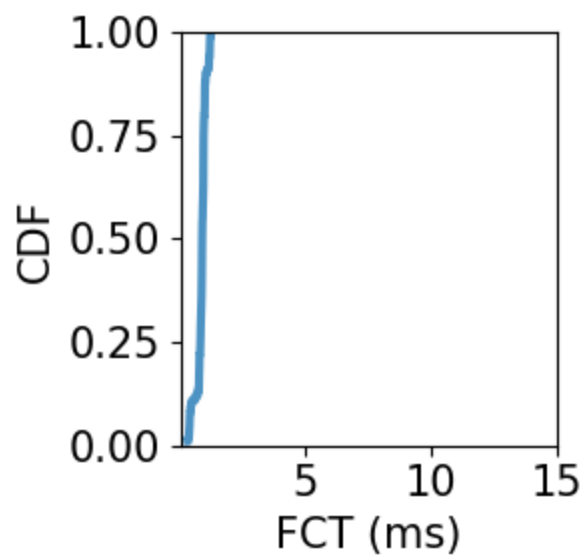
min: 0.53941000000000179, max: 1.5356799999999282

Figure



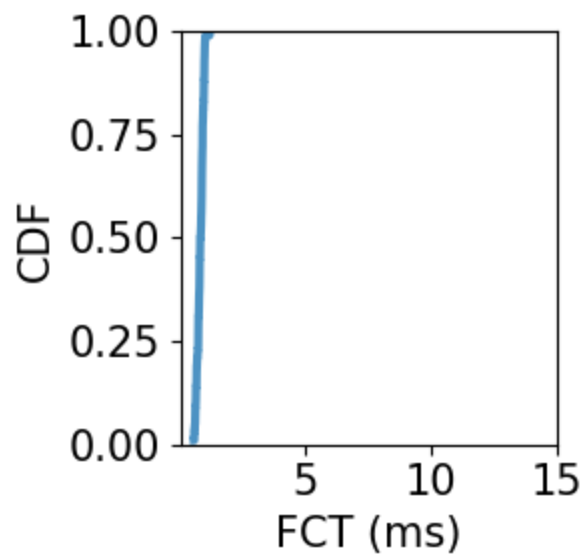
min: 0.30583299999964453, max: 1.2318840000000072

Figure



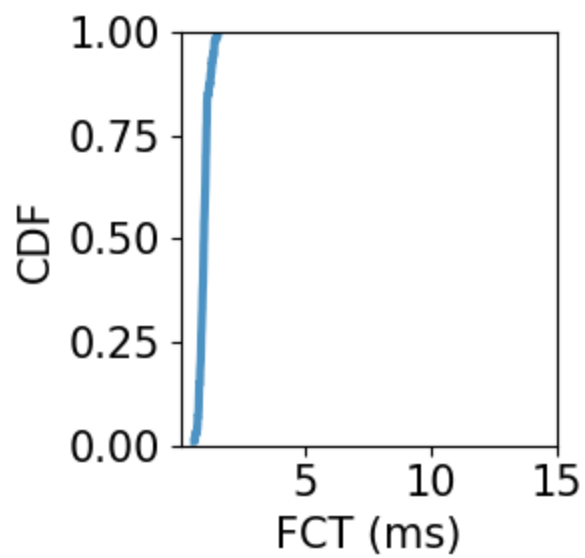
min: 0.5488989999999916, max: 1.187814999999759

Figure



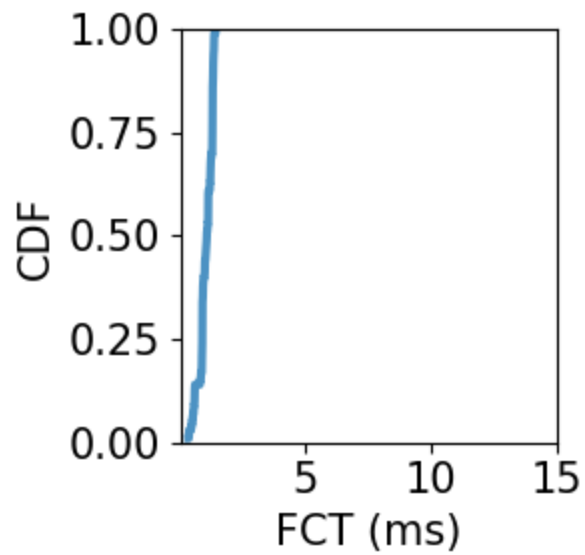
min: 0.5618309999988469, max: 1.5130130000000285

Figure



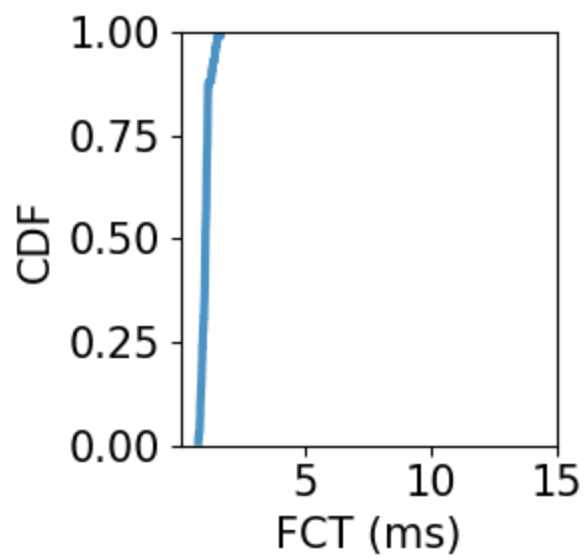
min: 0.32308800000000269, max: 1.42293400000007085

Figure



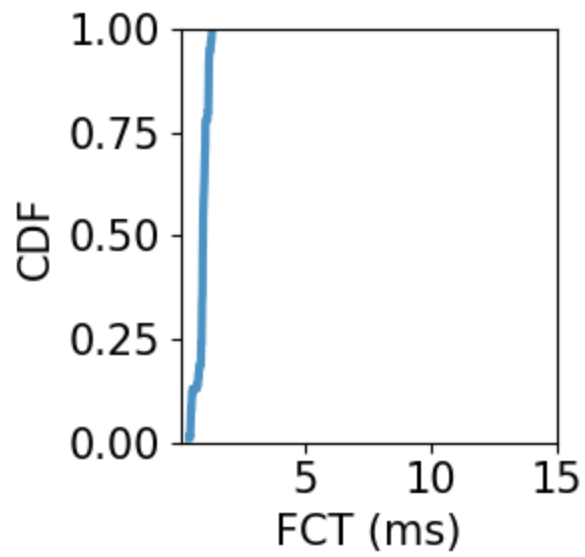
min: 0.7180260000012595, max: 1.6318430000001882

Figure



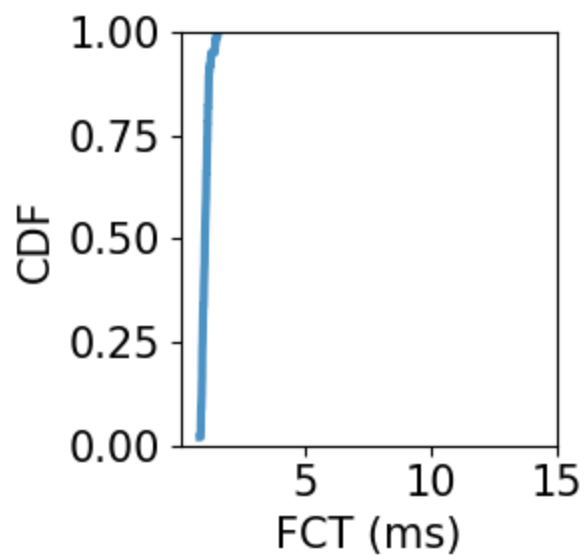
min: 0.36432399999952736, max: 1.2770139999993546

Figure



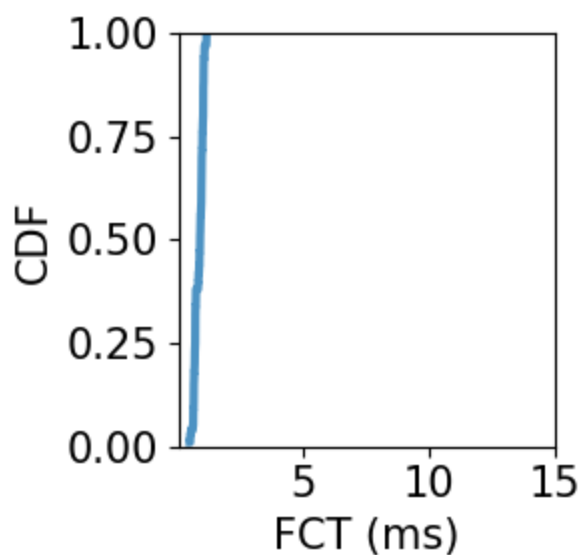
min: 0.7830559999995046, max: 1.5091180000013082

Figure



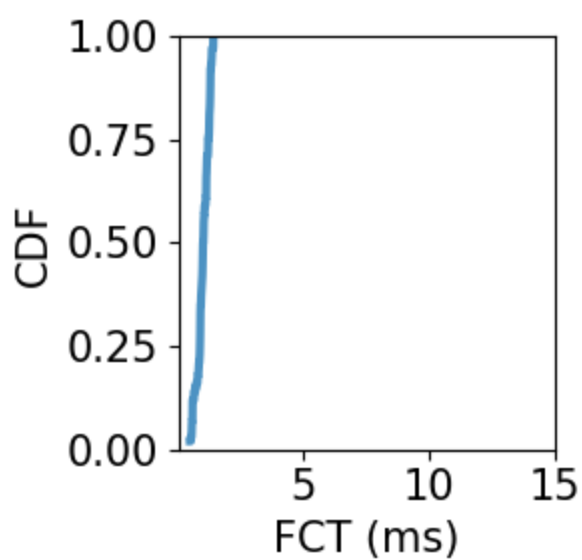
min: 0.46060300000050347, max: 1.135852000000881

Figure



min: 0.4671090000005762, max: 1.4094710000005506

Figure



```
In [11]: len(SENDER_TO_FLOW_TIMES_BY_BURST)
```

```
Out[11]: 100
```

```
In [12]: def parse_cwnd_line(line):  
    parts = line.strip().split(" ")  
    assert len(parts) == 2  
    time_sec, cwnd_bytes = parts  
    time_sec = float(time_sec)  
    cwnd_bytes = int(cwnd_bytes)  
    return time_sec, cwnd_bytes  
  
def parse_cwnds(flp):
```

```

with open(flp, "r", encoding="utf-8") as fil:
    return [parse_cwnd_line(line) for line in fil if line.strip()[0] != '\n']

def parse_sender(flp):
    return int(path.basename(flp).split("_")[0][6:])

def get_sender_to_cwnds_by_burst(
    exp_dir, burst_times, sender_to_flow_times_by_burst, suffix="cwnd"
):
    return {
        parse_sender(flp): separate_samples_into_bursts(
            # Read all CWND samples for this sender
            parse_cwnds(flp),
            burst_times,
            # Look up the start and end time for this sender
            sender_to_flow_times_by_burst[parse_sender(flp)],
            filter_on_flow_times=True,
            bookend=True,
        )
        for flp in [
            path.join(exp_dir, "logs", fln)
            for fln in os.listdir(path.join(exp_dir, "logs"))
            if fln.startswith("sender") and fln.endswith(f"_{suffix}.log")
        ]
    }

def graph_sender_cwnd(
    sender_to_cwnds_by_burst, num_bursts, graph_dir, prefix, ylabel="CWND (bytes)"
):
    for burst_idx in range(num_bursts):
        fig, axes = get_axes()
        ax = axes[0]

        # ax.set_title(
        #     f"CWND of active flows: Burst {burst_idx + 1} of {num_bursts}"
        # )
        ax.set_xlabel("time (seconds)")
        ax.set_ylabel(ylabel)

        for sender, bursts in sender_to_cwnds_by_burst.items():
            if not bursts[burst_idx]:
                continue
            xs, ys = zip(*bursts[burst_idx])
            ax.plot(xs, ys, label=sender, drawstyle="steps-post", alpha=0.8)

        ax.set_ylim(bottom=0)

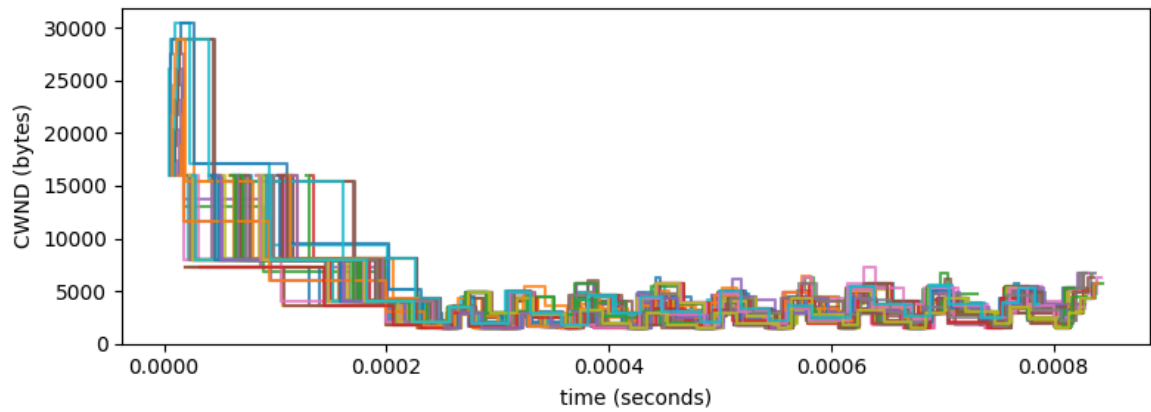
        show(fig)
        save(graph_dir, prefix, suffix=f"cwnd_{burst_idx}")

if RUN:
    SENDER_TO_CWNS_BY_BURST = get_sender_to_cwnds_by_burst(

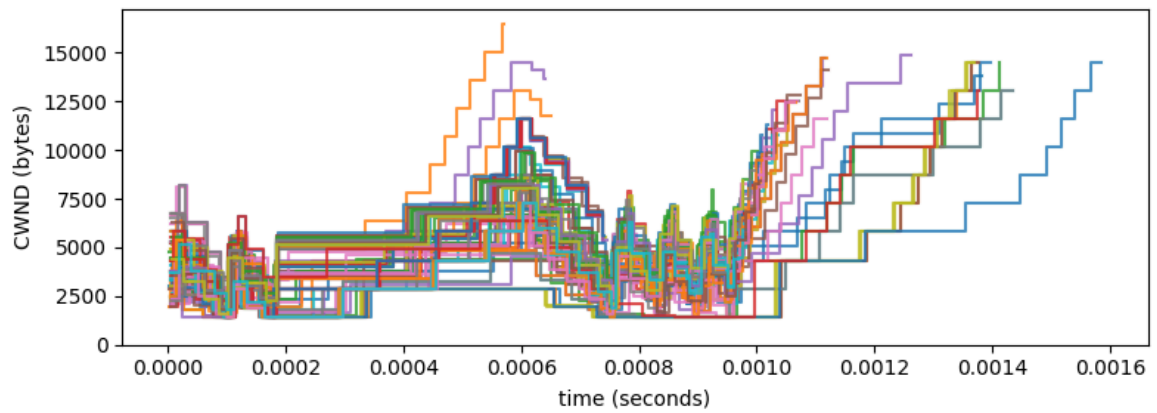
```

```
EXP_DIR, BURST_TIMES, SENDER_TO_FLOW_TIMES_BY_BURST  
)  
graph_sender_cwnd(SENDER_TO_CWNS_BY_BURST, NUM_BURSTS, GRAPH_DIR, EXP)
```

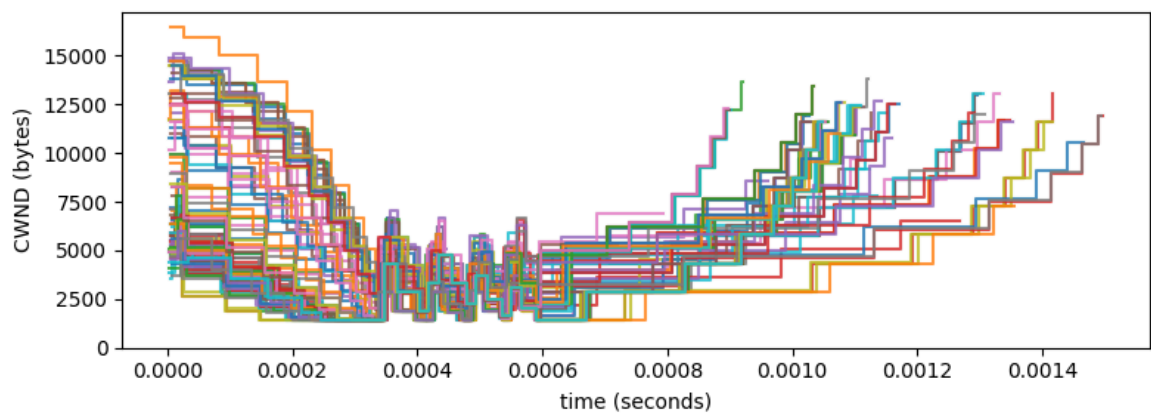
Figure



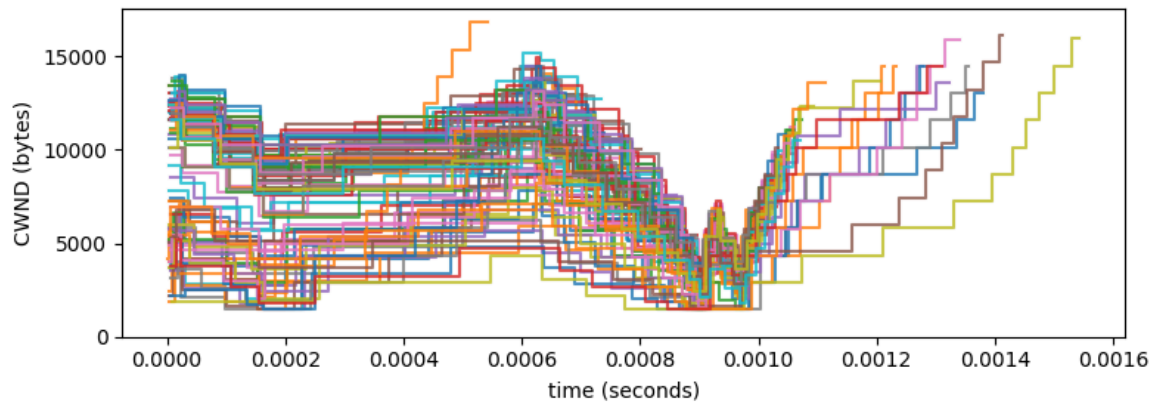
Figure



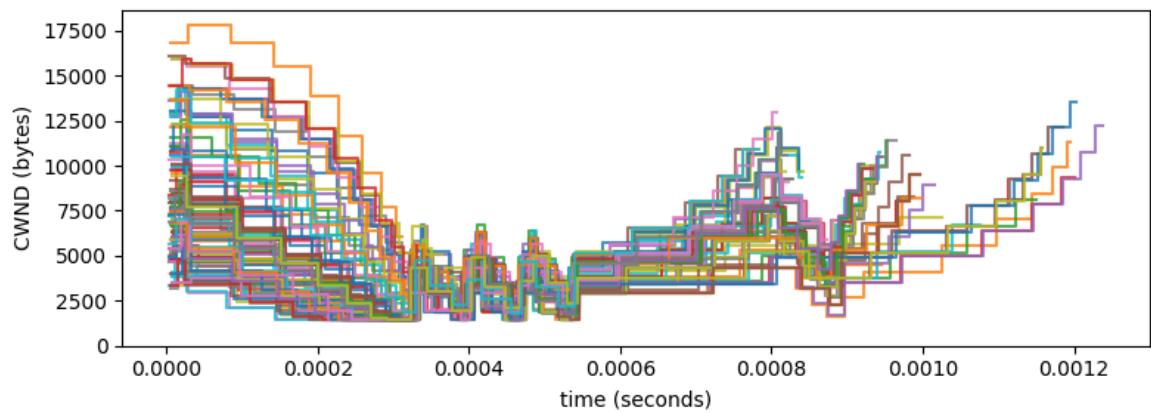
Figure



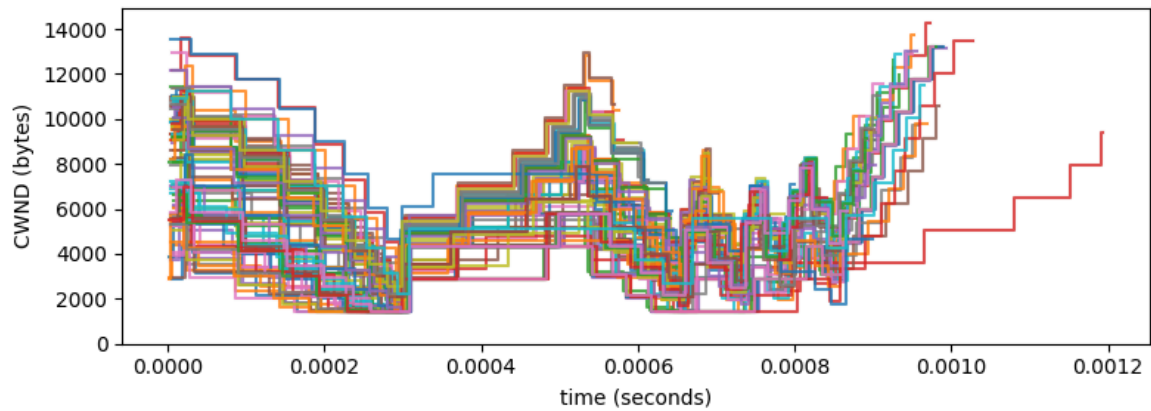
Figure



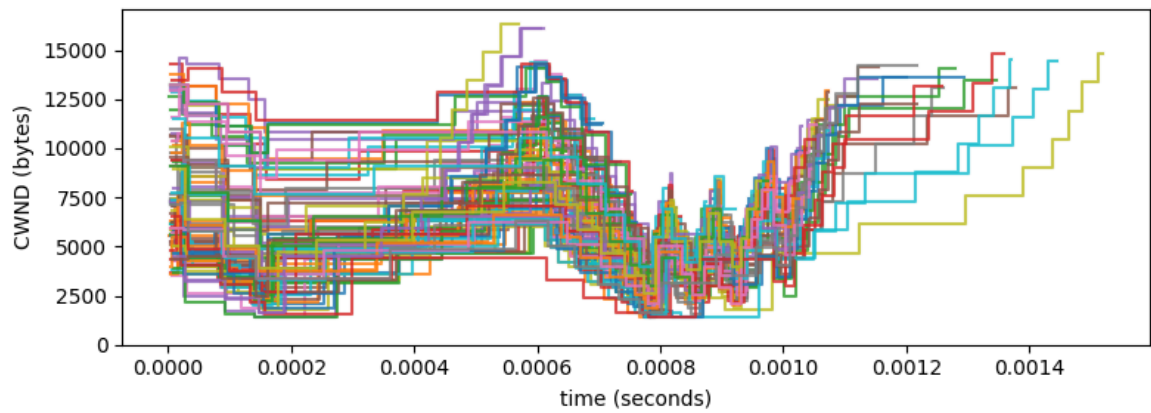
Figure



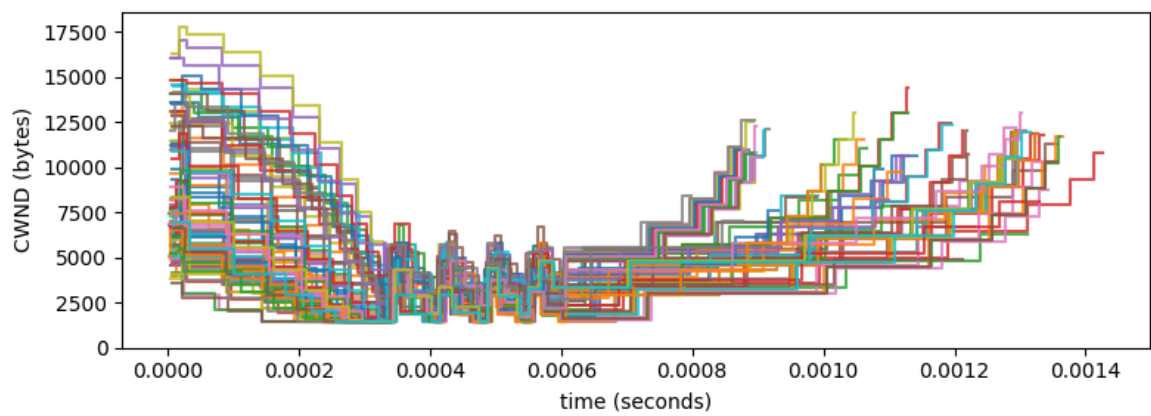
Figure



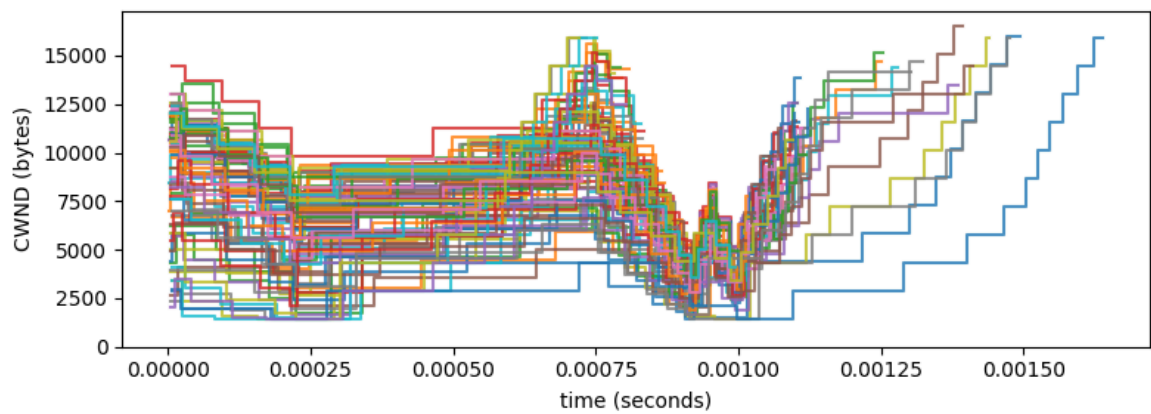
Figure



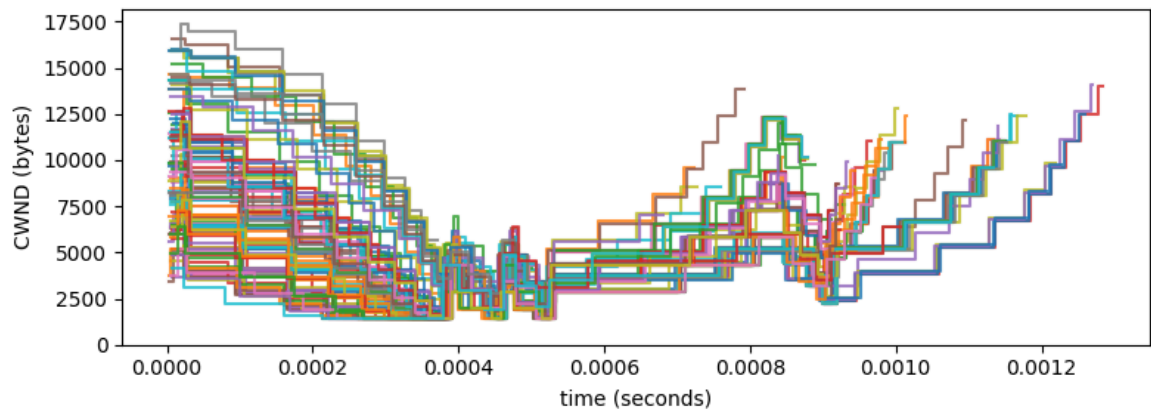
Figure



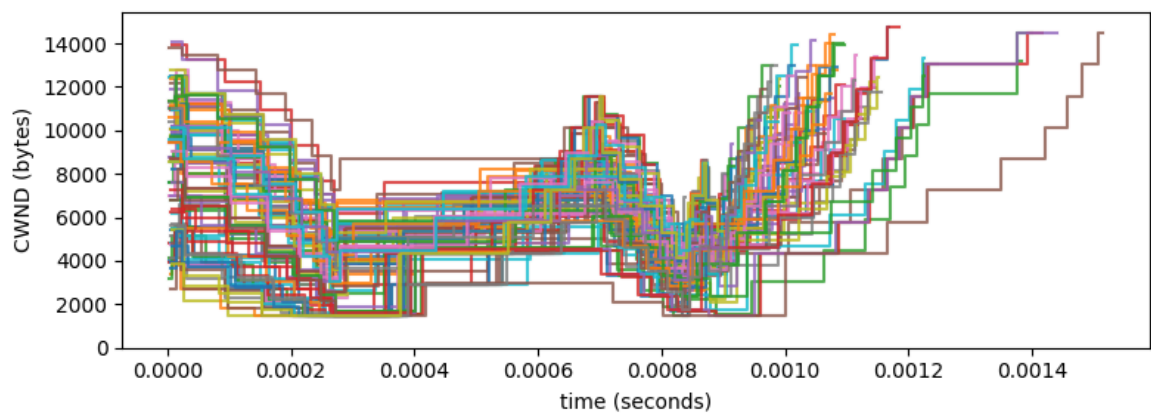
Figure



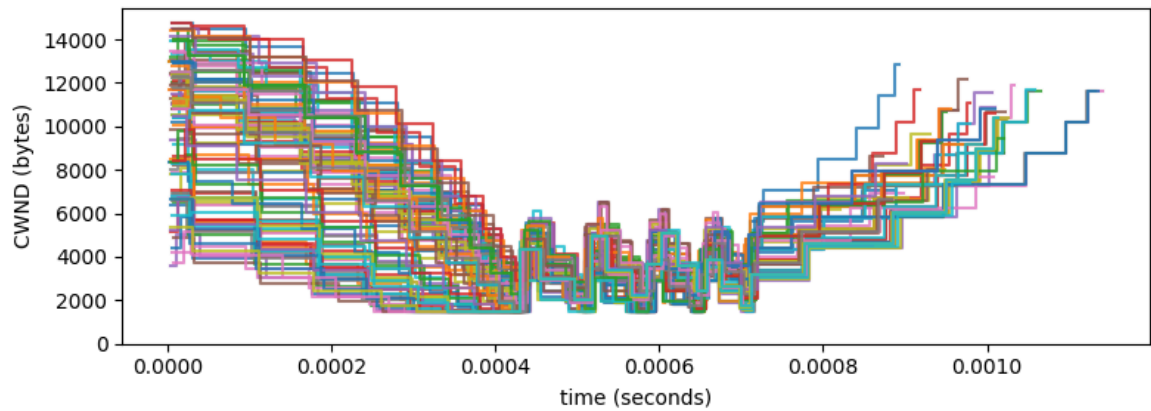
Figure



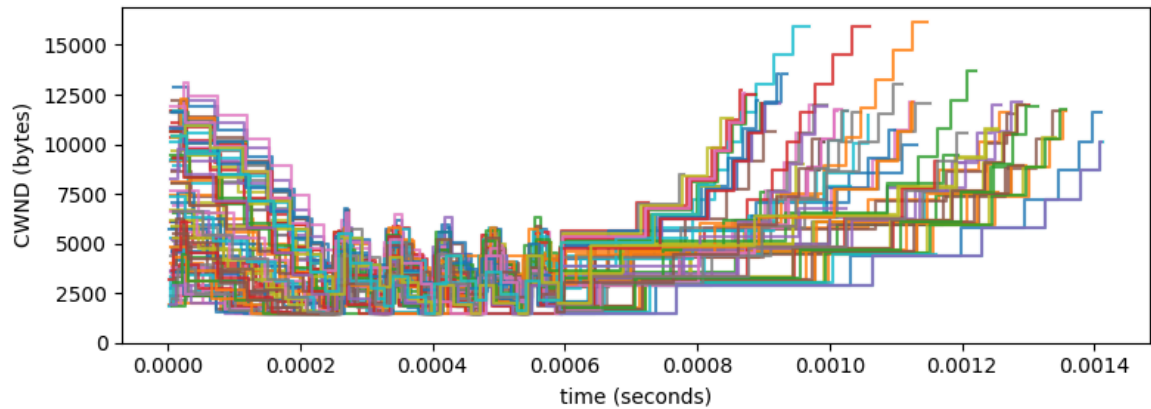
Figure



Figure



Figure

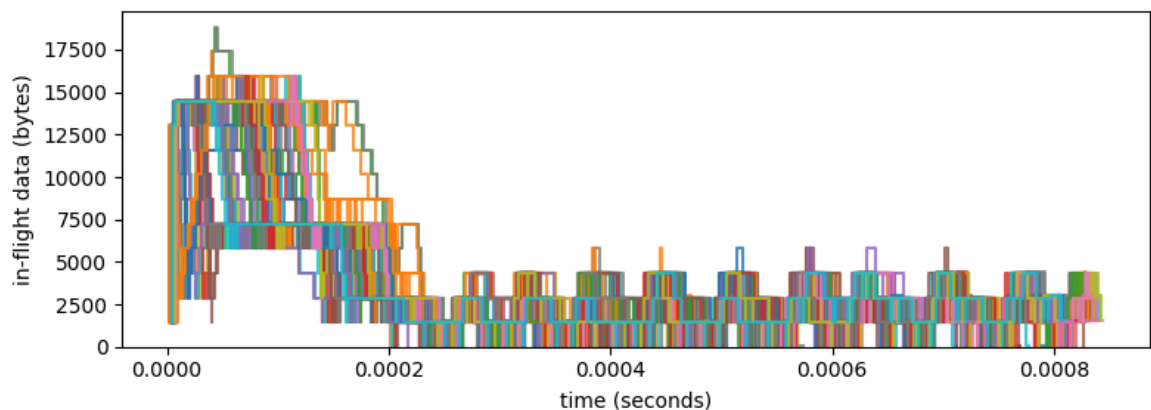


```
In [13]: def get_sender_to_inflight_by_burst(
exp_dir, burst_times, sender_to_flow_times_by_burst
):
    return get_sender_to_cwnds_by_burst(
        exp_dir, burst_times, sender_to_flow_times_by_burst, suffix="bytes_i
    )

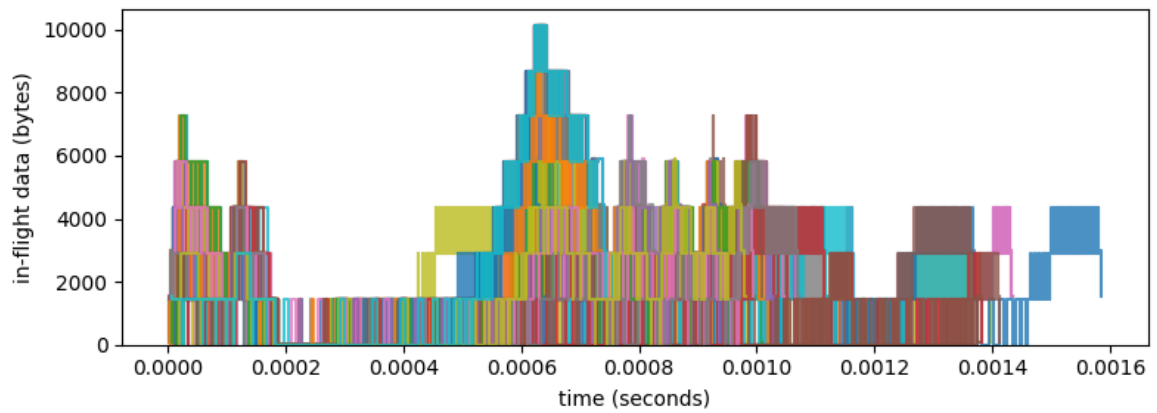
def graph_sender_inflight(sender_to_inflight_by_burst, num_bursts, graph_dir
graph_sender_cwnd(
    sender_to_inflight_by_burst,
    num_bursts,
    graph_dir,
    prefix,
    ylabel="in-flight data (bytes)",
)

if RUN:
    SENDER_TO_INFLIGHT_BY_BURST = get_sender_to_inflight_by_burst(
        EXP_DIR, BURST_TIMES, SENDER_TO_FLOW_TIMES_BY_BURST
    )
    graph_sender_inflight(SENDER_TO_INFLIGHT_BY_BURST, NUM_BURSTS, GRAPH_DIR
```

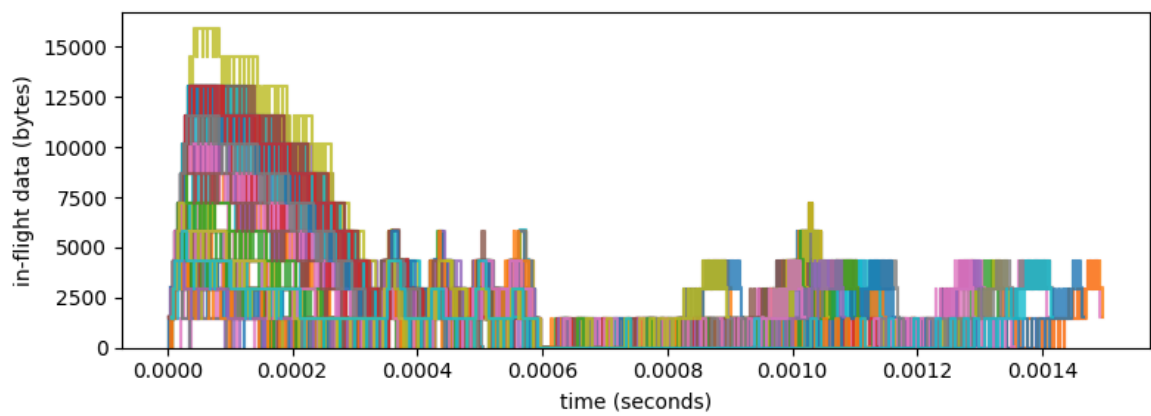
Figure



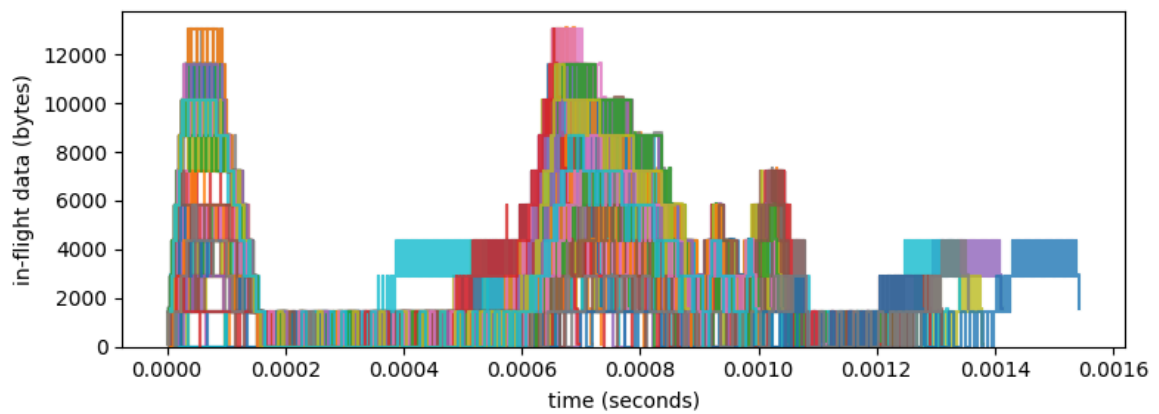
Figure



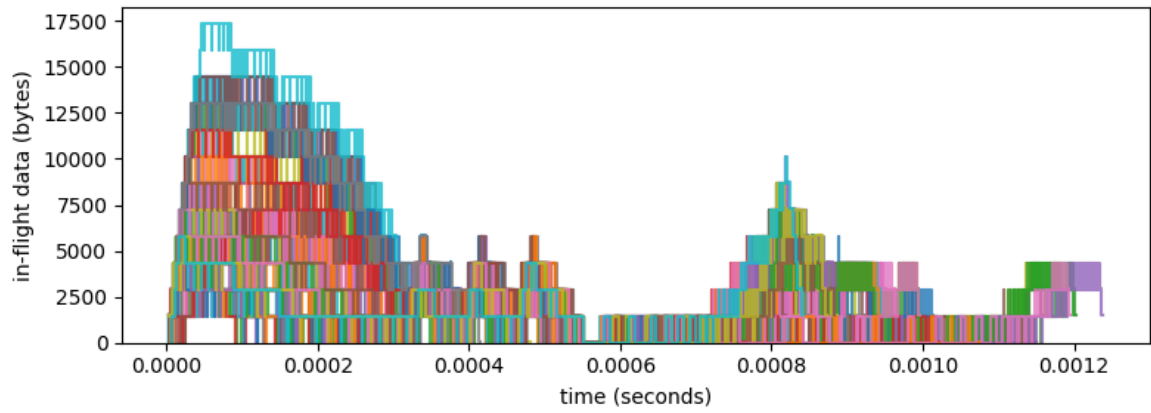
Figure



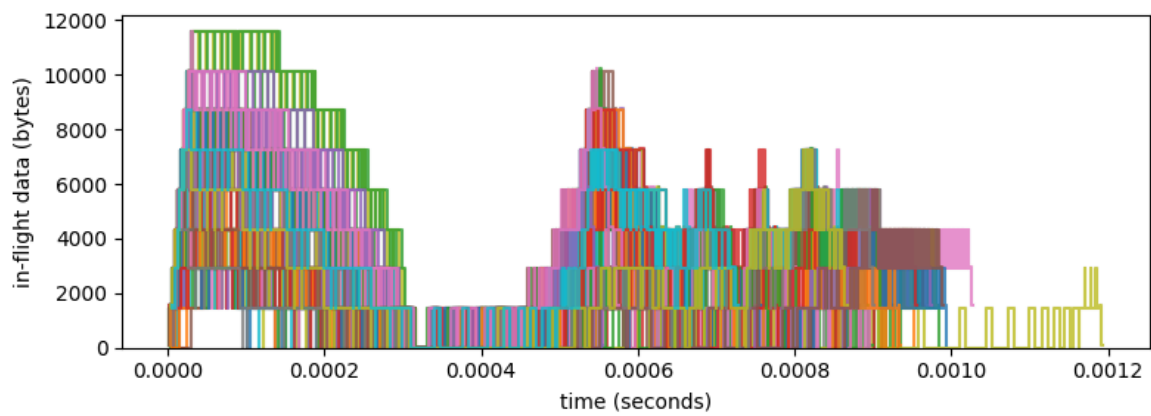
Figure



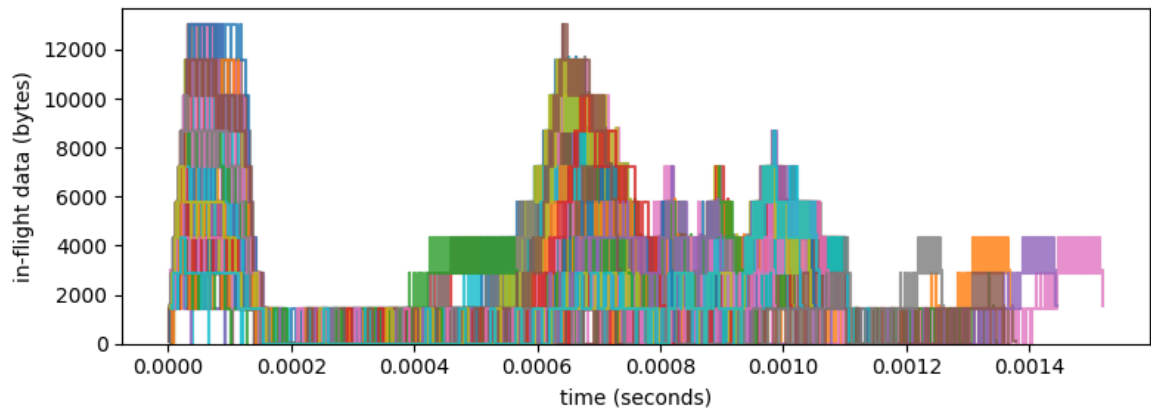
Figure



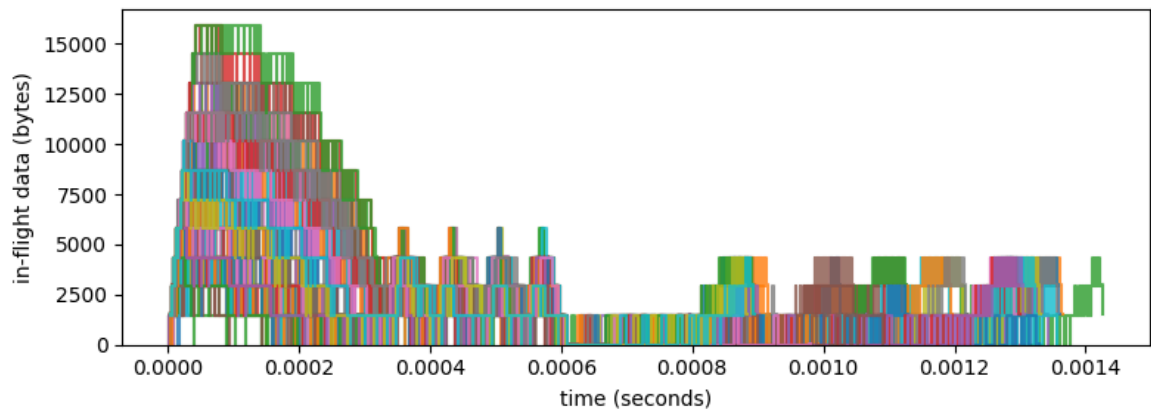
Figure



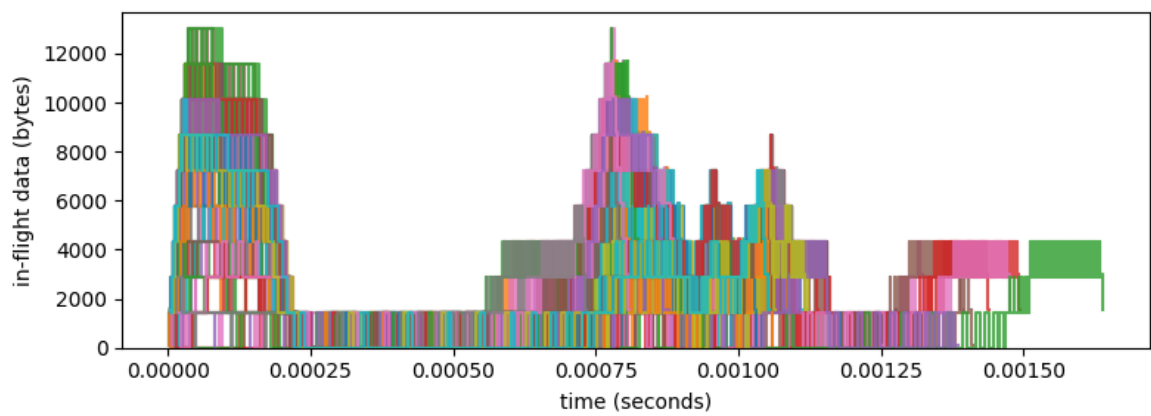
Figure



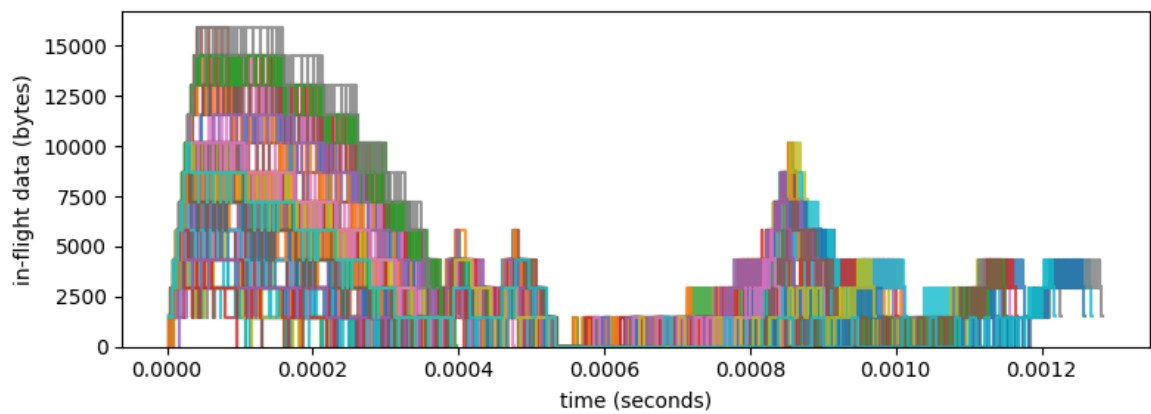
Figure



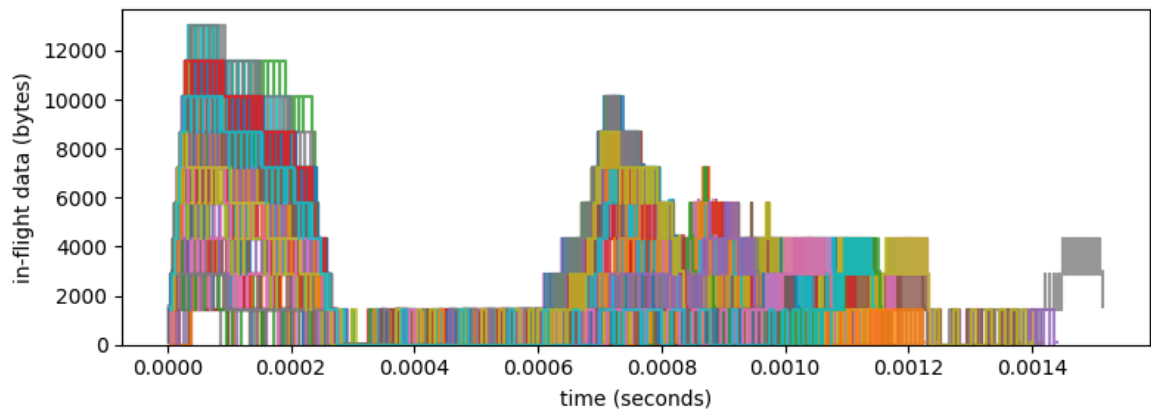
Figure



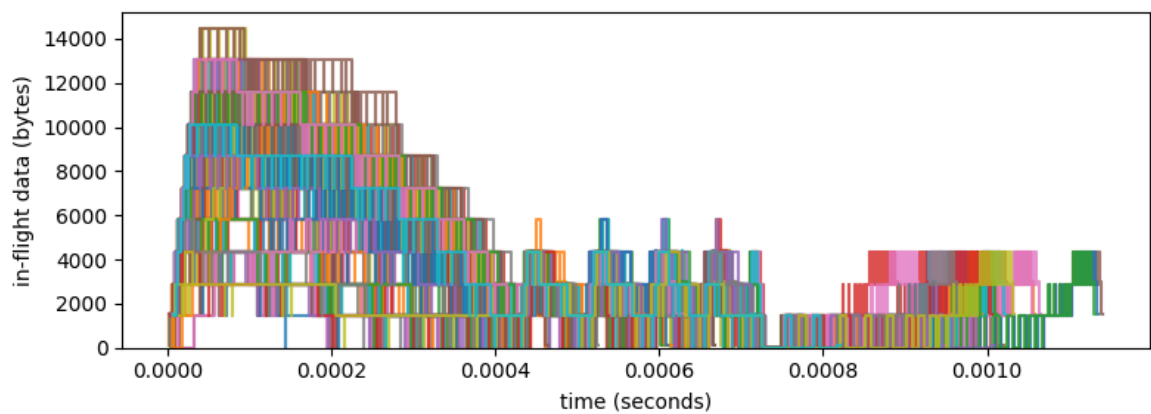
Figure



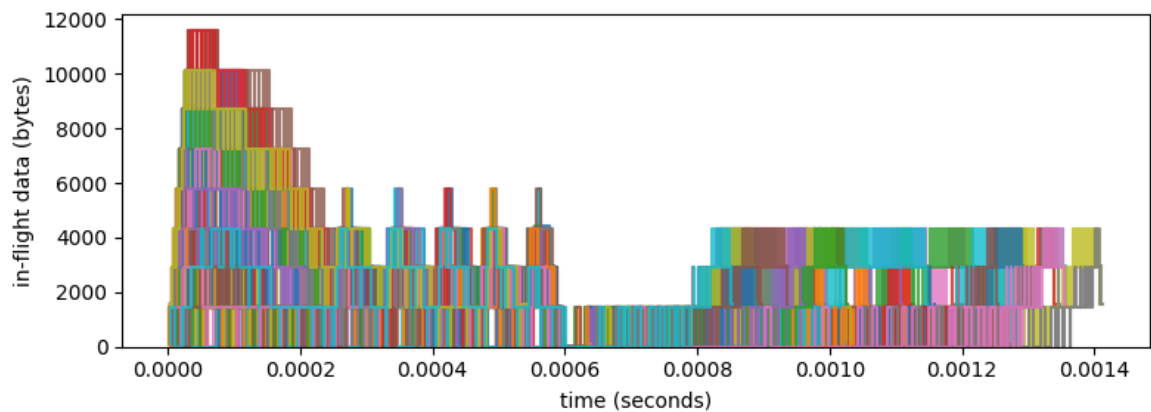
Figure



Figure



Figure



```
In [14]: # Inspired by https://stackoverflow.com/questions/10058227/calculating-mean-
def tolerant_metrics(xs, arrays, percentiles):
    # Map x value to index. Used to quickly determine where each array start
    # relative to the overall xs.
    xs_map = {
        # Do not need to do round(x, decimal_places) because xs are already
        # intervals of 1 / interp_delta.
        x: idx
        for idx, x in enumerate(xs)
    }
```

```

# Create 2d array to fit the largest array
combined_2d = np.ma.empty((len(xs), len(arrays)))
combined_2d.mask = True

for idx, array in enumerate(arrays):
    # Look up this array's start position
    # Do not need to do round(x, decimal_places) because xs are already
    # intervals of 1 / interp_delta.
    start_idx = xs_map[array[0][0]]

    source = array[: (len(combined_2d) - start_idx)]
    if len(array) != len(source):
        print(f"Warning: Dropping {len(array) - len(source)} samples!")

    # Verify alignment.
    assert xs[start_idx] == source[0][0]
    assert xs[start_idx + len(source) - 1] == source[-1][0]

    # combined_2d[start_idx : start_idx + len(array), idx] = list(zip(*a
    source = list(zip(*source))[1]
    combined_2d[start_idx : start_idx + len(array), idx] = source

return (
    combined_2d.mean(axis=-1),
    combined_2d.std(axis=-1),
    combined_2d.min(axis=-1),
    combined_2d.max(axis=-1),
    np.nanpercentile(
        np.ma.filled(np.ma.masked_where(combined_2d < 0, combined_2d), r
        percentiles,
        axis=-1,
    ),
    combined_2d.sum(axis=-1),
)

def step_interp(old_xs, old_ys, new_xs):
    # Lengths must be nonzero and agree.
    assert len(old_xs) > 0
    assert len(old_ys) > 0
    assert len(new_xs) > 0
    assert len(old_xs) == len(old_ys)
    # xs must be strictly non-decreasing.
    assert (np.diff(old_xs) >= 0).all(), np.diff(old_xs)
    assert (np.diff(new_xs) >= 0).all()
    # This is strictly interpolation, not extrapolation.
    assert new_xs[0] >= old_xs[0]
    assert new_xs[-1] <= old_xs[-1]

    new_ys = np.empty(len(new_xs))
    # Points to the next value in xs and ys that is past the current x we ar
    # interpolating.
    old_idx = 0
    for new_idx, new_x in enumerate(new_xs):
        # Move old_idx forward until it is at a position where the next elem

```

```

        # in old_xs is strictly greater than new_x.
        #
        # old_idx will never grow larger than len(old_xs) - 2
        while old_idx < len(old_xs) - 2 and new_x >= old_xs[old_idx + 1]:
            old_idx += 1

        # If old_idx is immediately before the last element in old_xs, then
        # check manually if we need to advance old_idx to the last element i
        # old_xs.
        if old_idx == len(old_xs) - 2:
            if new_x >= old_xs[len(old_xs) - 1]:
                old_idx += 1

        new_ys[new_idx] = old_ys[old_idx]

    assert len(new_xs) == len(new_ys)
    return new_ys

def interpolate_flows_for_burst(
    sender_to_x_by_burst, sender_to_x_by_burst_interp, burst_idx, interp_delta
):
    # Interpolate each flow at uniform intervals.
    for sender, bursts in sender_to_x_by_burst.items():
        if bursts[burst_idx]:
            assert len(bursts[burst_idx]) > 0
            new_xs = get_aligned_xs(
                bursts[burst_idx][0][0], bursts[burst_idx][-1][0], interp_delta
            )
            if len(new_xs) == 0:
                new_ys = np.empty(0)
            else:
                new_ys = step_interp(*zip(*bursts[burst_idx]), new_xs)
        else:
            new_xs = np.empty(0)
            new_ys = np.empty(0)
        sender_to_x_by_burst_interp[sender].append(list(zip(new_xs, new_ys)))

def get_sender_to_x_by_burst_interp(sender_to_x_by_burst, num_bursts, interp_delta):
    sender_to_x_by_burst_interp = collections.defaultdict(list)
    for burst_idx in range(num_bursts):
        interpolate_flows_for_burst(
            sender_to_x_by_burst,
            sender_to_x_by_burst_interp,
            burst_idx,
            interp_delta,
        )
    for bursts_interp in sender_to_x_by_burst_interp.values():
        assert len(bursts_interp) == num_bursts
    return sender_to_x_by_burst_interp

def get_metrics(
    sender_to_x_by_burst_interp,
    burst_idx,

```

```

    interp_delta,
    percentiles,
):
    # Extract the desired burst from each sender.
    # Throw away senders that do not have any samples for this burst.
    valid_senders = [
        bursts[burst_idx]
        for bursts in sender_to_x_by_burst_interp.values()
        if bursts[burst_idx]
    ]
    if len(valid_senders) != len(sender_to_x_by_burst_interp):
        print(
            f"Warning: Burst {burst_idx} has only "
            f"{len(valid_senders)}/{len(sender_to_x_by_burst_interp)} "
            "senders with at least one sample."
        )
    return get_metrics_helper(valid_senders, interp_delta, percentiles)

def get_metrics_helper(
    senders,
    interp_delta,
    percentiles,
):
    # senders is an array of data for burst i, with one element (subarray) for
    # sender:
    #
    #     senders = [
    #         [ samples from burst i for sender 0 ],
    #         ...
    #         [ samples from burst i for sender n - 1 ]
    #     ]

    # Determine the overall x-axis range for this burst, across all valid senders
    # print("max from senders:", max(samples[-1][0] for samples in senders))
    xs = get_aligned_xs(
        min(samples[0][0] for samples in senders),
        max(samples[-1][0] for samples in senders),
        interp_delta,
    )
    # print("max from aligned xs:", xs[-1])
    # print(" ".join(str(z) for z in list(zip(*senders[5]))[0]))

    # Calculate and verify metrics.
    avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, sum_ys = tolerant_metrics(
        xs, senders, percentiles
    )
    assert len(xs) == len(avg_ys)
    assert len(xs) == len(stdev_ys)
    assert len(xs) == len(min_ys)
    assert len(xs) == len(max_ys)
    assert len(xs) == percentiles_ys.shape[1]
    assert len(xs) == len(sum_ys)

    return xs, avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, sum_ys

```

```

def get_metrics_by_burst(
    sender_to_x_by_burst_interp, num_bursts, interp_delta, percentiles
):
    return [
        get_metrics(
            sender_to_x_by_burst_interp,
            burst_idx,
            interp_delta,
            percentiles,
        )
        for burst_idx in range(num_bursts)
    ]

def graph_cwnd_metrics(
    cwnd_metrics_by_burst,
    num_bursts,
    percentiles,
    graph_dir,
    prefix,
    ylabel="CWND (bytes)",
    fln="cwnd_analysis",
):
    for burst_idx in range(num_bursts):
        xs, avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, _ = cwnd_metrics_by_burst[burst_idx]

        # Left graph
        fig, axes = get_axes()
        ax = axes[0]
        ax.fill_between(xs, min_ys, max_ys, alpha=0.25, label="min/max")
        ax.fill_between(
            xs, avg_ys - stdev_ys, avg_ys + stdev_ys, alpha=0.5, label="avg"
        )
        ax.plot(xs, avg_ys, label="avg", alpha=0.8)
        # ax.set_title(
        #     f"CWND of active flows: Burst {burst_idx + 1} of {num_bursts}"
        # )
        ax.set_xlabel("time (seconds)")
        ax.set_ylabel(ylabel)
        ax.set_ylim(bottom=0)
        ax.legend()
        show(fig)
        save(graph_dir, prefix, suffix=f"{fln}_{burst_idx}_0")

        # Right graph
        fig, axes = get_axes()
        ax = axes[0]
        ax.plot(xs, avg_ys, label="avg", alpha=0.8)
        # Bottom of lowest bar is percentiles[0], which is designed to be the
        for idx in range(1, percentiles_ys.shape[0]):
            ax.fill_between(
                xs,
                percentiles_ys[idx - 1],
                percentiles_ys[idx],
            )

```

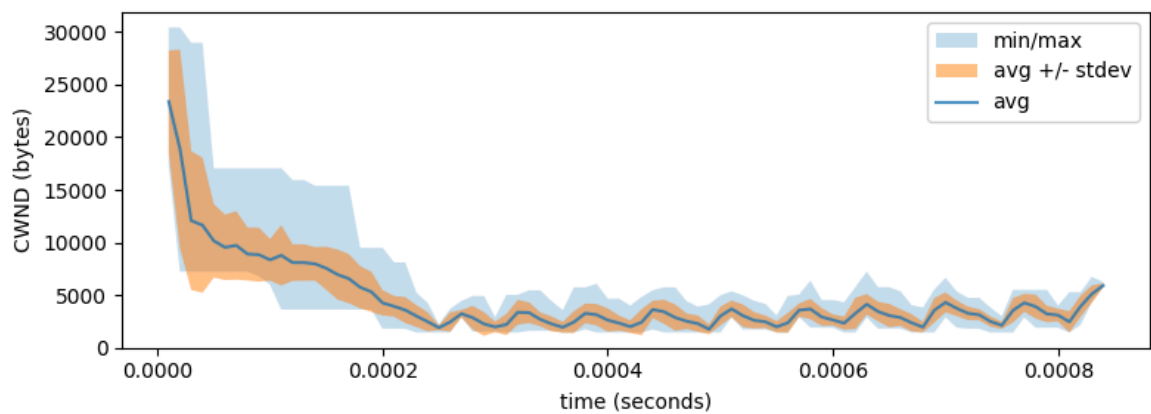
```

        alpha=0.5,
        label=f"p{percentiles[idx]}",
    )
    # ax.set_title(
    #     f"CWND of active flows: Burst {burst_idx + 1} of {num_bursts}"
    # )
    ax.set_xlabel("time (seconds)")
    ax.set_ylabel ylabel
    ax.set_ylim(bottom=0)
    ax.legend()
    show(fig)
    save(graph_dir, prefix, suffix=f"{fln}_{burst_idx}_1")

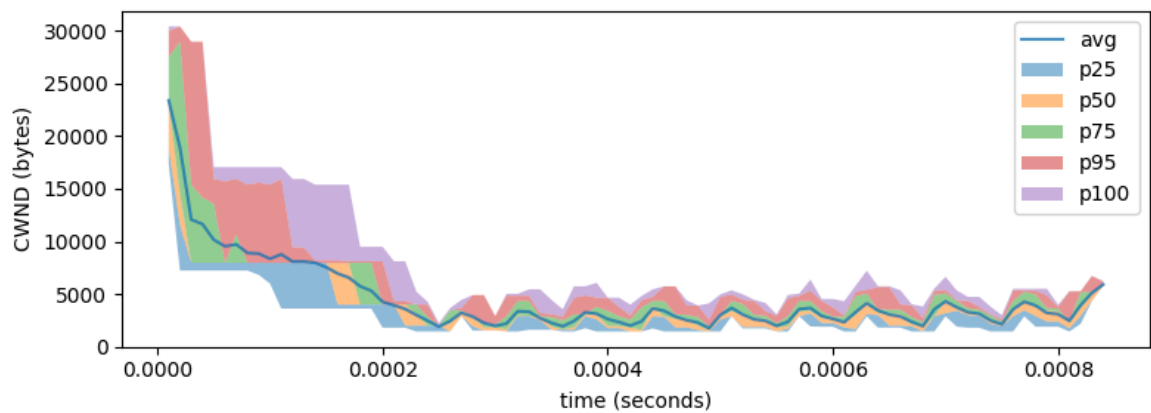
if RUN:
    INTERP_DELTA = 1e5
    PERCENTILES = [0, 25, 50, 75, 95, 100]
    SENDER_TO_CWNS_BY_BURST_INTERP = get_sender_to_x_by_burst_interp(
        SENDER_TO_CWNS_BY_BURST, NUM_BURSTS, INTERP_DELTA
    )
    CWND_METRICS_BY_BURST = get_metrics_by_burst(
        SENDER_TO_CWNS_BY_BURST_INTERP, NUM_BURSTS, INTERP_DELTA, PERCENTIL
    )
    graph_cwnd_metrics(
        CWND_METRICS_BY_BURST,
        NUM_BURSTS,
        PERCENTILES,
        GRAPH_DIR,
        EXP,
    )

```

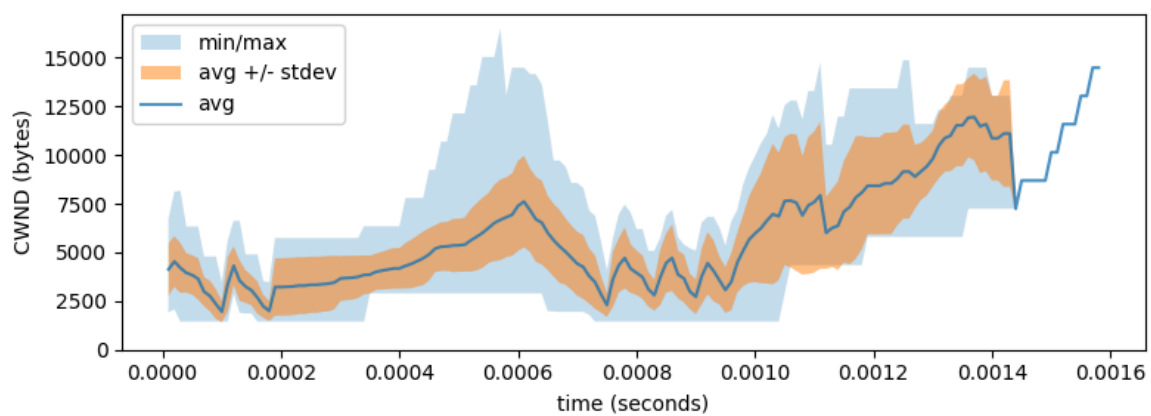
Figure



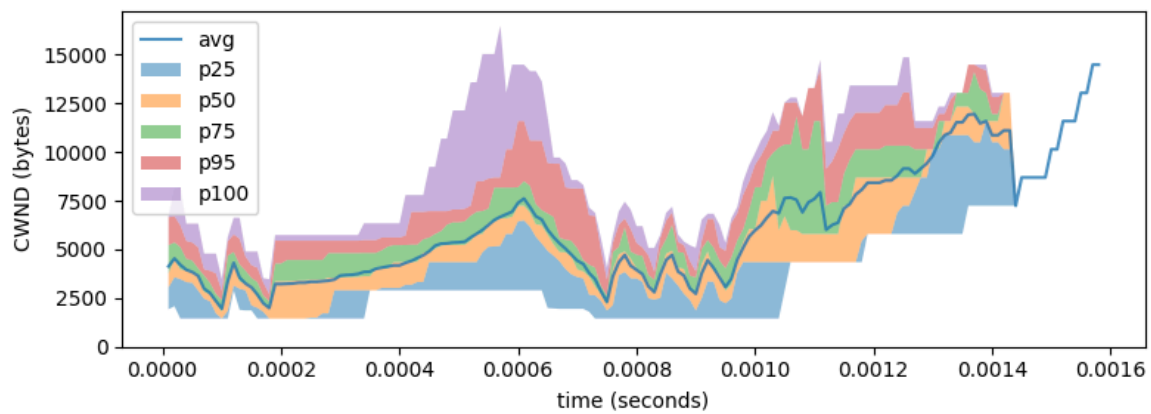
Figure



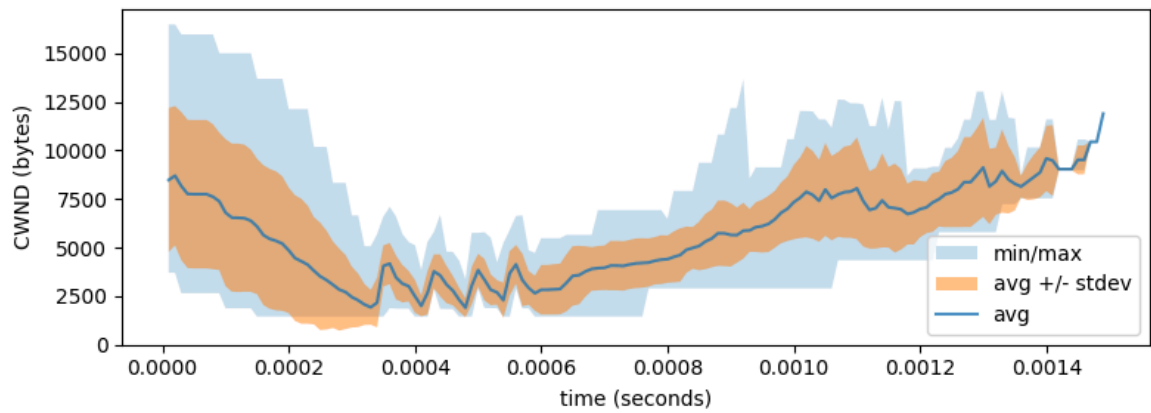
Figure



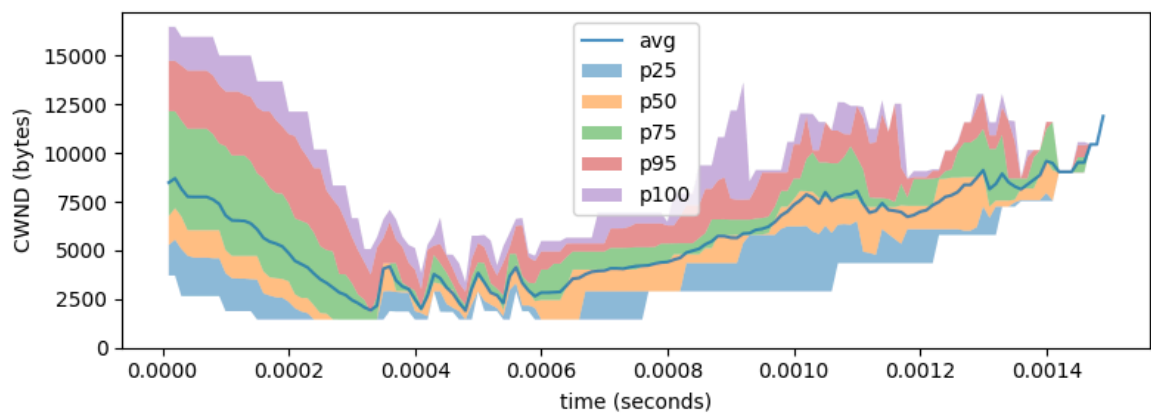
Figure



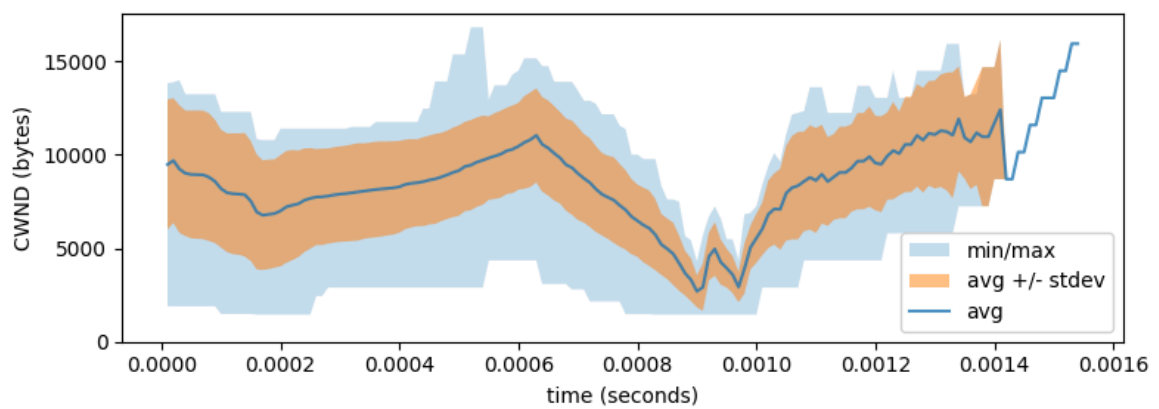
Figure



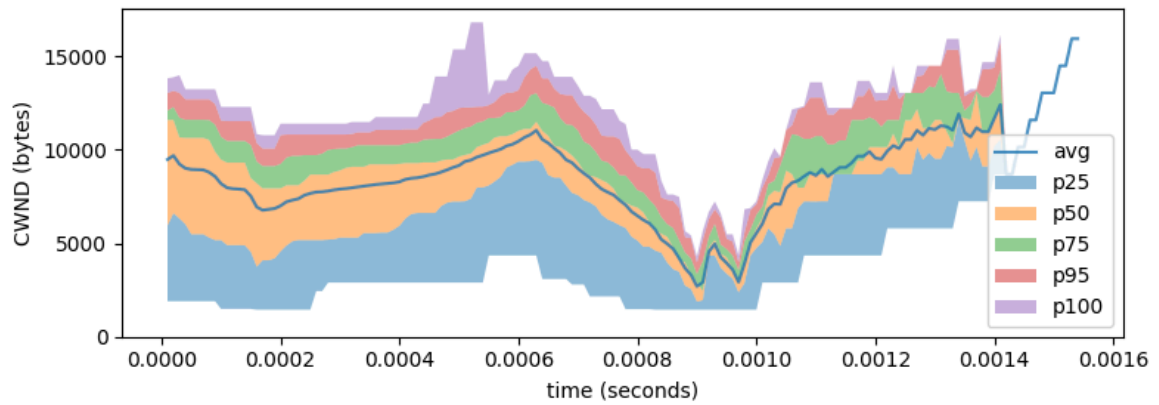
Figure



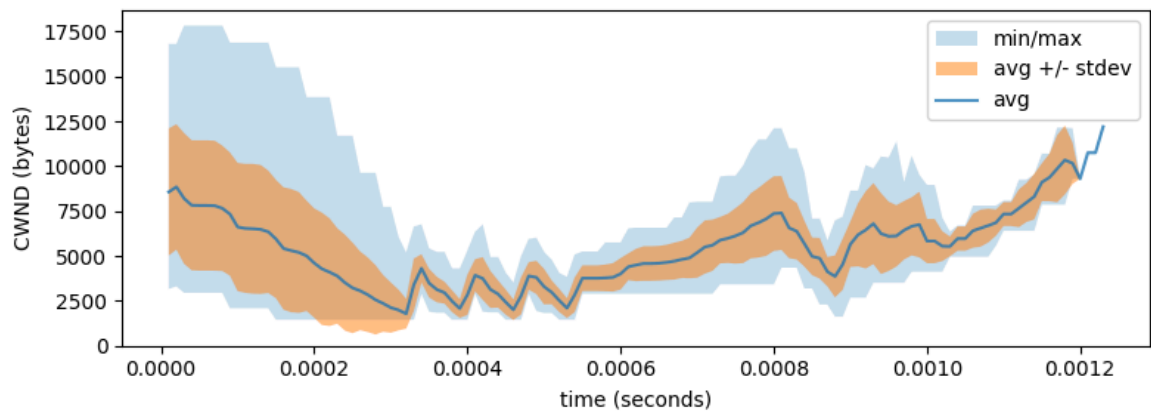
Figure



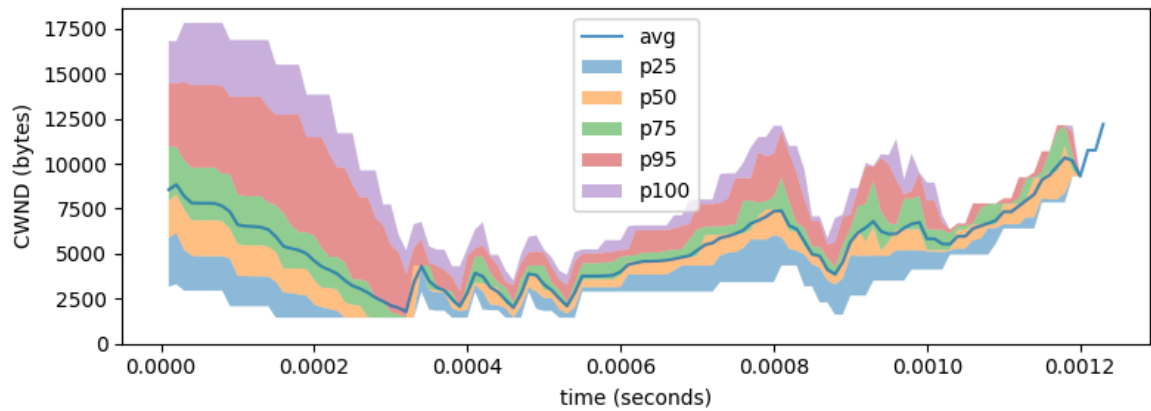
Figure



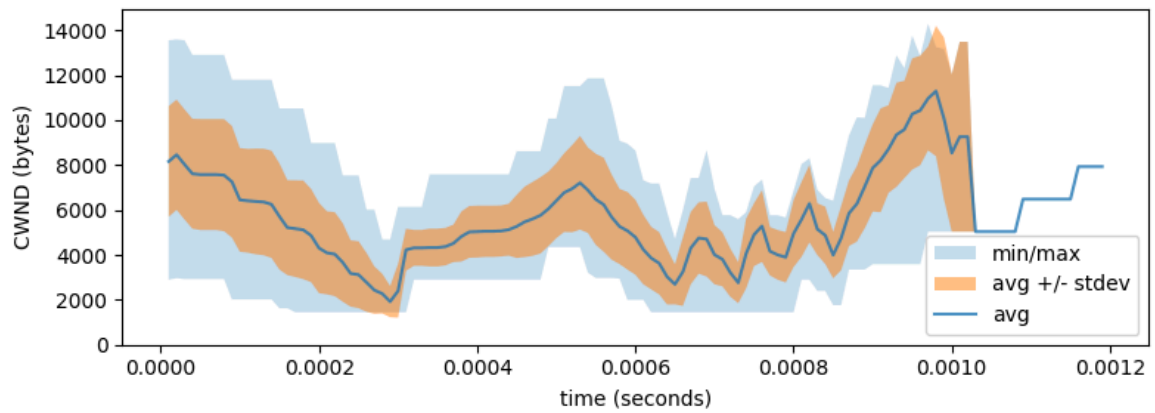
Figure



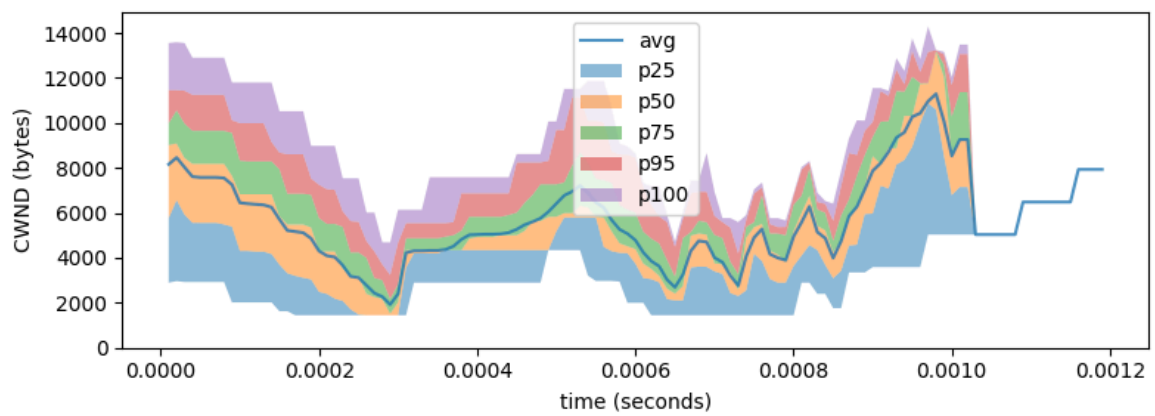
Figure



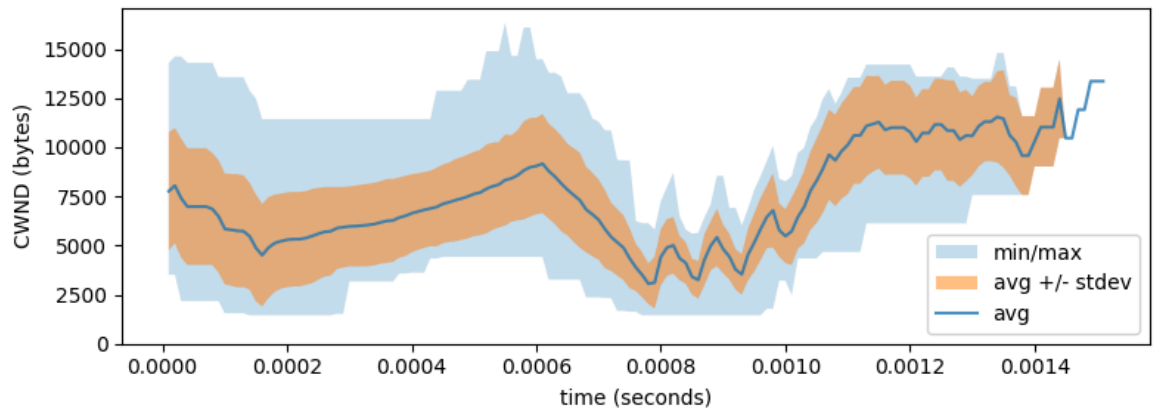
Figure



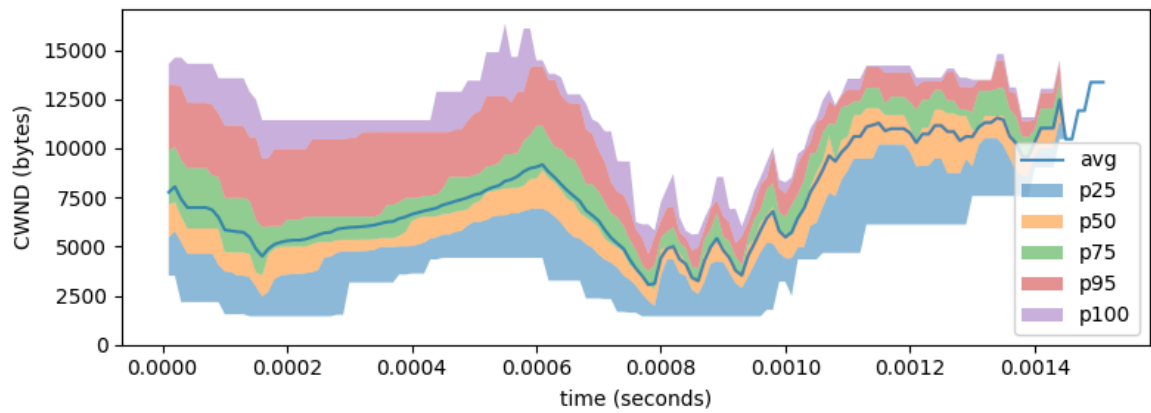
Figure



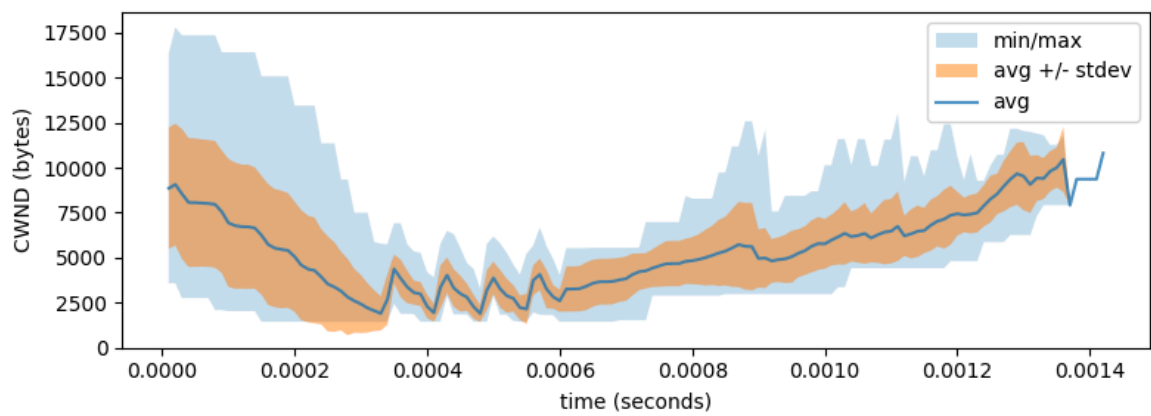
Figure



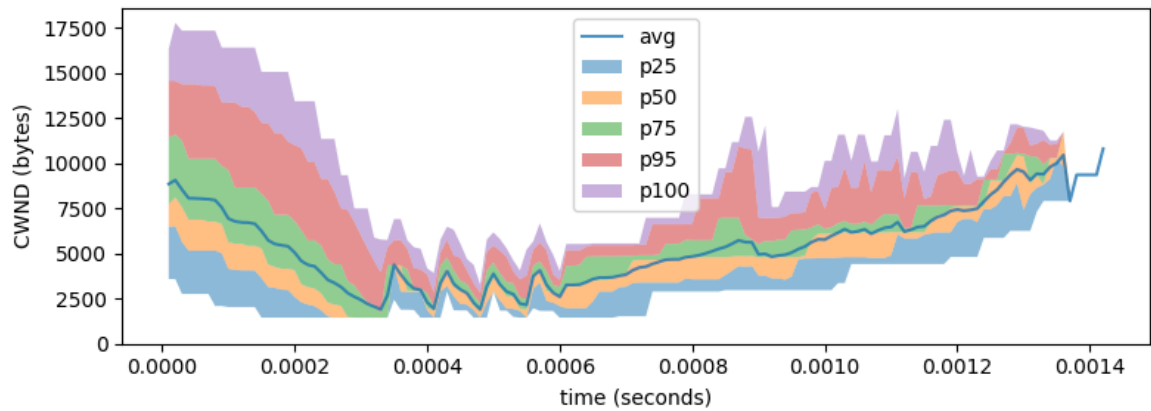
Figure



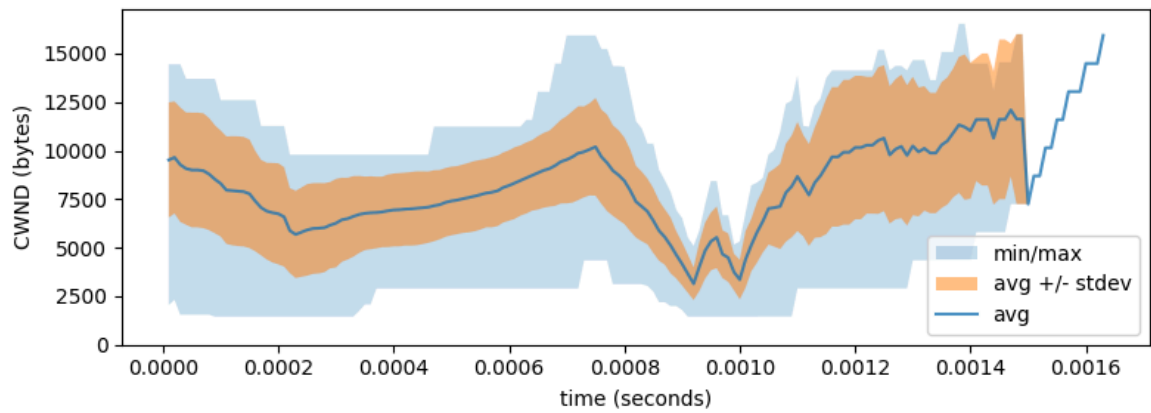
Figure



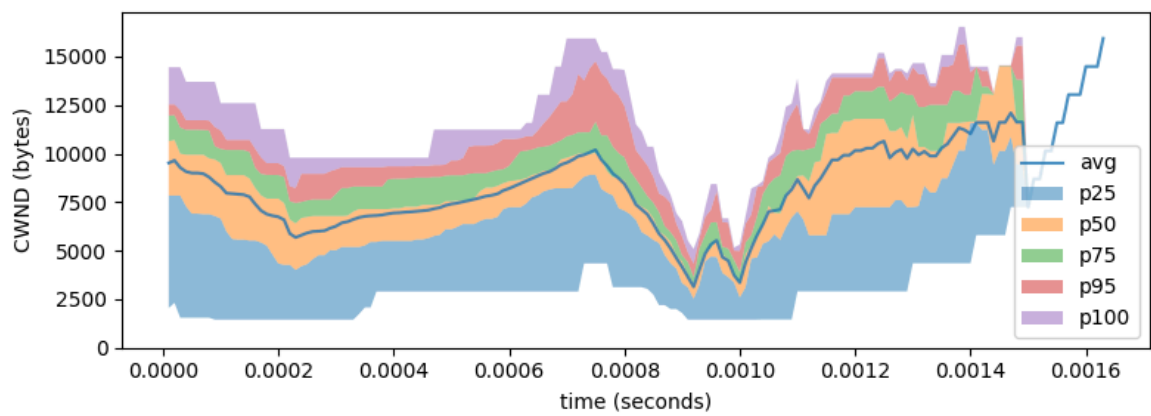
Figure



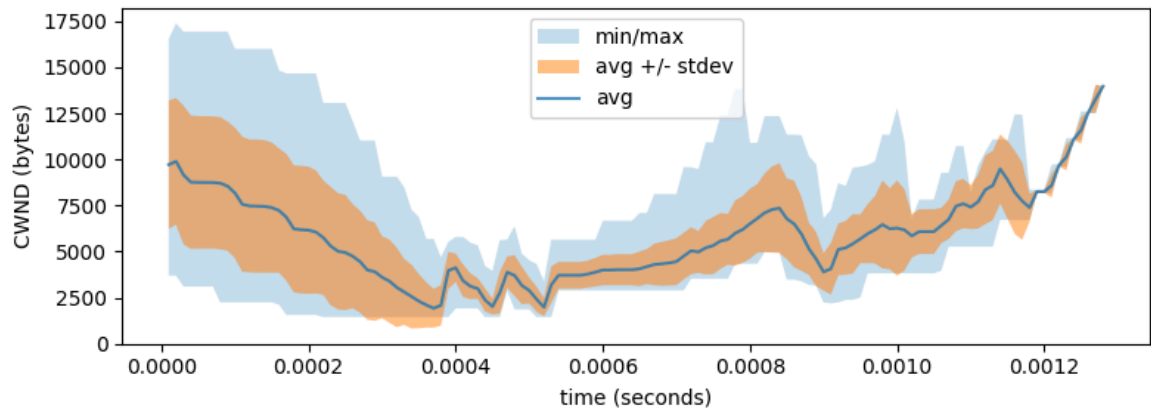
Figure



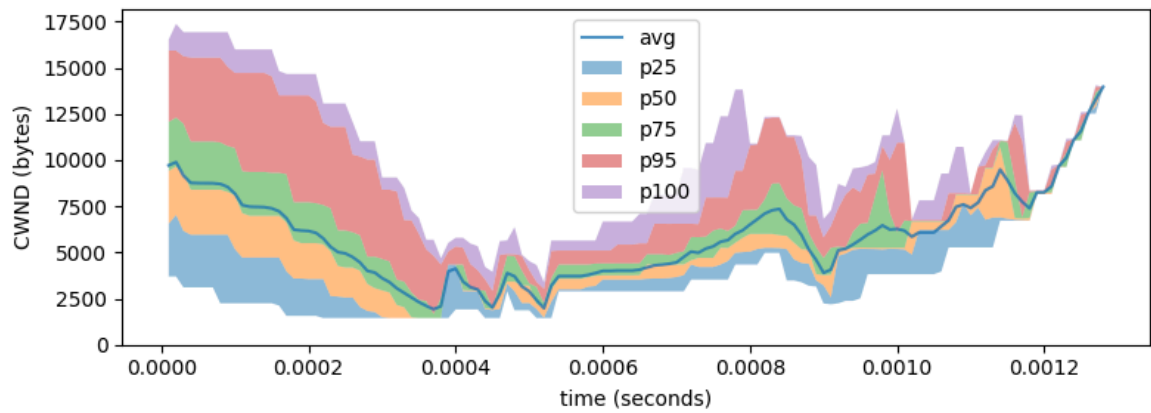
Figure



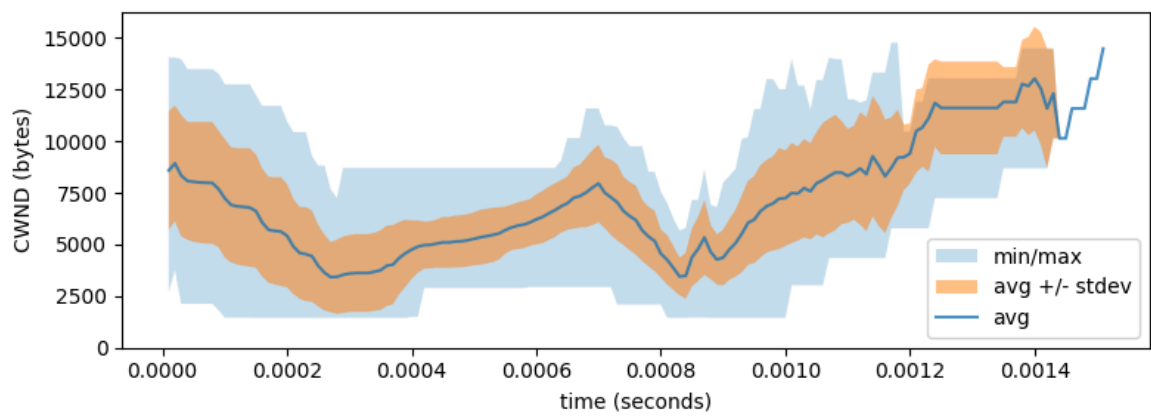
Figure



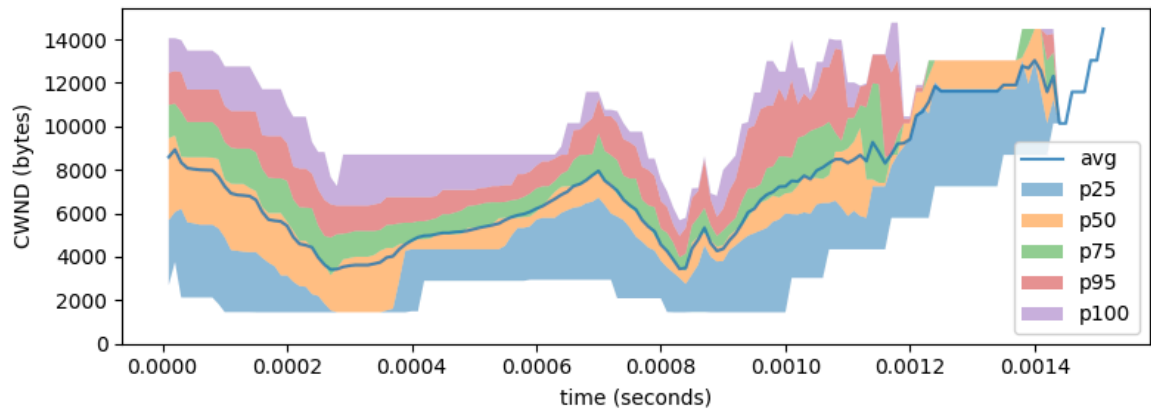
Figure



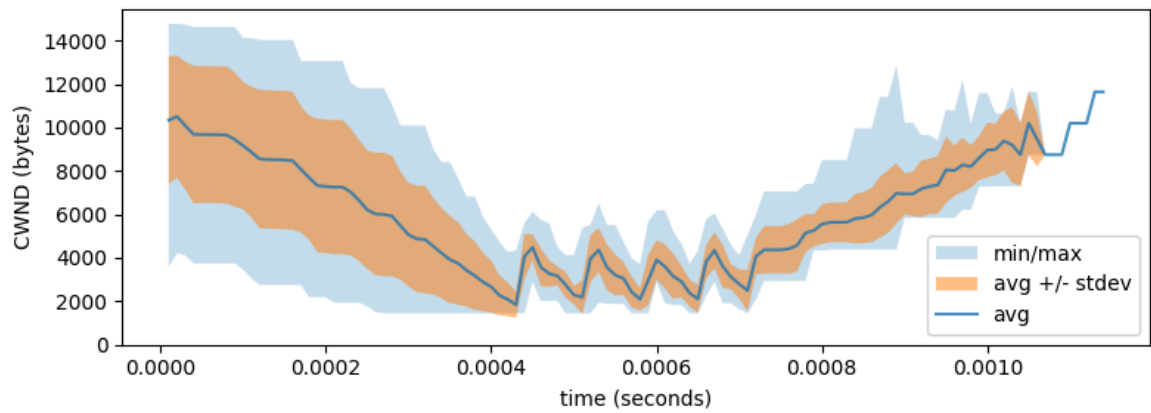
Figure



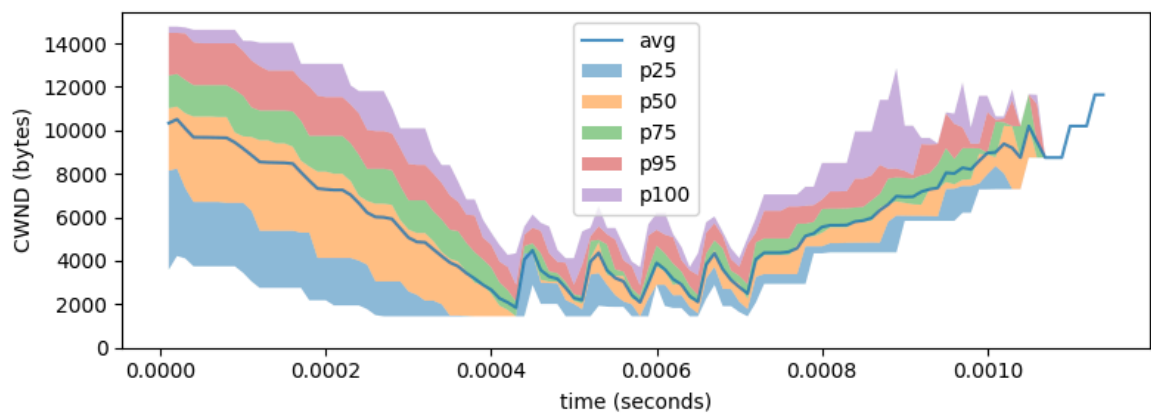
Figure



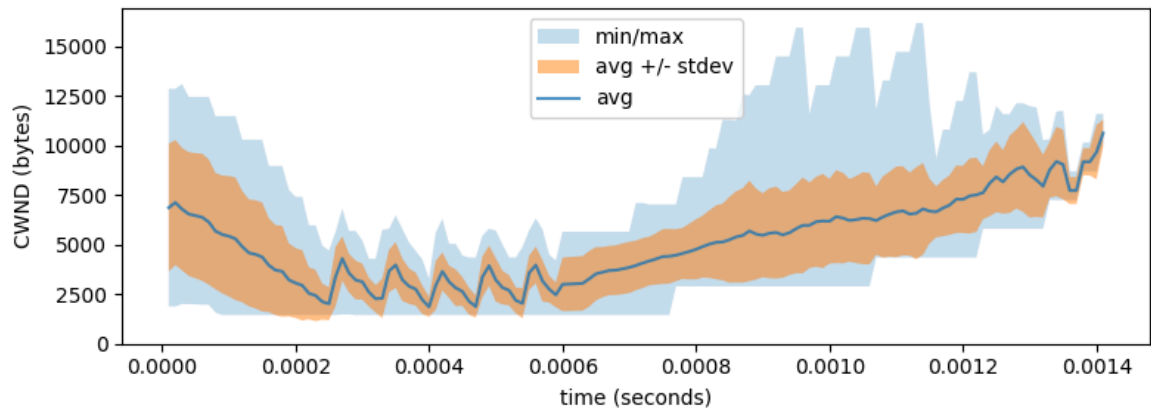
Figure



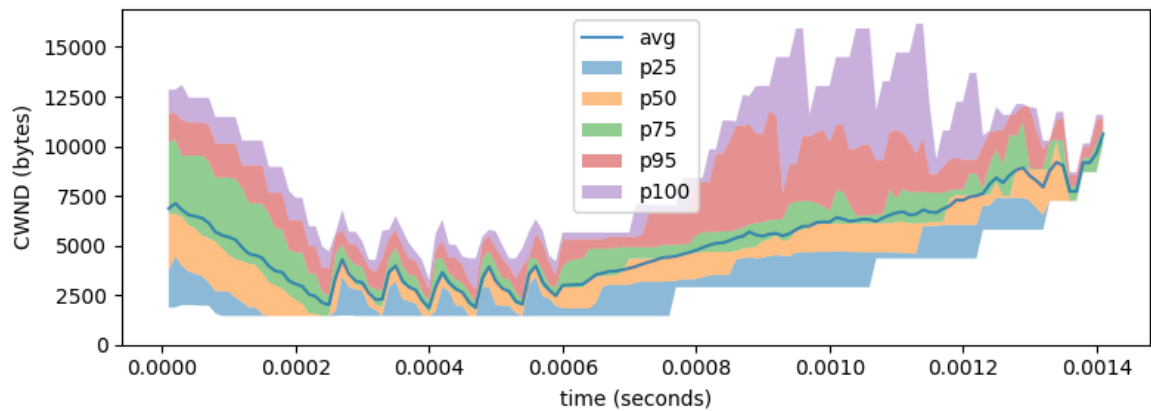
Figure



Figure



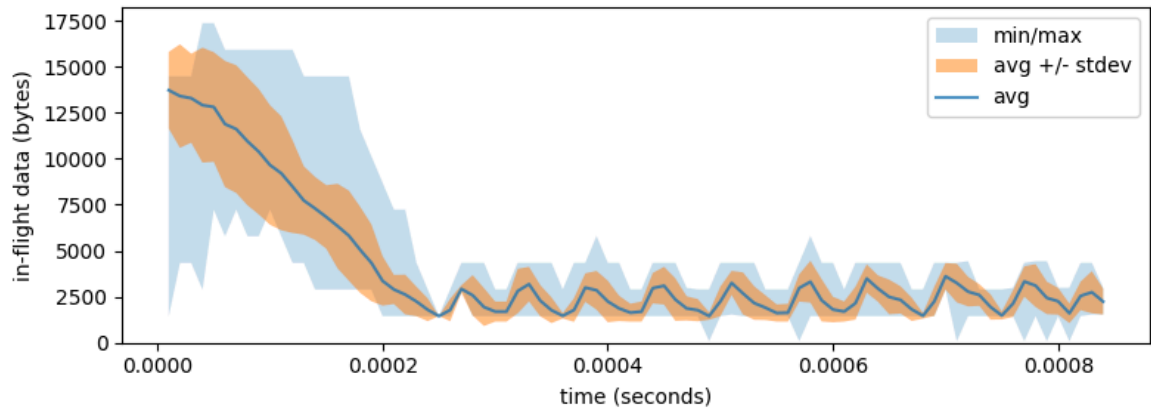
Figure



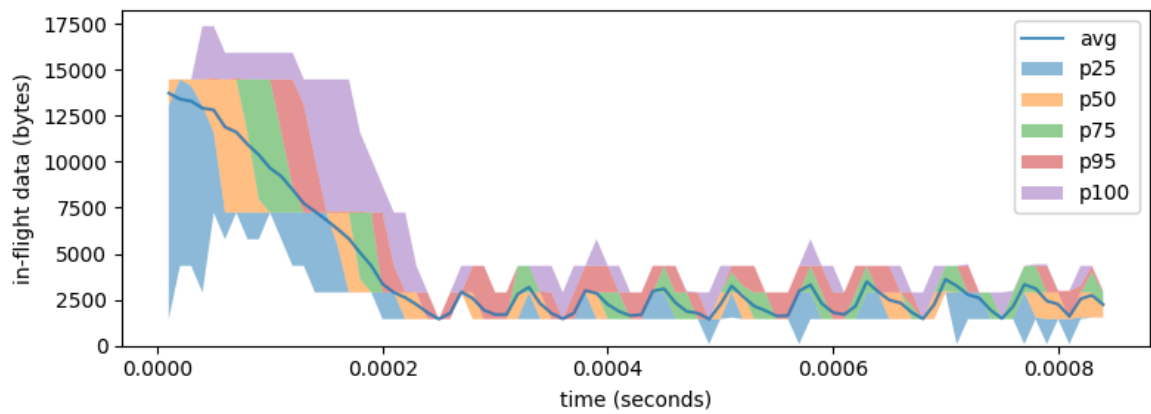
```
In [15]: def graph_inflight_metrics(
    inflight_metrics_by_burst, num_bursts, percentiles, graph_dir, prefix
):
    graph_cwnd_metrics(
        inflight_metrics_by_burst,
        num_bursts,
        percentiles,
        graph_dir,
        prefix,
        ylabel="in-flight data (bytes)",
        fln="inflight_analysis",
    )

    if RUN:
        SENDER_TO_INFLIGHT_BY_BURST_INTERP = get_sender_to_x_by_burst_interp(
            SENDER_TO_INFLIGHT_BY_BURST, NUM_BURSTS, INTERP_DELTA
        )
        INFLIGHT_METRICS_BY_BURST = get_metrics_by_burst(
            SENDER_TO_INFLIGHT_BY_BURST_INTERP, NUM_BURSTS, INTERP_DELTA, PERCENTILES
        )
        graph_inflight_metrics(
            INFLIGHT_METRICS_BY_BURST,
            NUM_BURSTS,
            PERCENTILES,
            GRAPH_DIR,
            EXP,
        )
```

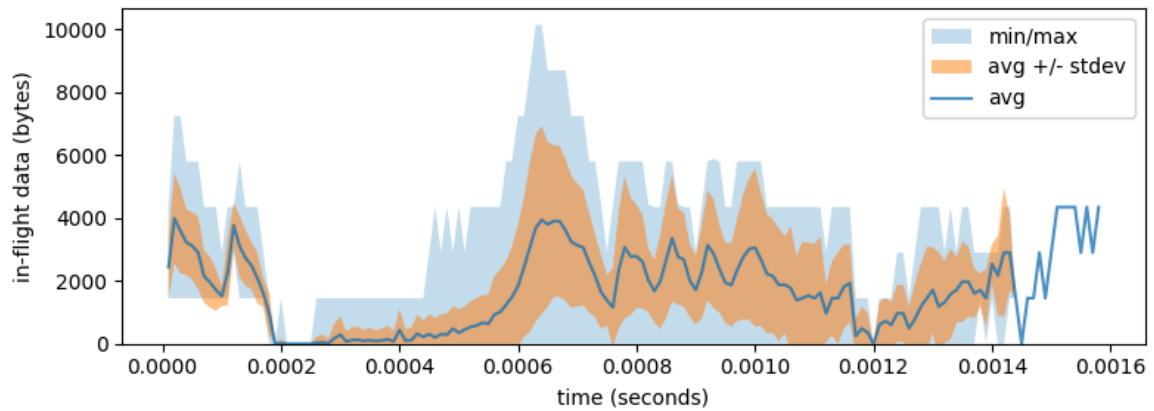
Figure



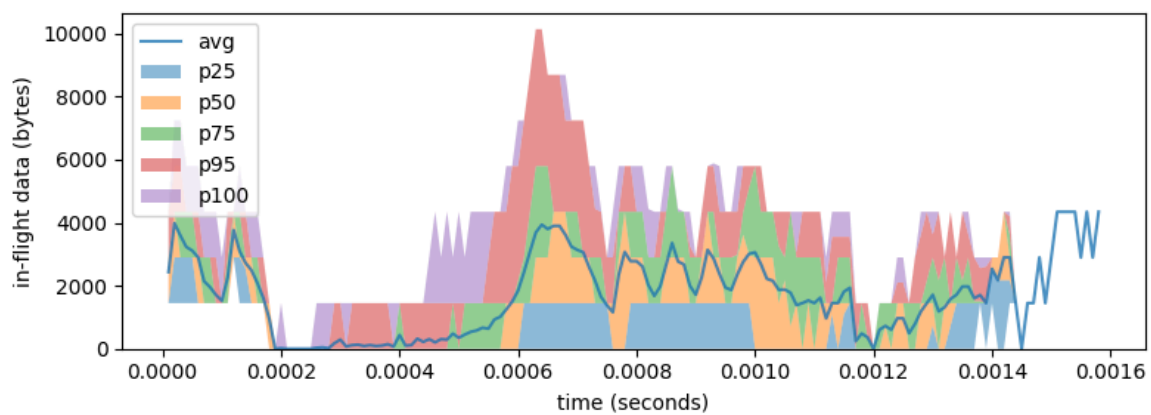
Figure



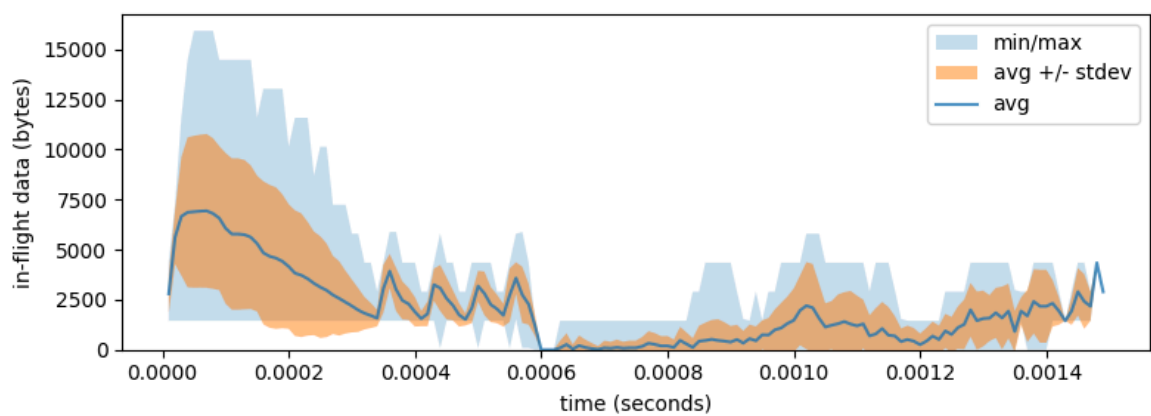
Figure



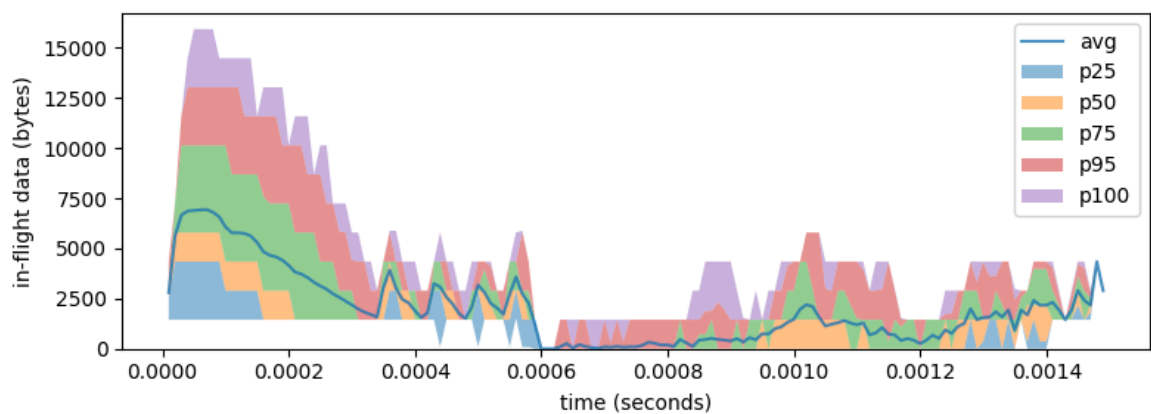
Figure



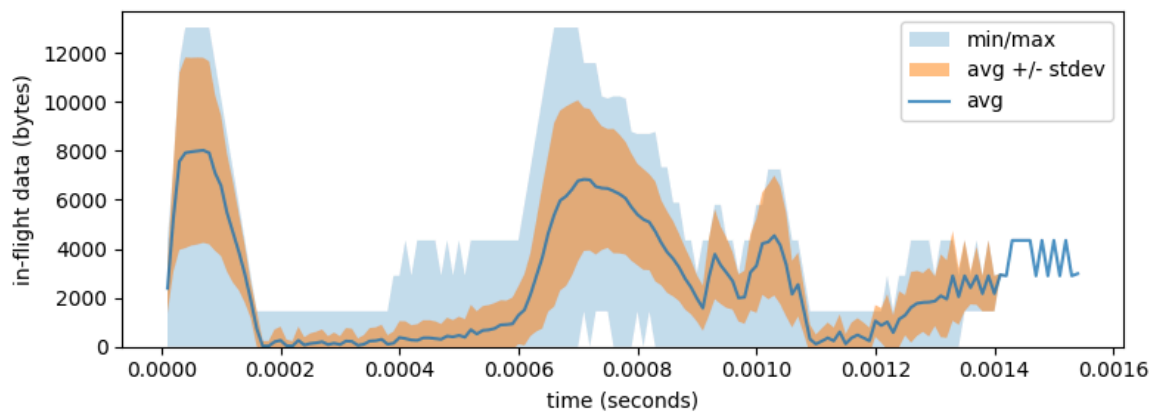
Figure



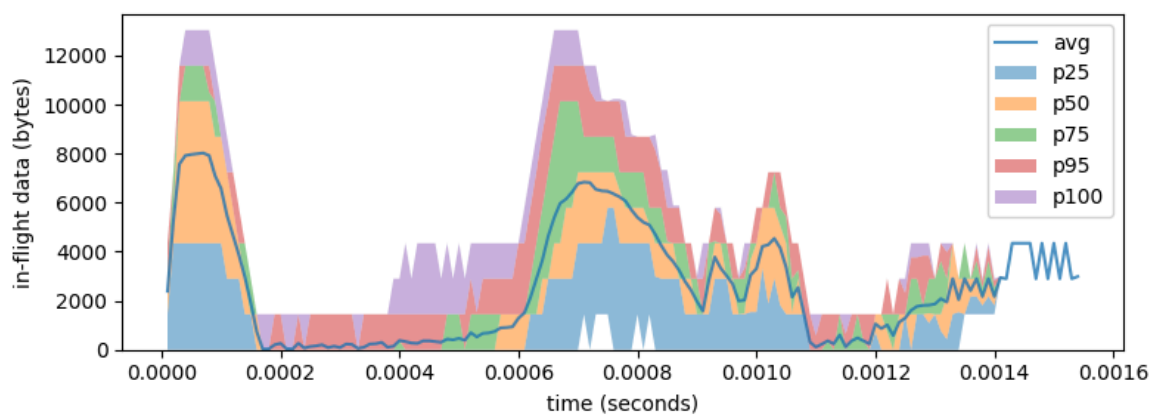
Figure



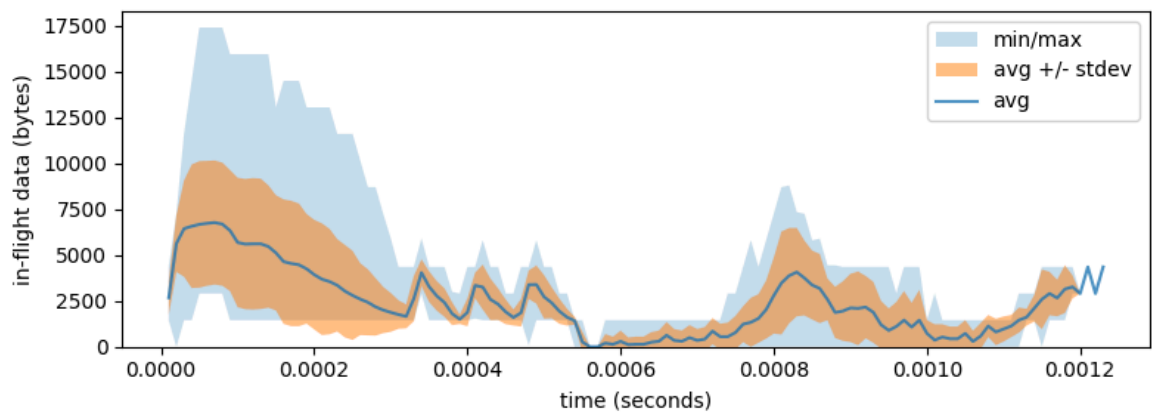
Figure



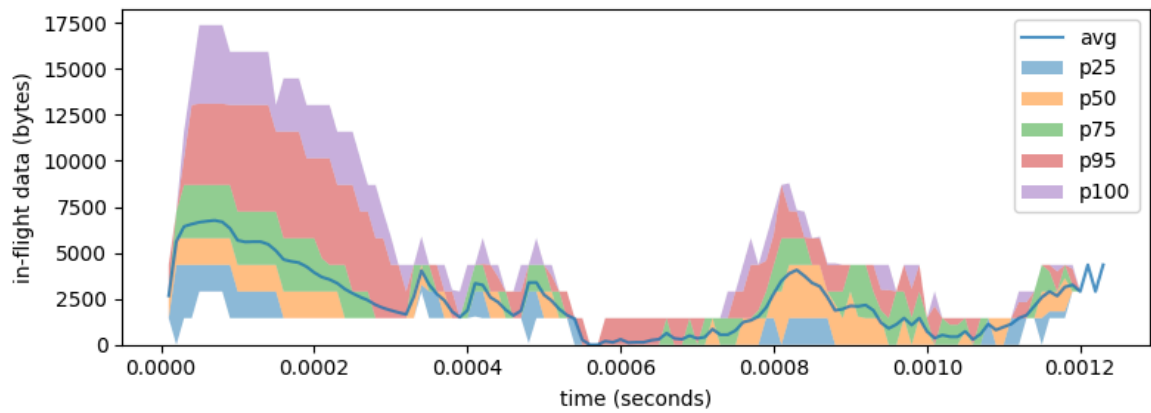
Figure



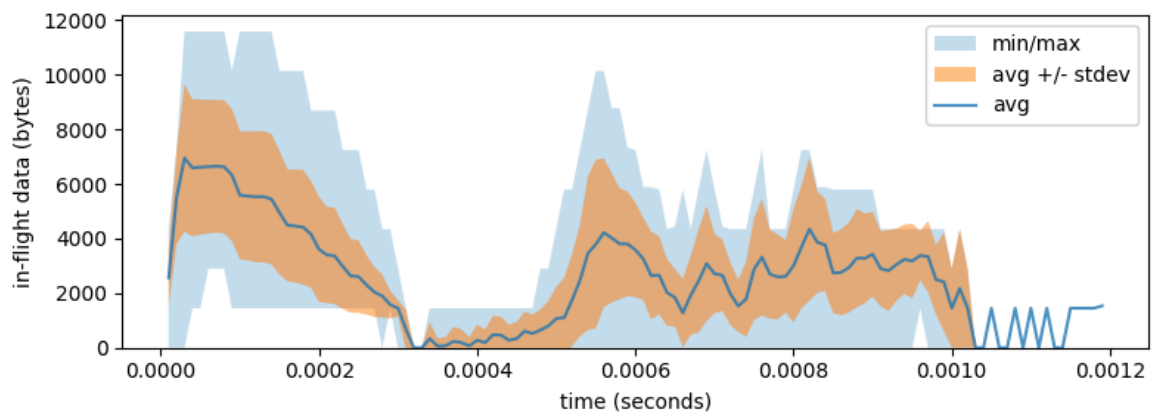
Figure



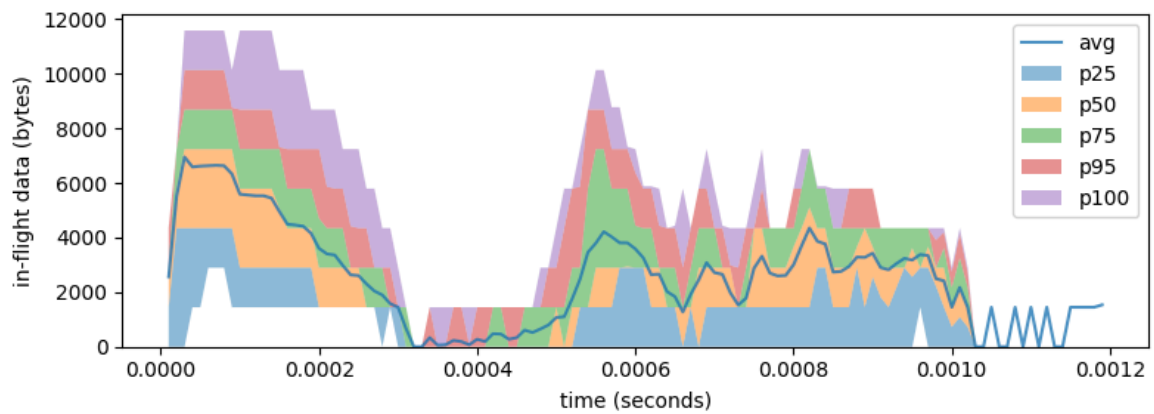
Figure



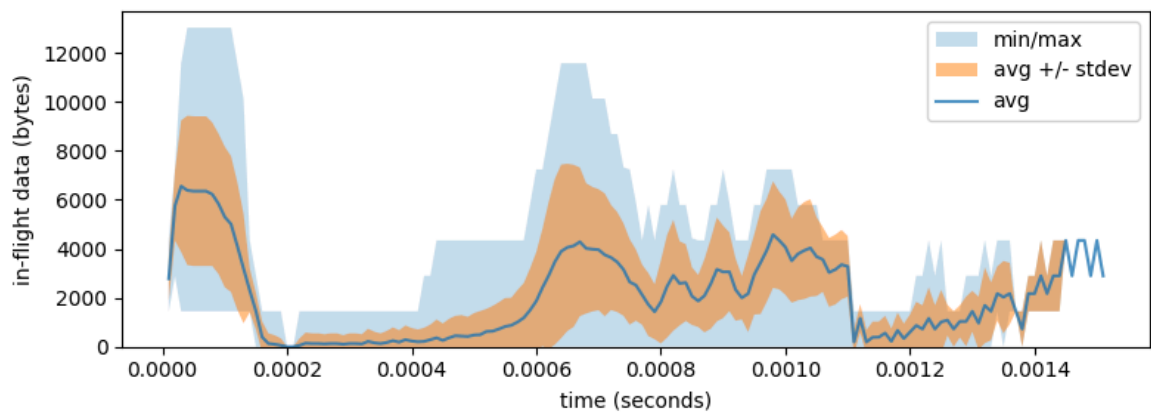
Figure



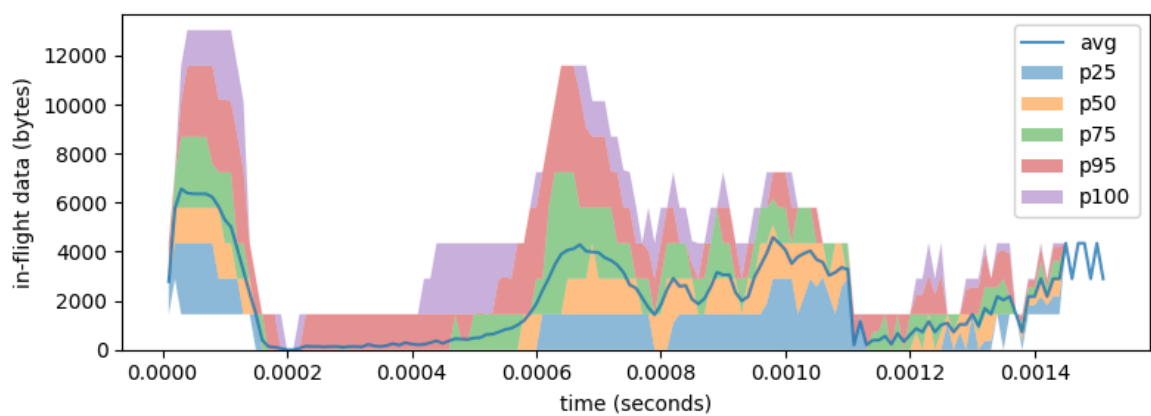
Figure



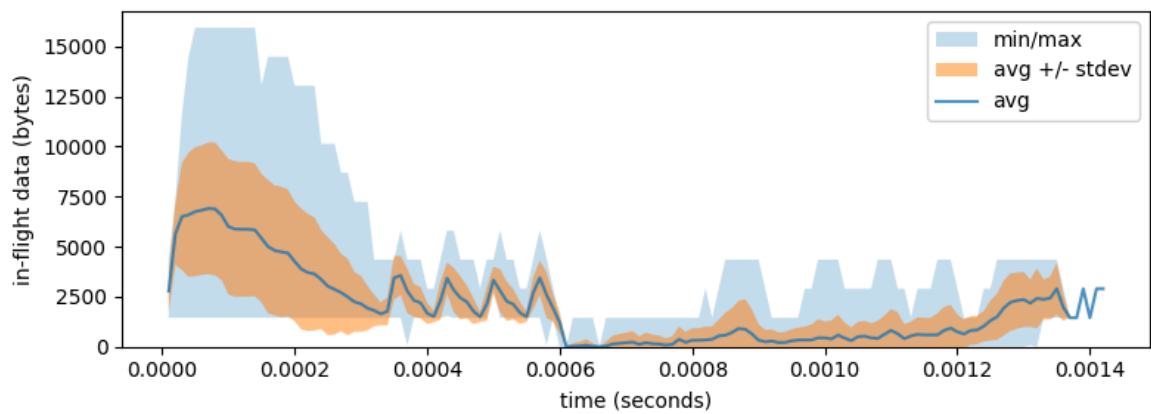
Figure



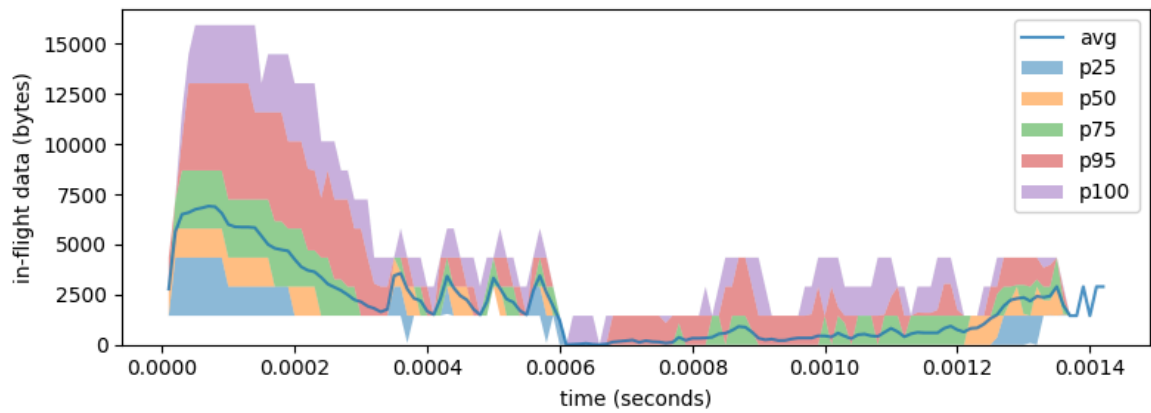
Figure



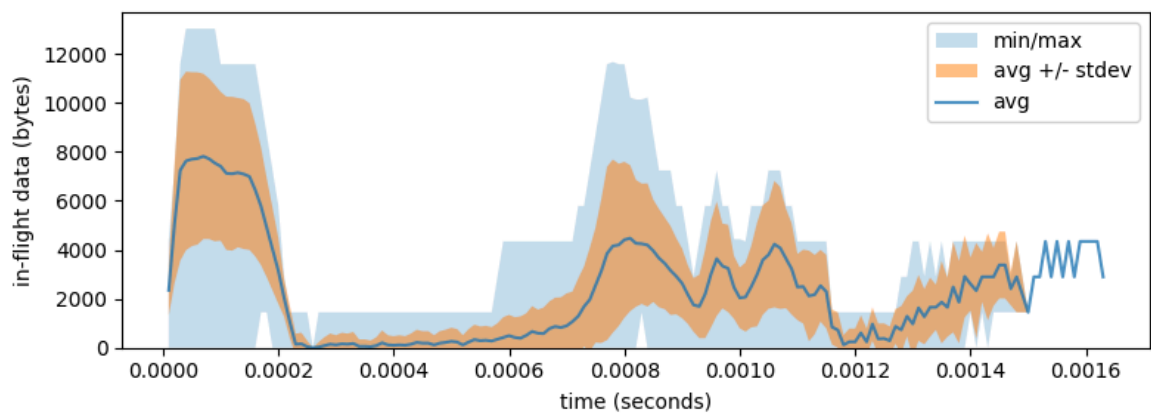
Figure



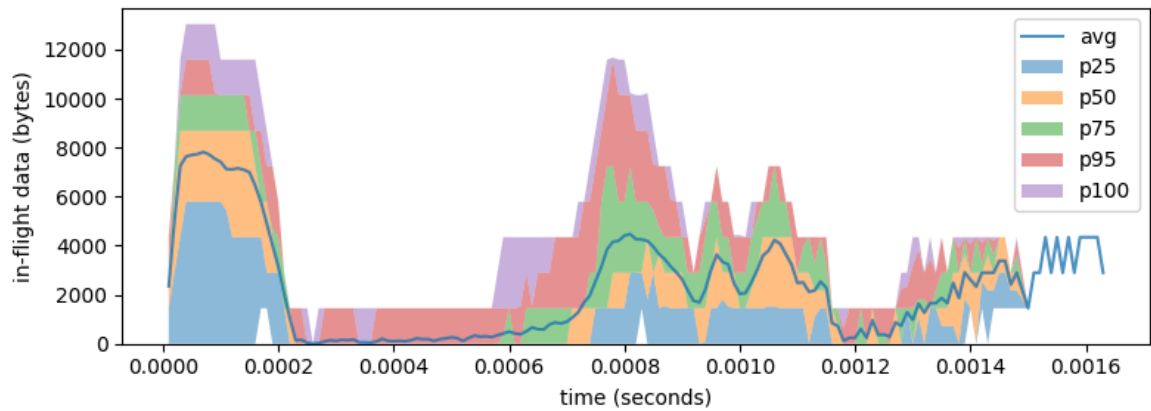
Figure



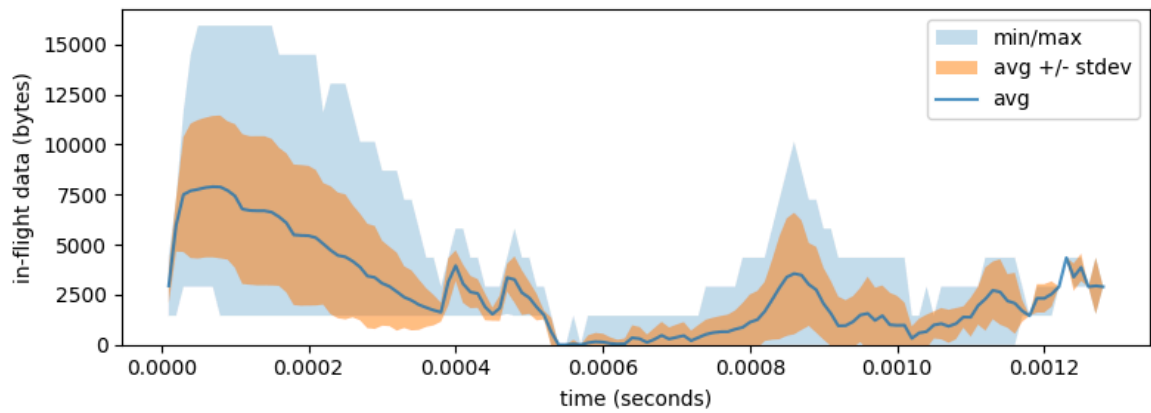
Figure



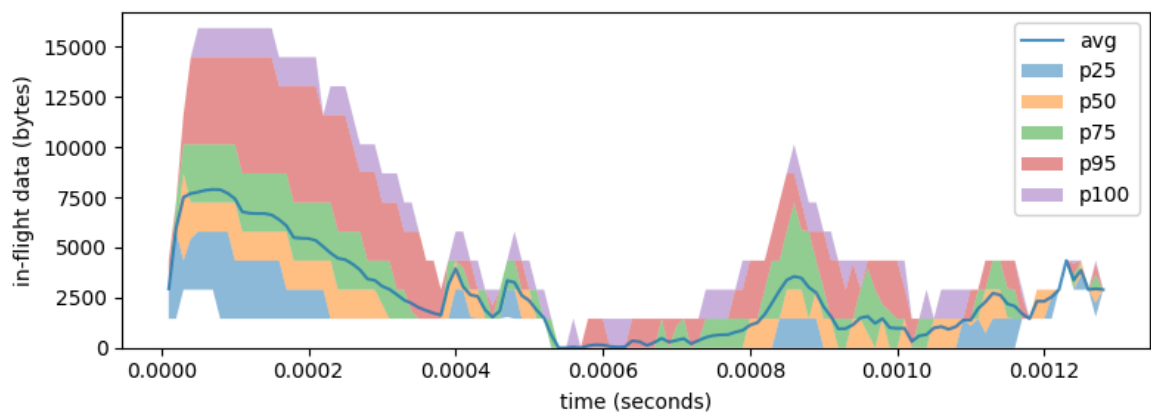
Figure



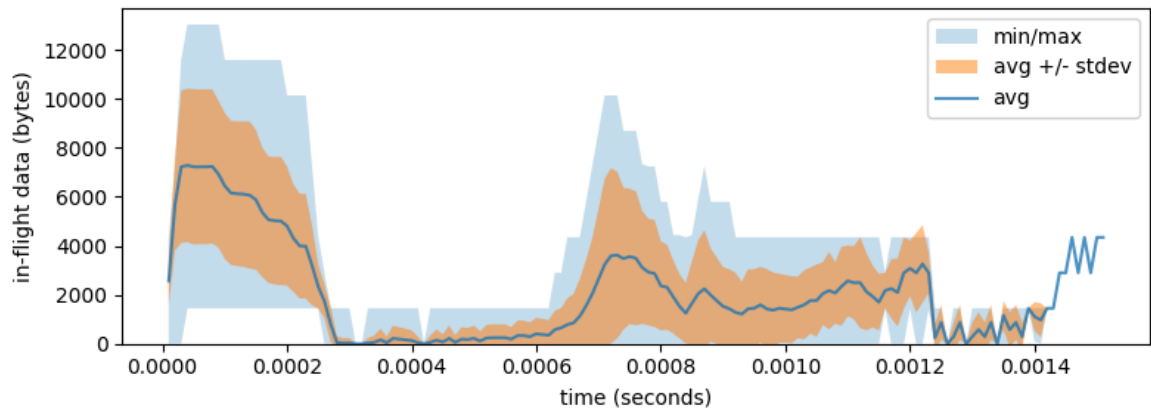
Figure



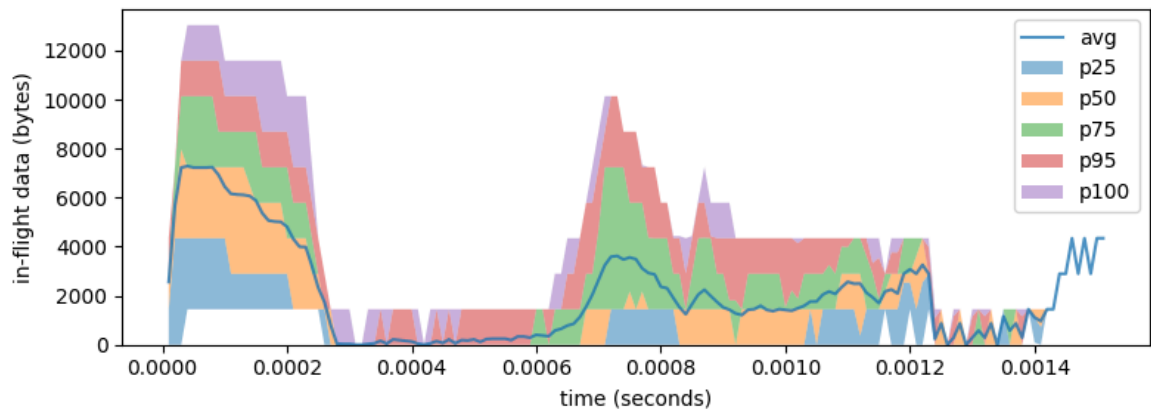
Figure



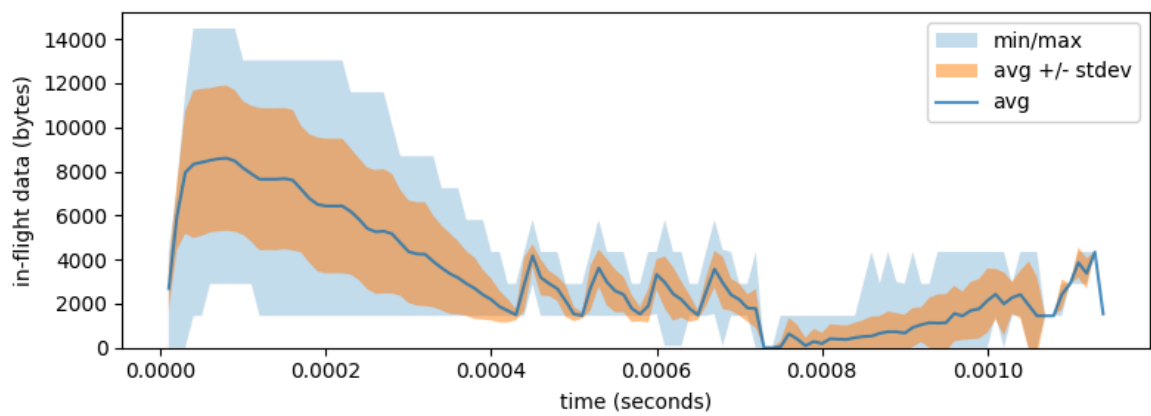
Figure



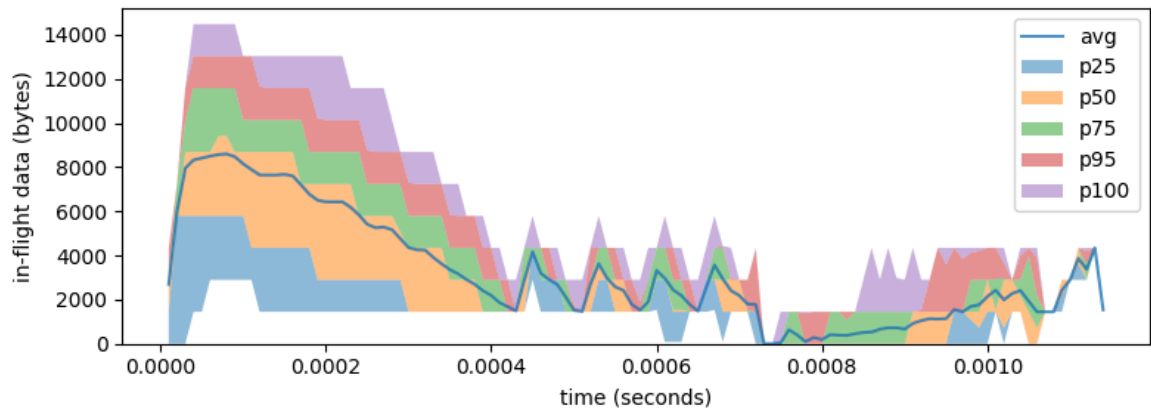
Figure



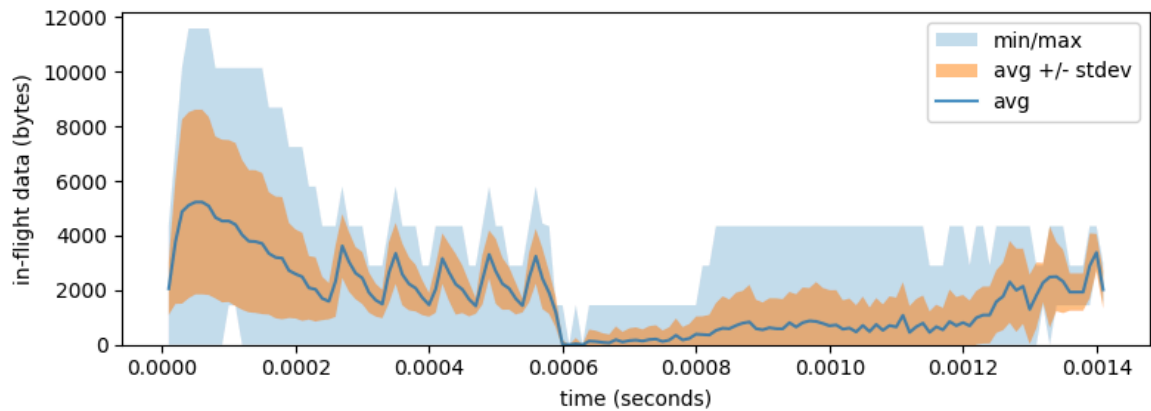
Figure



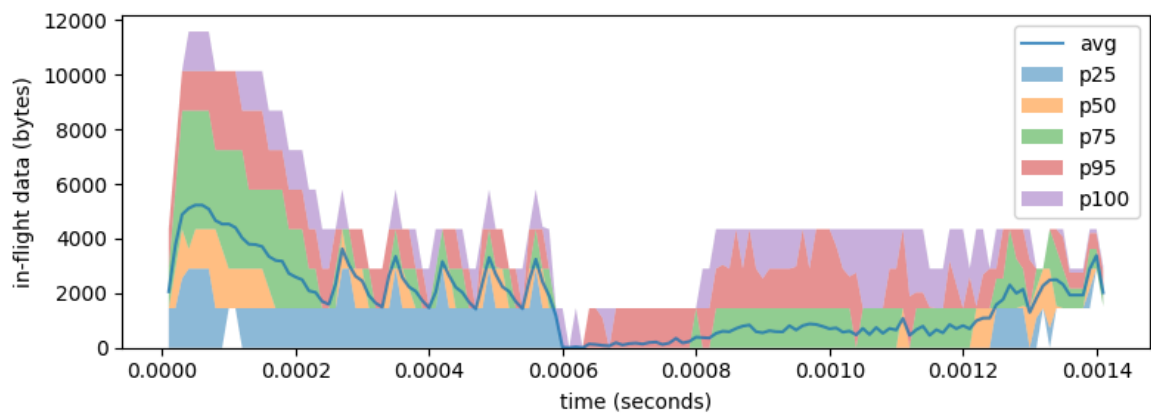
Figure



Figure



Figure



```
In [16]: def calculate_average_queue_length(
lengths_by_burst, interp_delta, bandwidth_bps, bytes_per_packet
):
    avg_q_length_by_burst = []
    for lengths in lengths_by_burst:
        old_xs, old_ys = zip(*lengths)
        new_xs = get_aligned_xs(old_xs[0], old_xs[-1], interp_delta)
        new_ys = step_interp(old_xs, old_ys, new_xs)
        avg_q_packets = new_ys.mean()
        avg_q_bytes = avg_q_packets * bytes_per_packet
        avg_q_us = avg_q_bytes / (bandwidth_bps / 8) * 1e6
        avg_q_length_by_burst.append((avg_q_packets, avg_q_bytes, avg_q_us))
    return avg_q_length_by_burst

def print_avg_q_length(
lengths_by_burst, num_bursts, interp_delta, bandwidth_bps, bytes_per_packet
):
    for burst_idx, (
        avg_q_packets,
        avg_q_bytes,
        avg_q_us,
    ) in enumerate(
        calculate_average_queue_length(
            lengths_by_burst, interp_delta, bandwidth_bps, bytes_per_packet

```

```

    )
):
    print(
        f"Burst {burst_idx + 1} of {num_bursts} - "
        f"Average queue length: {avg_q_packets:.2f} packets, "
        f"{avg_q_bytes:.2f} bytes, {avg_q_us:.2f} us"
    )

if RUN:
    BYTES_PER_PACKET = 1500
    BANDWIDTH_BITSPS = CONFIG["smallLinkBandwidthMbps"] * 1e6
    print_avg_q_length(
        INCAST_Q_METRICS["lengths"],
        NUM_BURSTS,
        INTERP_DELTA,
        BANDWIDTH_BITSPS,
        BYTES_PER_PACKET,
    )

```

```

Burst 1 of 13 - Average queue length: 191.87 packets, 287803.57 bytes, 23.02
us
Burst 2 of 13 - Average queue length: 37.52 packets, 56283.02 bytes, 4.50 us
Burst 3 of 13 - Average queue length: 72.64 packets, 108960.00 bytes, 8.72 u
s
Burst 4 of 13 - Average queue length: 87.14 packets, 130712.90 bytes, 10.46
us
Burst 5 of 13 - Average queue length: 87.94 packets, 131915.32 bytes, 10.55
us
Burst 6 of 13 - Average queue length: 89.51 packets, 134262.50 bytes, 10.74
us
Burst 7 of 13 - Average queue length: 56.91 packets, 85371.71 bytes, 6.83 us
Burst 8 of 13 - Average queue length: 76.32 packets, 114482.52 bytes, 9.16 u
s
Burst 9 of 13 - Average queue length: 70.84 packets, 106253.05 bytes, 8.50 u
s
Burst 10 of 13 - Average queue length: 102.29 packets, 153430.23 bytes, 12.2
7 us
Burst 11 of 13 - Average queue length: 69.61 packets, 104407.89 bytes, 8.35
us
Burst 12 of 13 - Average queue length: 146.98 packets, 220473.91 bytes, 17.6
4 us
Burst 13 of 13 - Average queue length: 55.94 packets, 83915.49 bytes, 6.71 u
s

```

```

In [17]: def graph_estimated_queue_ingress_rate(
    lengths_by_burst,
    bandwidth_bps,
    bytes_per_packet,
    interp_delta,
    graph_dir,
    prefix,
):
    for burst_idx, lengths in enumerate(lengths_by_burst):
        fig, axes = get_axes()
        ax = axes[0]

```

```

old_xs, old_ys = zip(*lengths)
new_xs = get_aligned_xs(old_xs[0], old_xs[-1], interp_delta)
new_ys = step_interp(old_xs, old_ys, new_xs)
new_ys *= 8 * bytes_per_packet

# ax.set_title(
#     f"Estimated queue ingress rate: Burst {burst_idx + 1} of {num_
# )
ax.set_xlabel("time (seconds)")
ax.set_ylabel("ingress rate (Gbps)")

dydxs = np.gradient(new_ys, new_xs)
# If queue is not empty, then it is by definition draining at the
# bandwidth. So if y is not 0, then add back the bandwidth to the
# gradient to calculate the true ingress rate instead of the net rat
dydxs = np.array(
    [
        dydx if y == 0 else (dydx + bandwidth_bps)
        for dydx, y in zip(dydxs, new_ys)
    ]
)
dydxs /= 1e9

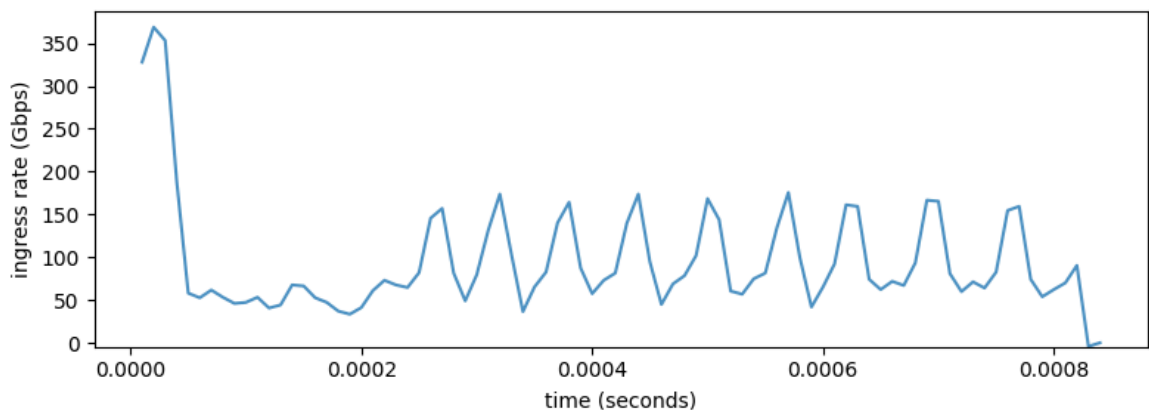
ax.plot(new_xs, dydxs, alpha=0.8)
ax.set_ylim(bottom=min(0, min(dydxs) * 1.1))

show(fig)
save(graph_dir, prefix, suffix=f"queue_ingress_rate_{burst_idx}")

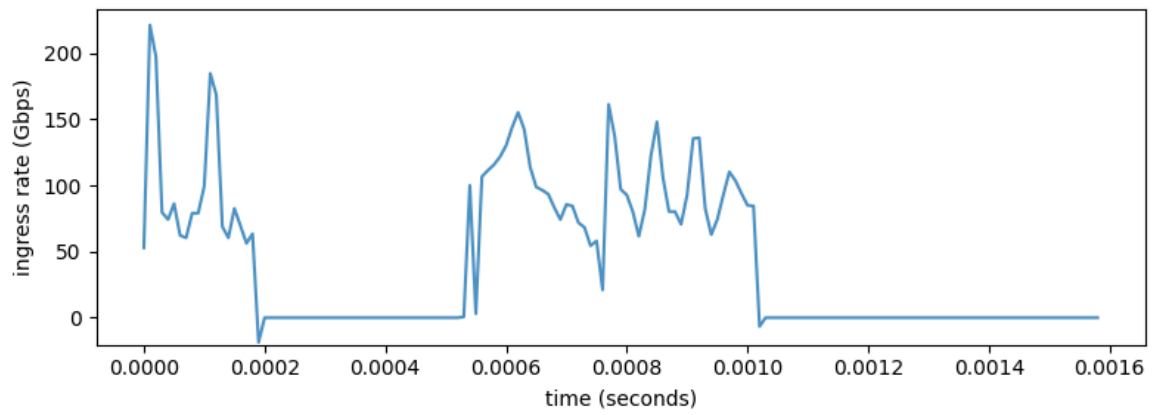
if RUN:
    graph_estimated_queue_ingress_rate(
        INCAST_Q_METRICS["lengths"],
        BANDWIDTH_BITSPS,
        BYTES_PER_PACKET,
        INTERP_DELTA,
        GRAPH_DIR,
        EXP,
    )

```

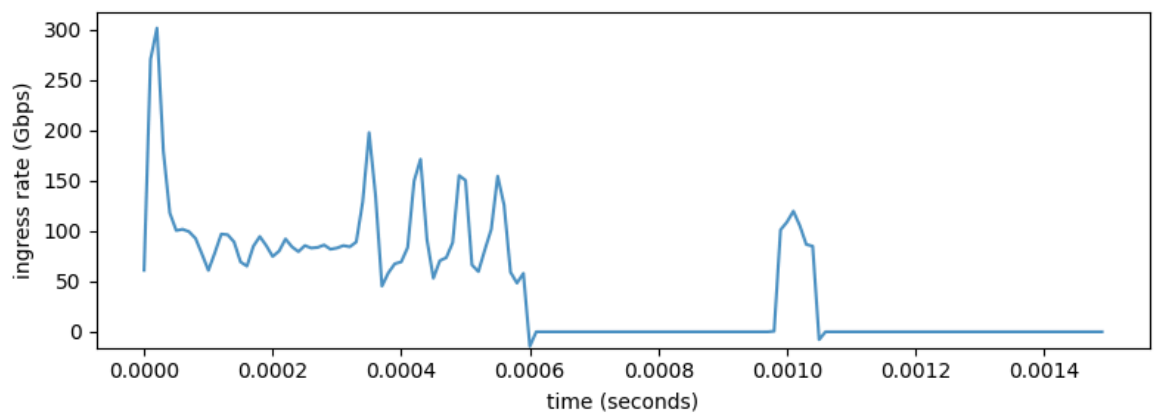
Figure



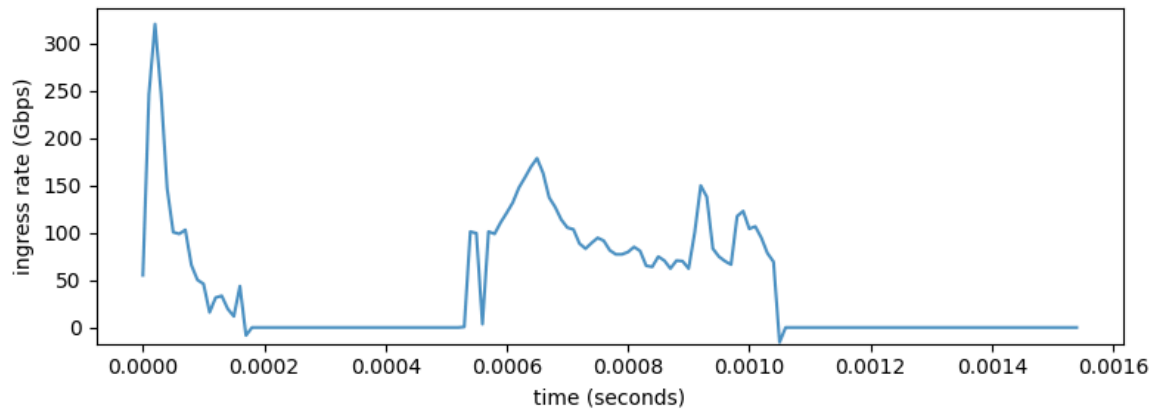
Figure



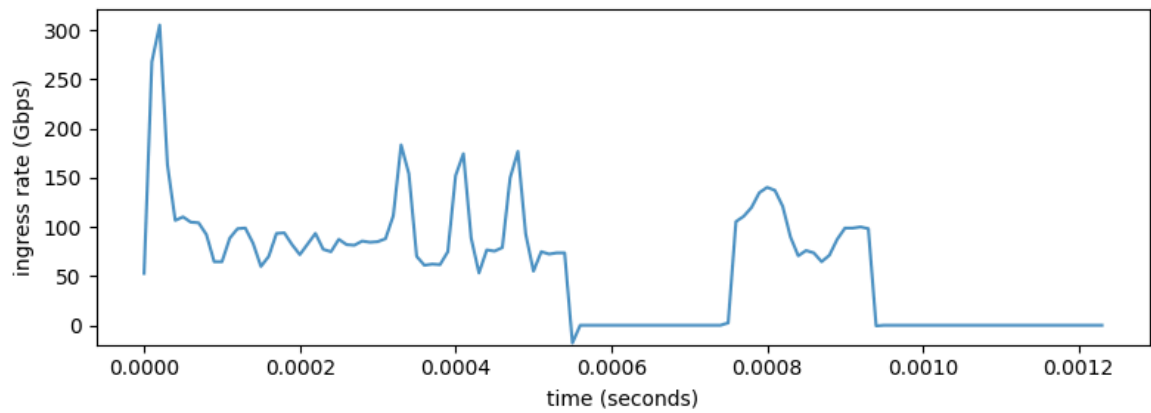
Figure



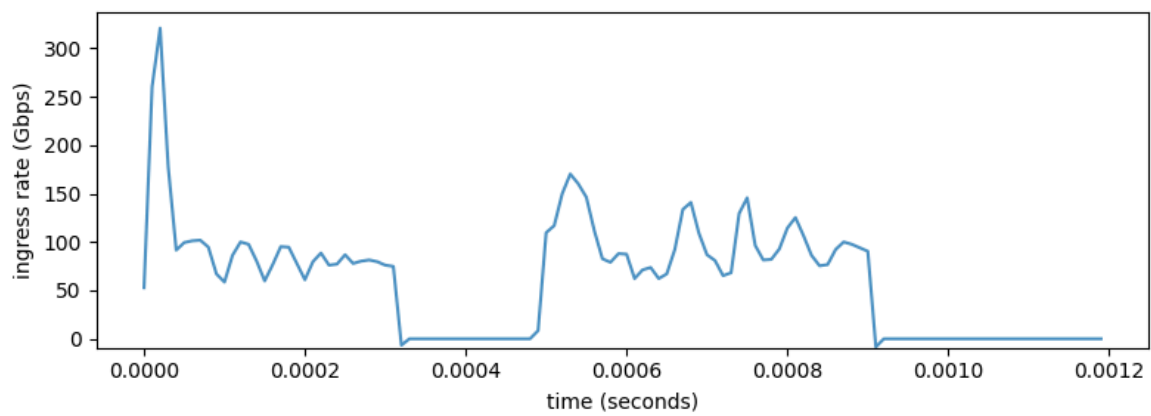
Figure



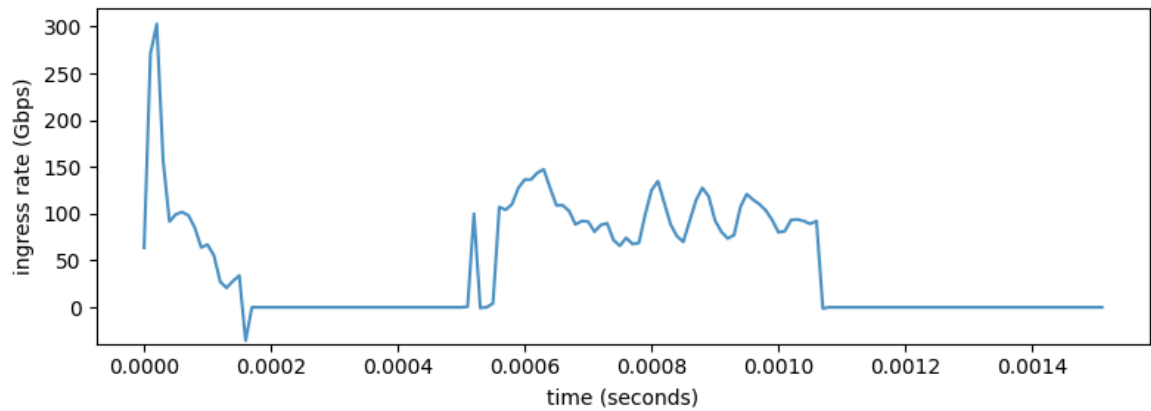
Figure



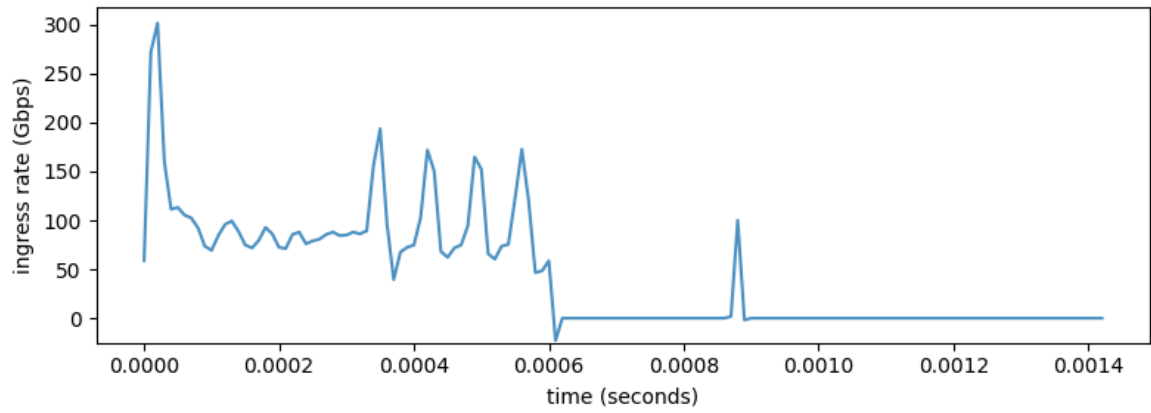
Figure



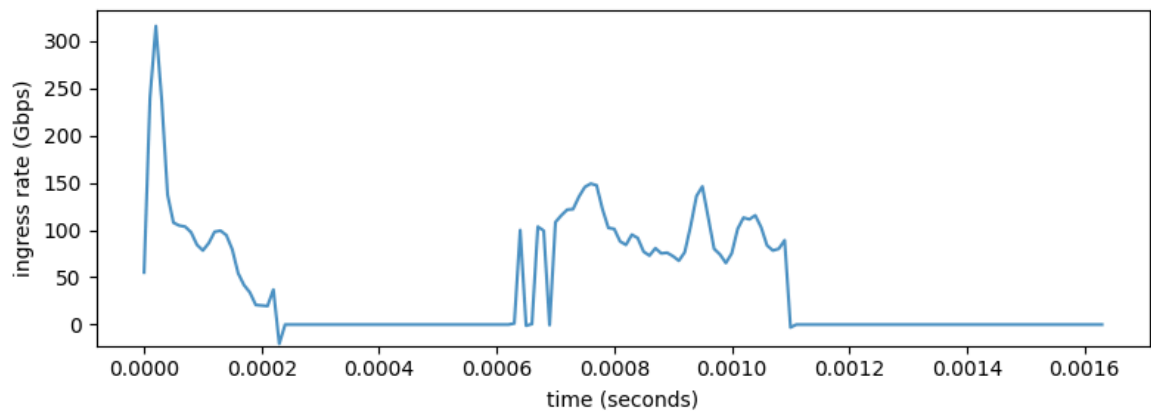
Figure



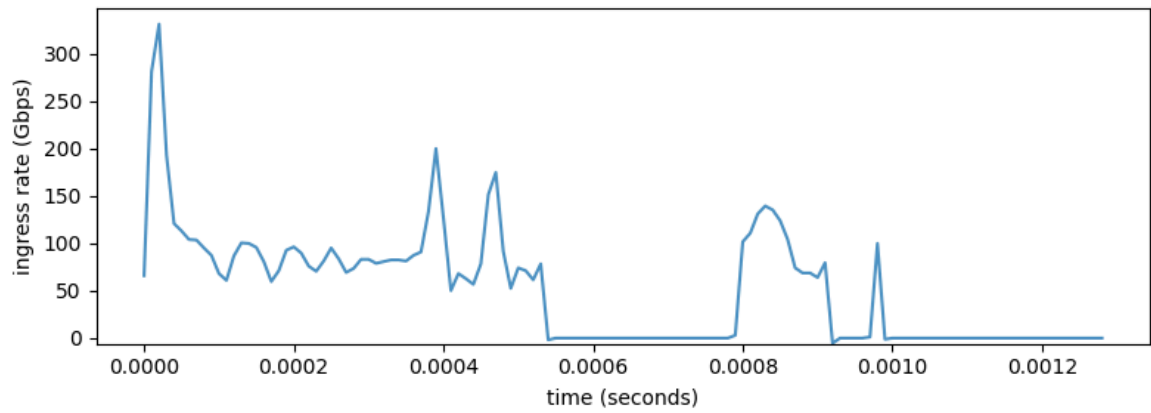
Figure



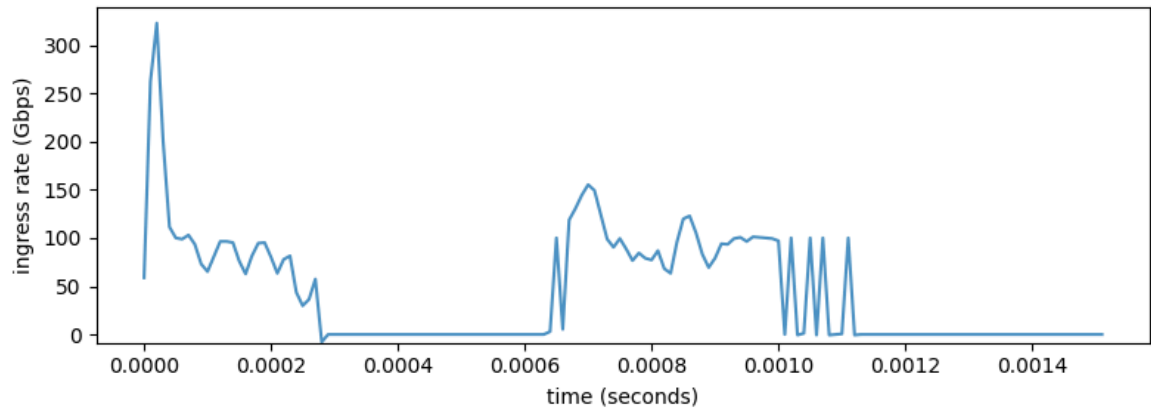
Figure



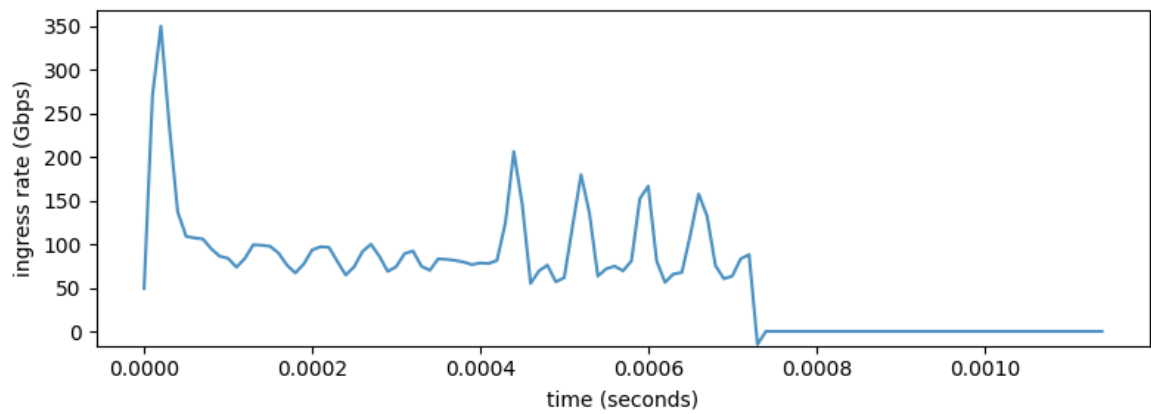
Figure



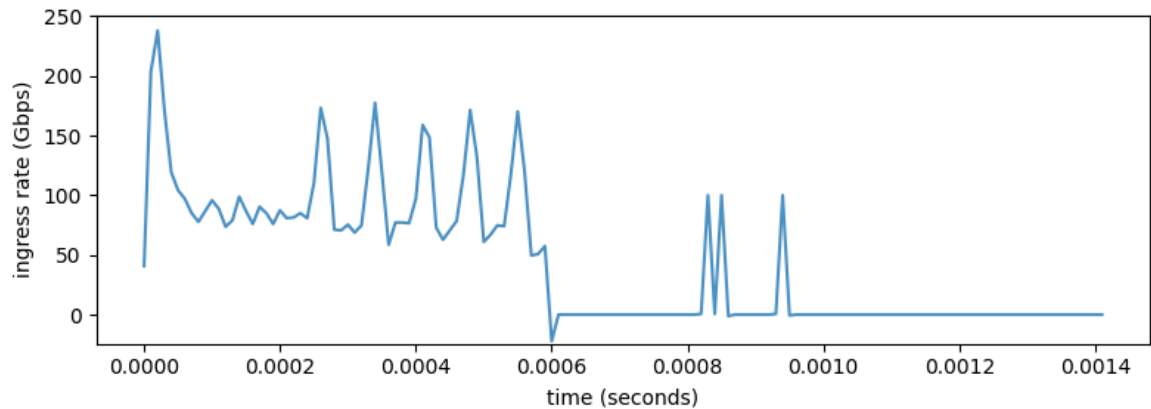
Figure



Figure



Figure



```
In [18]: def get_cwnd_metrics_across_bursts(
sender_to_cwnds_by_burst_interp, num_bursts, interp_delta, percentiles
):
    # We always ignore the first burst, since it is different than the other
    # due to slow start.
    if num_bursts == 1:
        print(
            "Error: No results because we ignore the first burst, but there is"
        )
    return
```

```

# Flatten all senders and bursts.
to_avg = []
# Throw away the first burst, because it always looks different.
for burst_idx in range(1, num_bursts):
    # Find the earliest start time for a flow in this burst.
    for bursts in sender_to_cwnds_by_burst_interp.values():
        # Throw away bursts with no samples.
        if not bursts[burst_idx]:
            continue
        to_avg.append(bursts[burst_idx])

xs, avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, _ = get_metrics_helper(
    to_avg, interp_delta, percentiles
)

# To calculate the average total CWND across bursts, first we need to sum
to_avg = []
# Throw away the first burst, because it always looks different.
for burst_idx in range(1, num_bursts):
    to_sum = []
    for bursts in sender_to_cwnds_by_burst_interp.values():
        # Throw away bursts with no samples.
        if not bursts[burst_idx]:
            continue
        to_sum.append(bursts[burst_idx])
    # Sum the ys across the flows in this burst
    xs, _, _, _, _, sum_ys = get_metrics_helper(
        to_sum, interp_delta, percentiles
    )
    to_avg.append(list(zip(xs, sum_ys)))
# Now average the totals across all bursts
xs, total_ys, _, _, _, _ = get_metrics_helper(to_avg, interp_delta, percentiles)

return xs, avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, total_ys

def graph_aggregate_cwnd_across_bursts(
    cwnd_metrics_across_bursts,
    percentiles,
    graph_dir,
    prefix,
    ylabel="CWND",
    fln="combined_cwnd_analysis",
):
    (
        xs,
        avg_ys,
        _,
        _,
        percentiles_ys,
        totals_ys,
    ) = cwnd_metrics_across_bursts
    xs = xs * 1e3
    avg_ys = avg_ys / 1e3

```

```

# print(sum_ys[-10:])

# Left graph
fig, axes = get_axes()
ax = axes[0]
totals_ys = totals_ys / BDP_BYTES
ax.plot(xs, totals_ys, linewidth=LINESIZE, alpha=0.8)
ax.set_xlabel("time (ms)", fontsize=FONTSIZE)
ax.set_ylabel(f"total {ylabel}\n(x BDP)", fontsize=FONTSIZE)
ax.tick_params(axis="x", labelsize=FONTSIZE)
ax.tick_params(axis="y", labelsize=FONTSIZE)
ax.set_xlim(left=-0.01 * xs[-1], right=1.01 * xs[-1])
ax.set_ylim(bottom=-0.01 * max(totals_ys), top=1.1 * max(totals_ys))
show(fig)
save(graph_dir, prefix, suffix=f"{fln}_0")

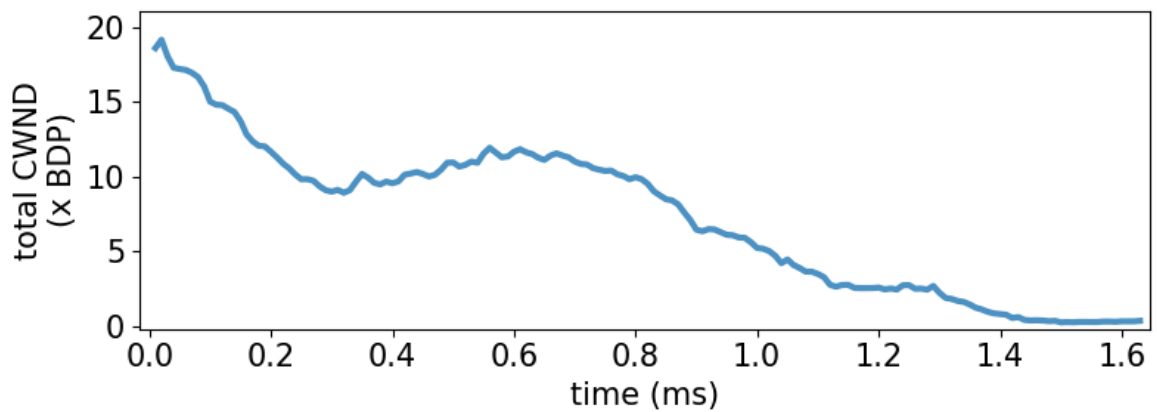
# Right graph
fig, axes = get_axes()
ax = axes[0]
ax.plot(xs, avg_ys, linewidth=LINESIZE, label="avg", alpha=0.8)
max_y = max(avg_ys)
for idx in range(1, percentiles_ys.shape[0]):
    prev = percentiles_ys[idx - 1]
    prev = prev / 1e3
    nxt = percentiles_ys[idx]
    nxt = nxt / 1e3
    ax.fill_between(
        xs,
        prev,
        nxt,
        alpha=0.5,
        label=f"p{percentiles[idx]}",
    )
    max_y = max(max_y, *nxt)
# ax.set_title("CWND of active flows across all bursts")
ax.set_xlabel("time (ms)", fontsize=FONTSIZE)
ax.set_ylabel(f"per-flow {ylabel}\n(KB)", fontsize=FONTSIZE)
ax.tick_params(axis="x", labelsize=FONTSIZE)
ax.tick_params(axis="y", labelsize=FONTSIZE)
ax.set_xlim(left=-0.01 * xs[-1], right=1.01 * xs[-1])
ax.set_ylim(bottom=-0.01 * max_y, top=1.1 * max_y)
ax.legend(loc="upper center", fontsize=FONTSIZE, ncols=3)
show(fig)
save(graph_dir, prefix, suffix="combined_cwnd_analysis_1")

if RUN:
    BDP_BYTES = (
        CONFIG["smallLinkBandwidthMbps"] * 1e6 / 8 * 6 * CONFIG["delayPerLir
    )
    CWND_METRICS_ACROSS_BURSTS = get_cwnd_metrics_across_bursts(
        SENDER_TO_CWNS_BY_BURST_INTERP, NUM_BURSTS, INTERP_DELTA, PERCENTIL
    )
    graph_aggregate_cwnd_across_bursts(
        CWND_METRICS_ACROSS_BURSTS,

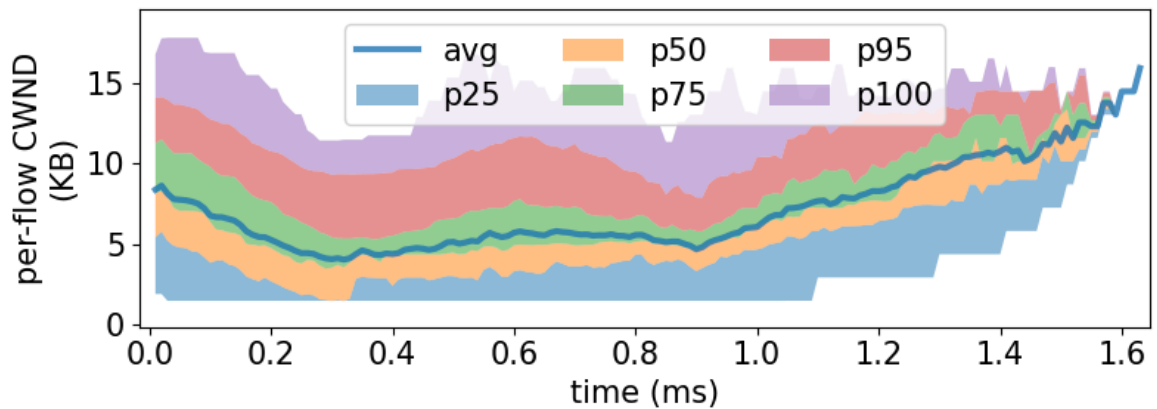
```

```
PERCENTILES,
GRAPH_DIR,
EXP,
)
```

Figure



Figure



```
In [19]: def get_inflight_metrics_across_bursts(
    sender_to_inflight_by_burst_interp, num_bursts, interp_delta, percentile
):
    return get_cwnd_metrics_across_bursts(
        sender_to_inflight_by_burst_interp, num_bursts, interp_delta, percentile
    )

def graph_aggregate_inflight_across_bursts(
    inflight_metrics_across_bursts,
    percentiles,
    graph_dir,
    prefix,
):
    graph_aggregate_cwnd_across_bursts(
        inflight_metrics_across_bursts,
        percentiles,
        graph_dir,
        prefix,
        ylabel="in-flight data",
```

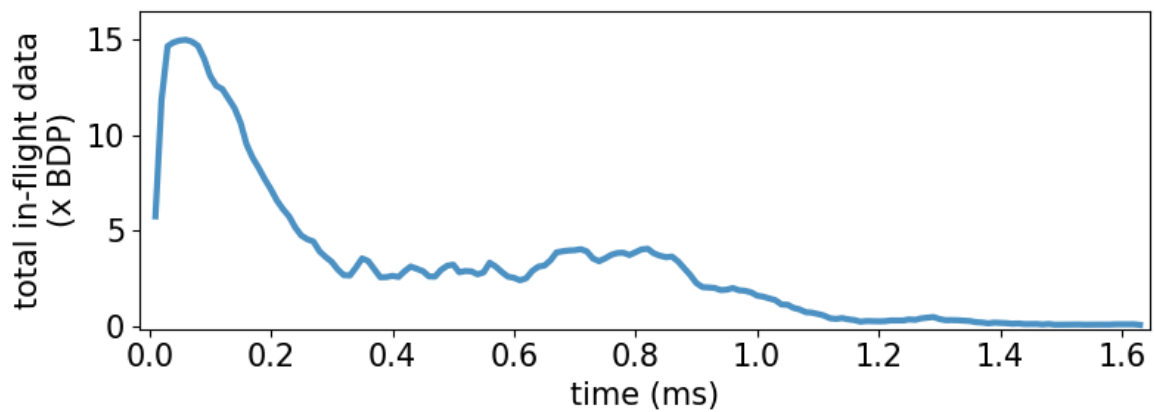
```

        fln="combined_inflight_analysis",
    )

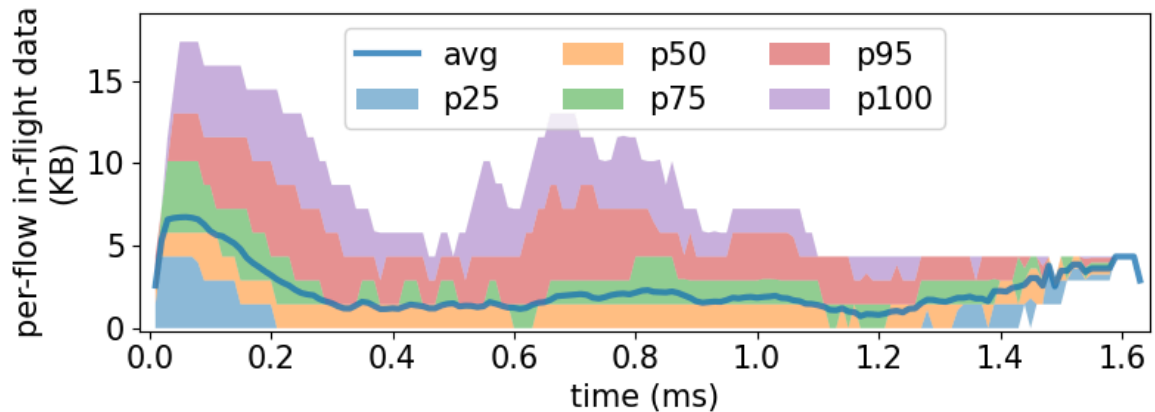
    if RUN:
        INFLIGHT_METRICS_ACROSS_BURSTS = get_inflight_metrics_across_bursts(
            SENDER_TO_INFLIGHT_BY_BURST_INTERP, NUM_BURSTS, INTERP_DELTA, PERCENTILES
        )
        graph_aggregate_inflight_across_bursts(
            INFLIGHT_METRICS_ACROSS_BURSTS,
            PERCENTILES,
            GRAPH_DIR,
            EXP,
        )

```

Figure



Figure



```

In [20]: def graph_total_cwnd(
    cwnd_metrics_by_burst,
    num_bursts,
    bdp_bytes,
    graph_dir,
    prefix,
    ylabel="CWND",
    fln="total_cwnd",
):
    for burst_idx in range(num_bursts):

```

```

xs, _, _, _, _, _, sum_ys = cwnd_metrics_by_burst[burst_idx]
xs = xs * 1e3

fig, axes = get_axes()
ax = axes[0]

sum_ys_KB = sum_ys / 1e3
ax.plot(xs, sum_ys_KB, label=f"Total {ylabel}", alpha=0.8)
max_y = max(sum_ys_KB)

# Draw a line at the BDP
bdp_kbytes = bdp_bytes / 1e3
ax.plot(
    [xs[0], xs[-1]],
    [bdp_kbytes, bdp_kbytes],
    label="BDP",
    color="orange",
    linestyle="dashed",
    alpha=0.8,
)

# ax.set_title(f"Total CWND in bytes: Burst {burst_idx + 1} of {num_
ax.set_xlabel("time (seconds)", fontsize=FONTSIZE)
ax.set_ylabel(f"total {ylabel}\n(KB)", fontsize=FONTSIZE)
ax.set_xlim(left=-0.01 * xs[-1], right=1.01 * xs[-1])
ax.set_ylim(bottom=-0.01 * max_y, top=1.1 * max_y)
ax.tick_params(axis="x", labelsize=FONTSIZE)
ax.tick_params(axis="y", labelsize=FONTSIZE)
ax.legend()
show(fig)
save(graph_dir, prefix, suffix=f"{fln}_0")

fig, axes = get_axes()
ax = axes[0]

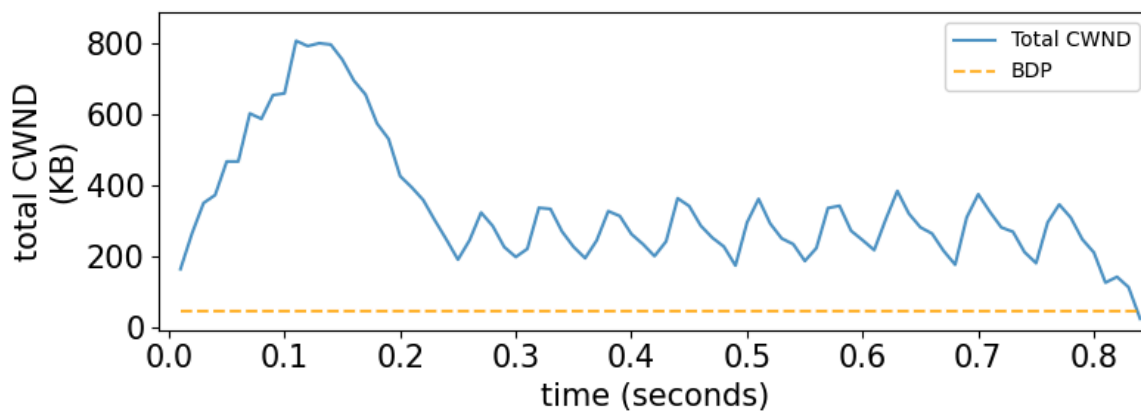
ys_bdp = [y / bdp_bytes for y in sum_ys]
ax.plot(
    xs,
    ys_bdp,
    # label="Total CWND as a multiple of BDP",
    alpha=0.8,
)
max_y = max(ys_bdp)

# ax.set_title(
#     f"Total CWND in multiples of BDP: Burst {burst_idx + 1} of {nu
# )
ax.set_xlabel("time (ms)", fontsize=FONTSIZE)
ax.set_ylabel(f"total {ylabel}\n(x BDP)", fontsize=FONTSIZE)
ax.set_xlim(left=-0.01 * xs[-1], right=1.01 * xs[-1])
ax.set_ylim(bottom=-0.01 * max_y, top=1.1 * max_y)
ax.tick_params(axis="x", labelsize=FONTSIZE)
ax.tick_params(axis="y", labelsize=FONTSIZE)
show(fig)
save(graph_dir, prefix, suffix=f"{fln}_1")

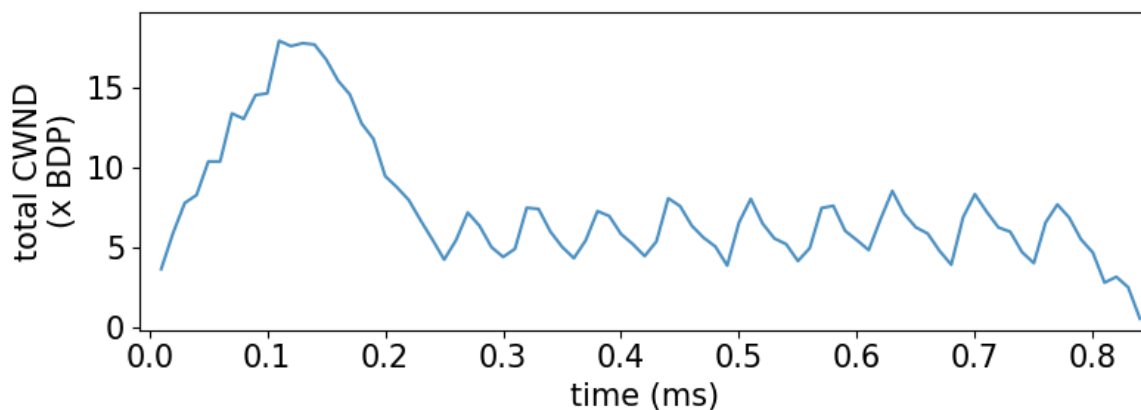
```

```
if RUN:
    graph_total_cwnd(
        CWND_METRICS_BY_BURST,
        NUM_BURSTS,
        BDP_BYTES,
        GRAPH_DIR,
        EXP,
    )
```

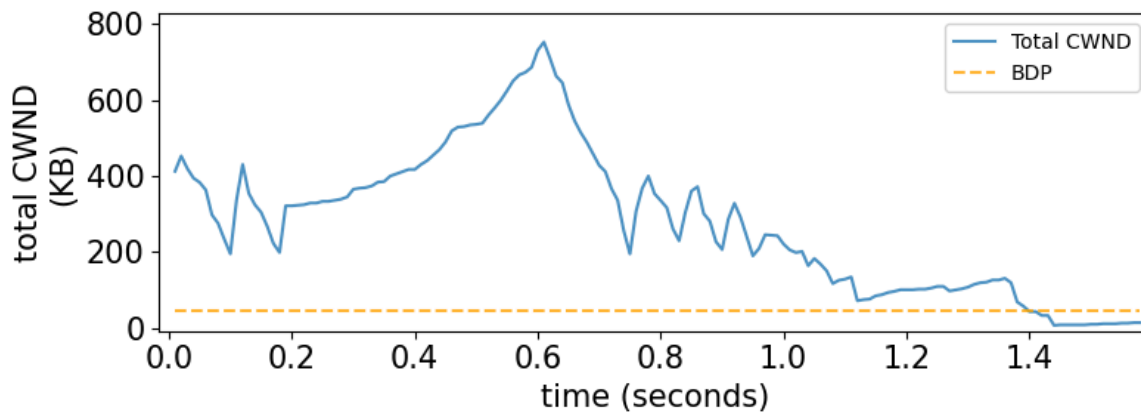
Figure



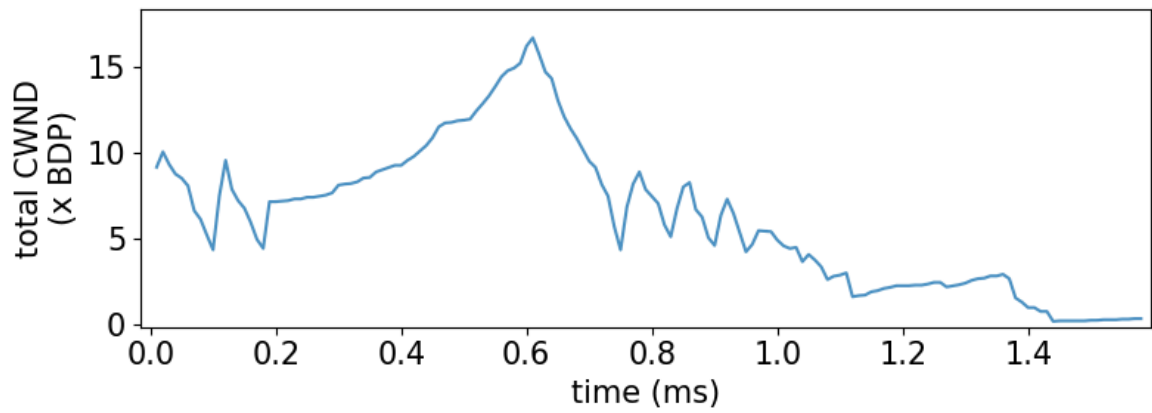
Figure



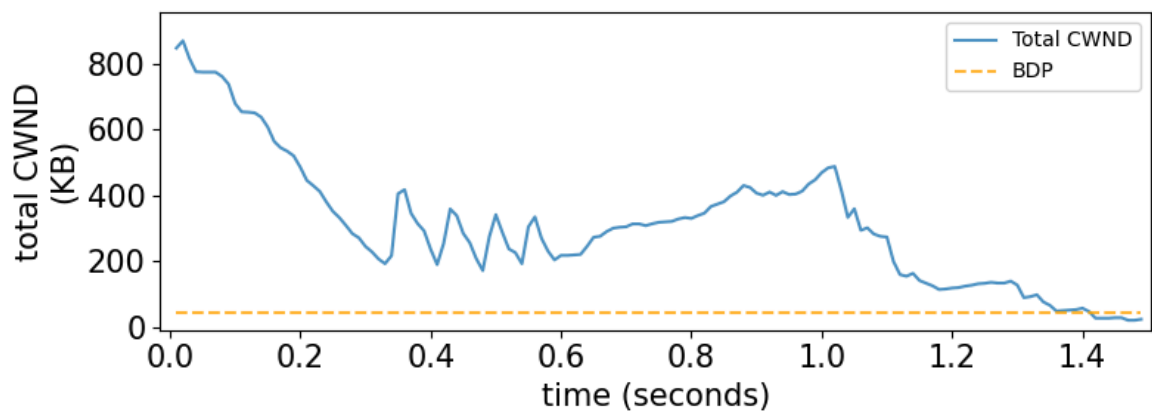
Figure



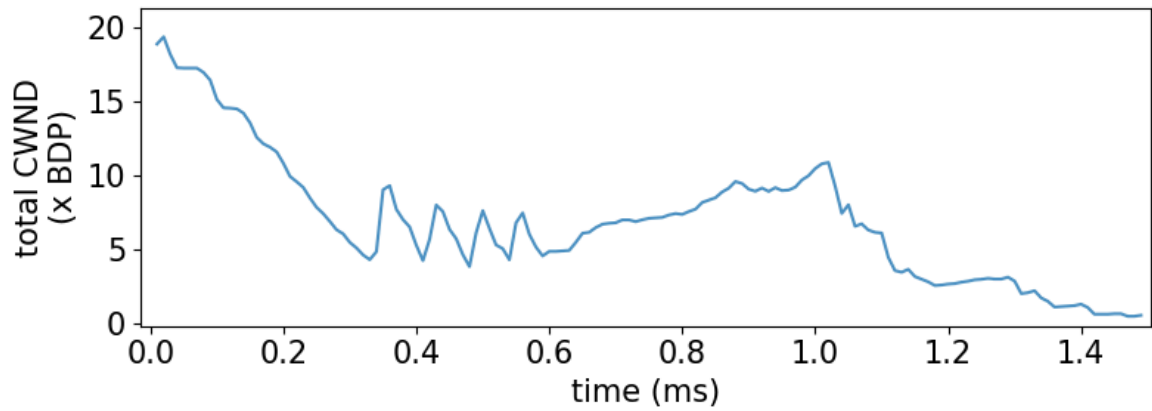
Figure



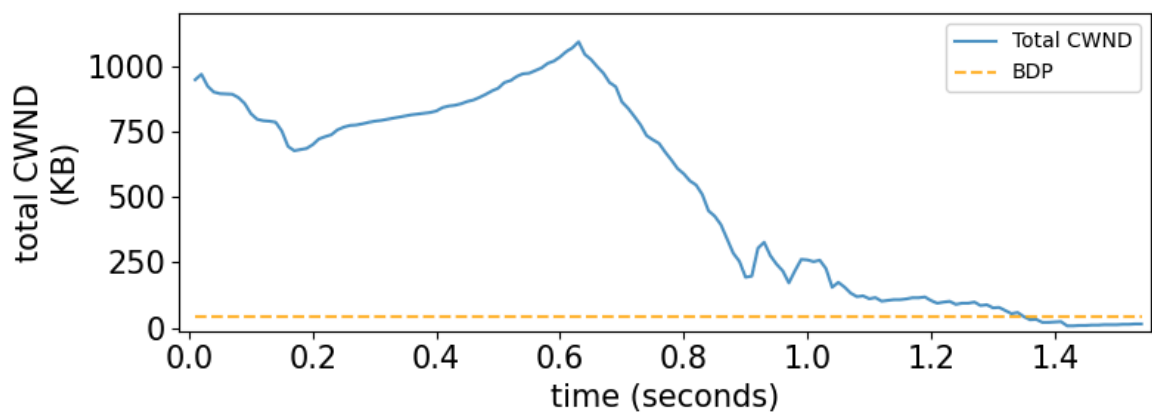
Figure



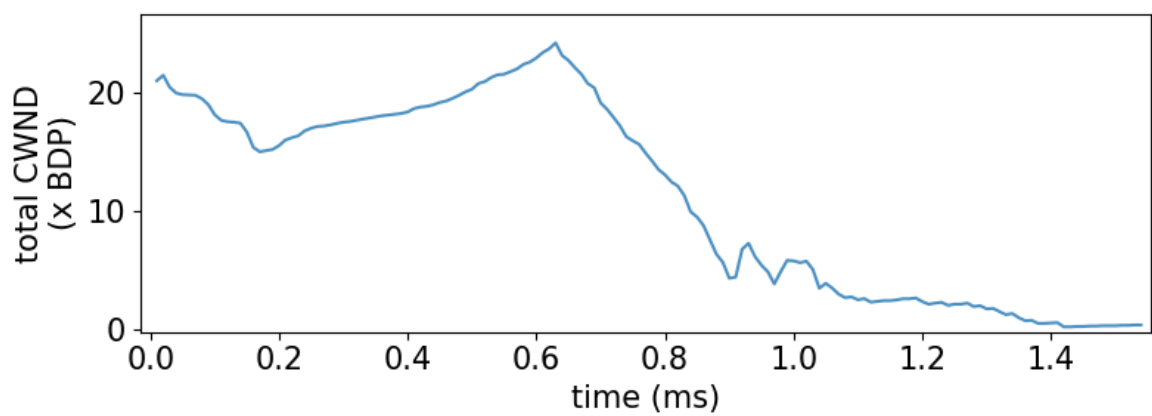
Figure



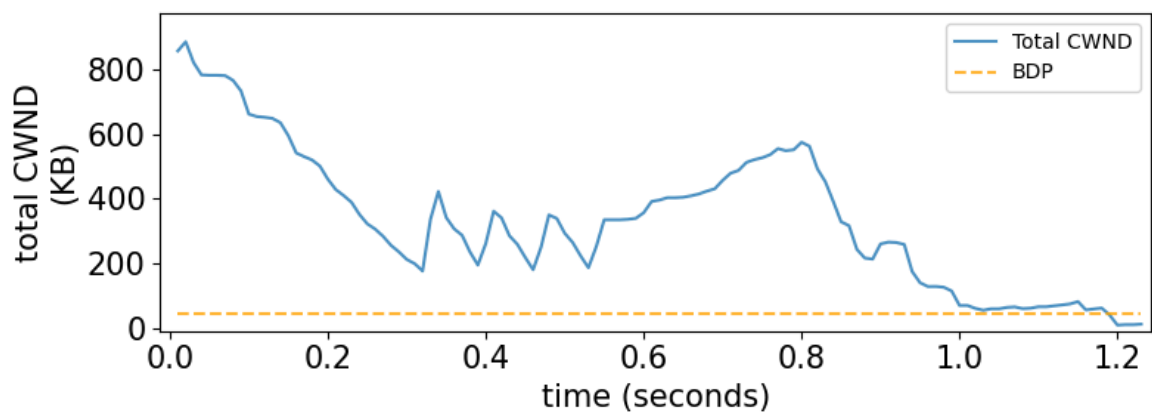
Figure



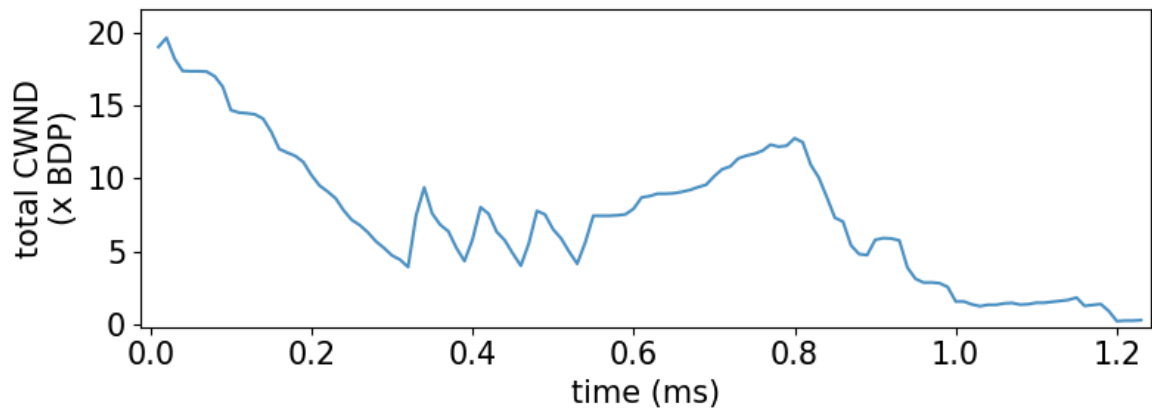
Figure



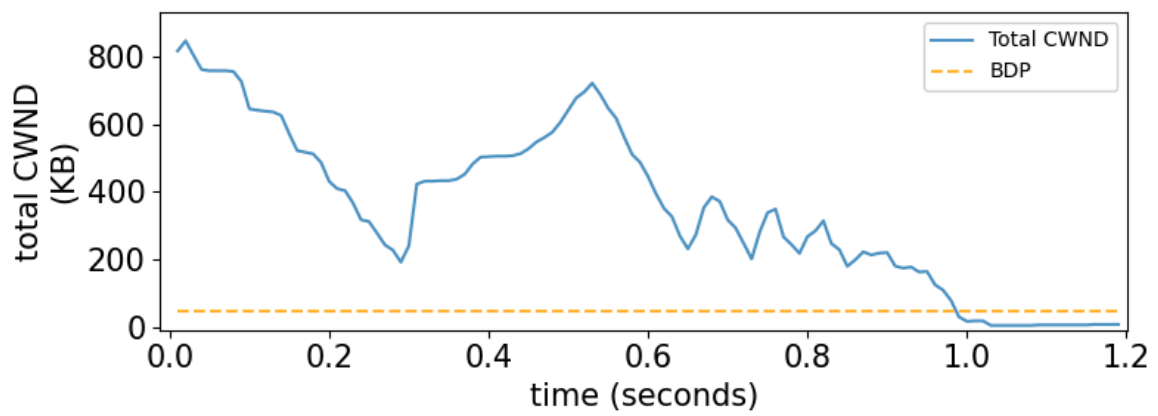
Figure



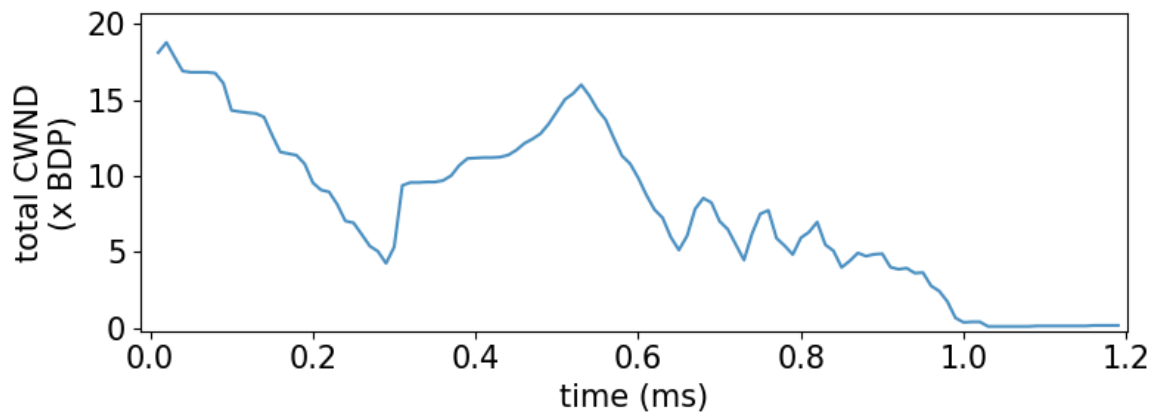
Figure



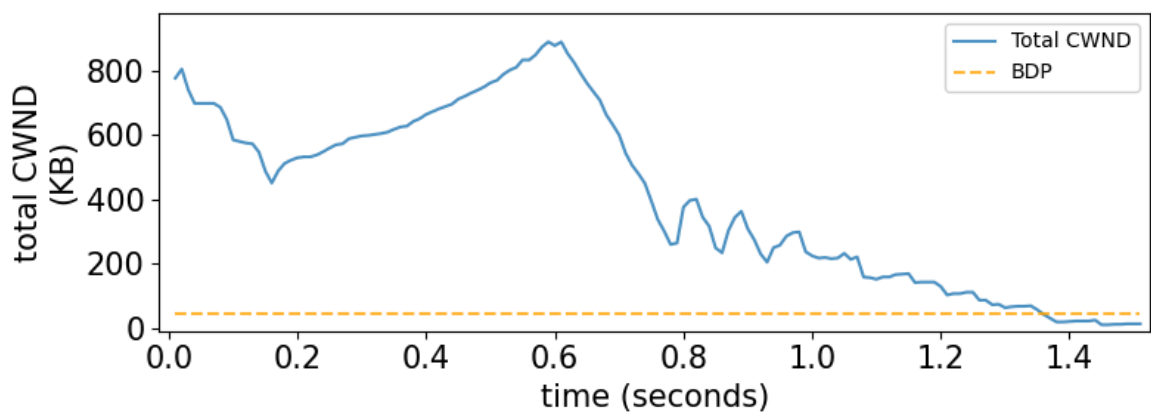
Figure



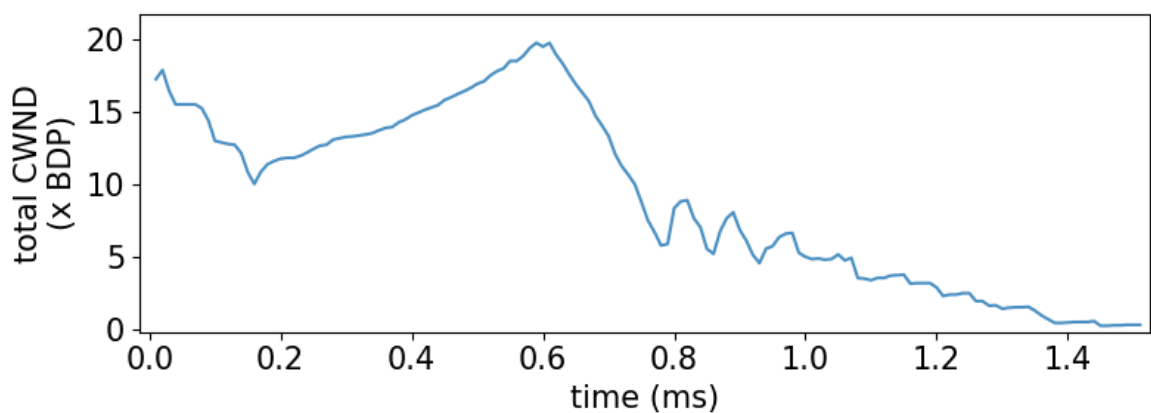
Figure



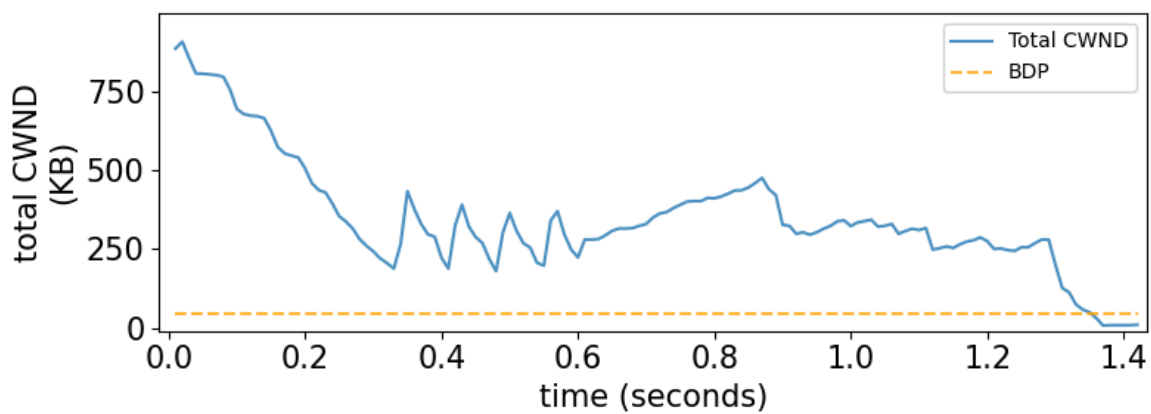
Figure



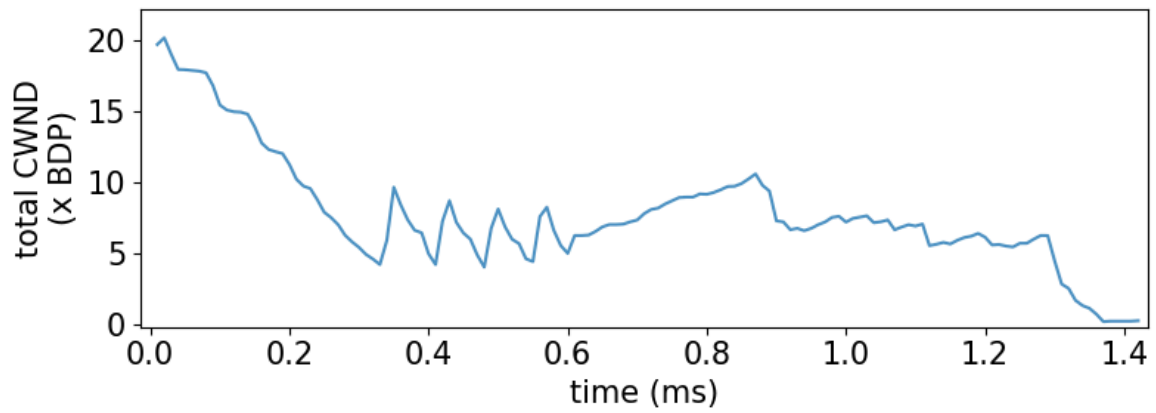
Figure



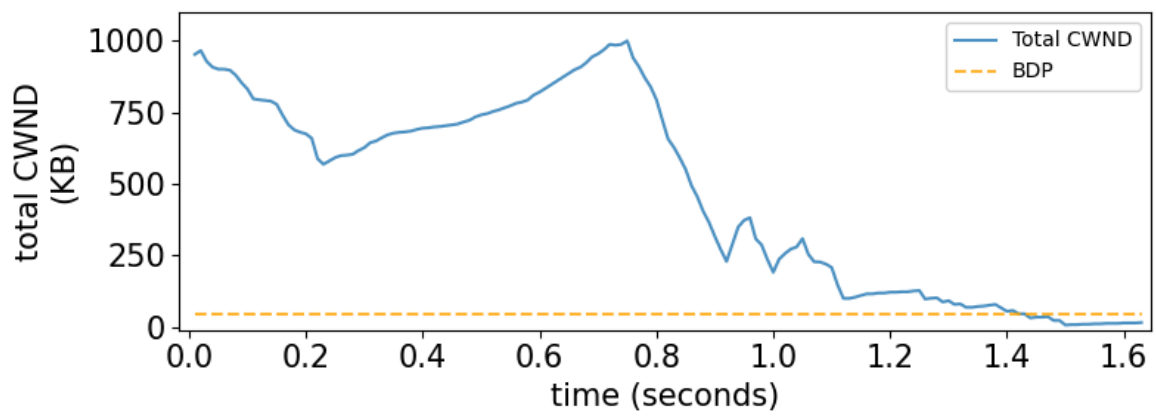
Figure



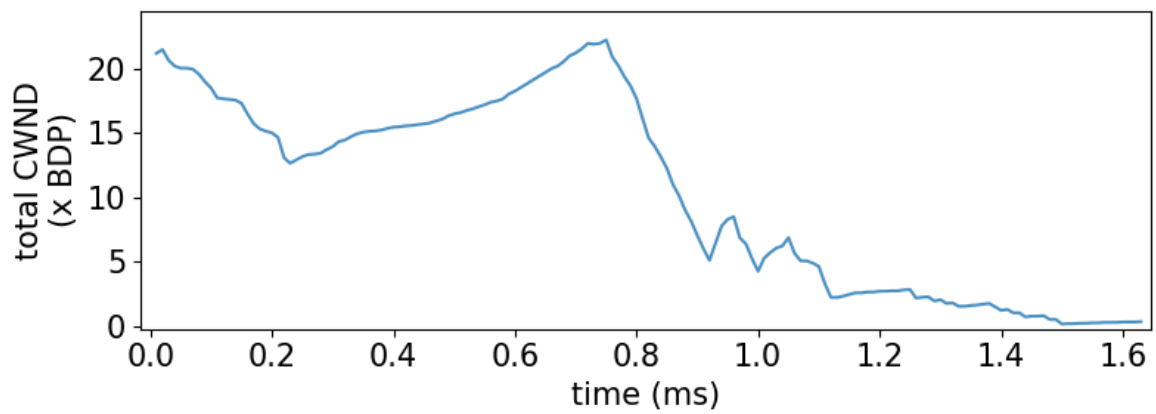
Figure



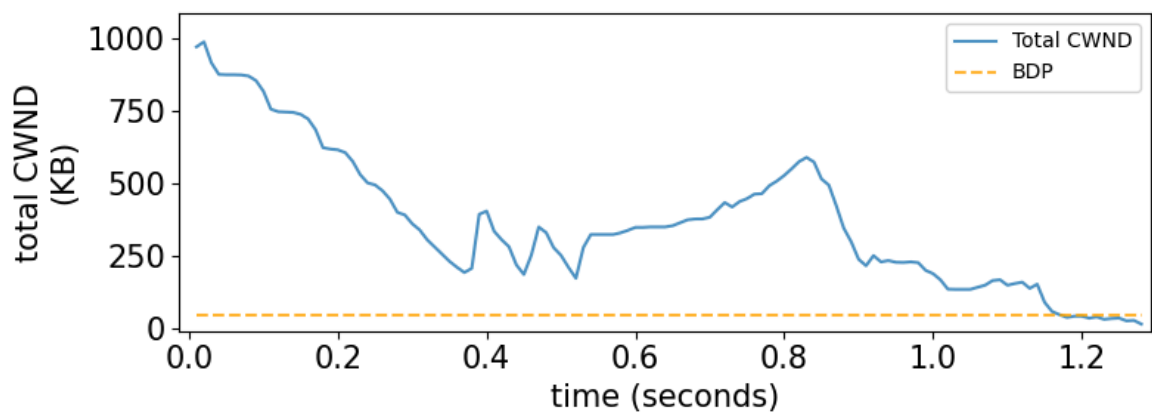
Figure



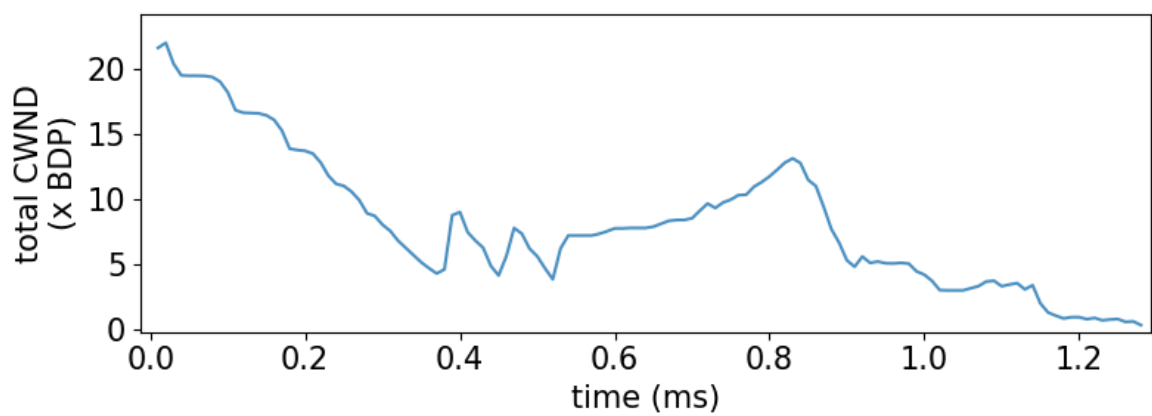
Figure



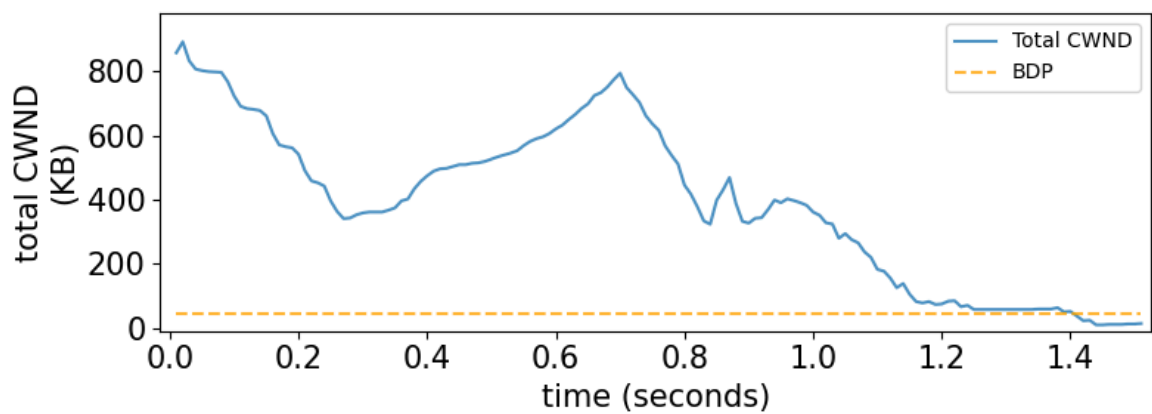
Figure



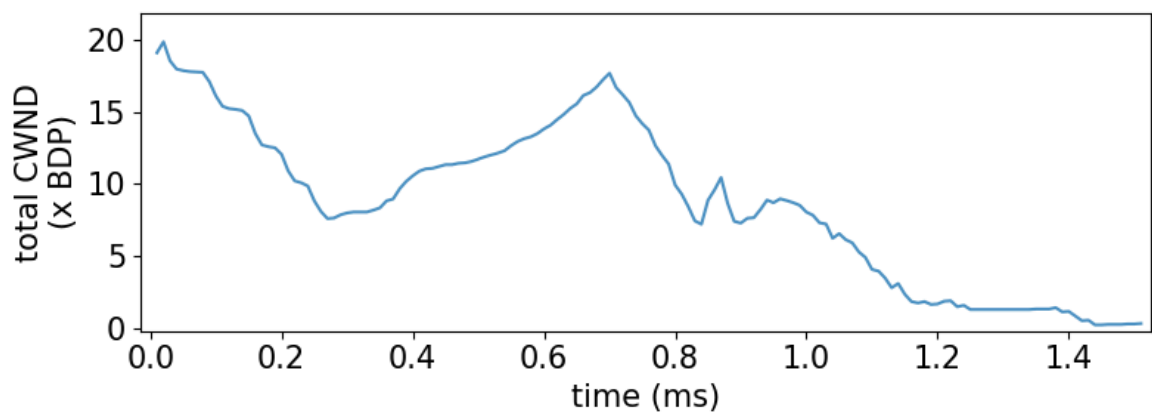
Figure



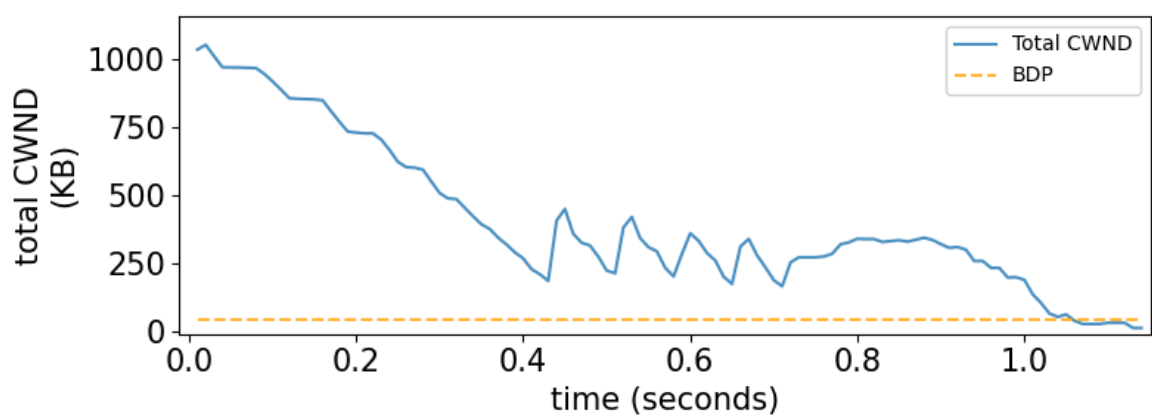
Figure



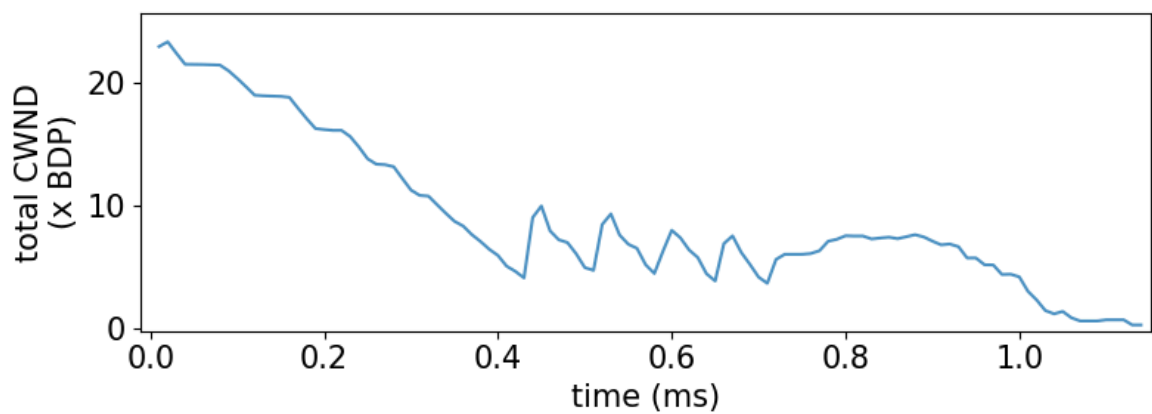
Figure



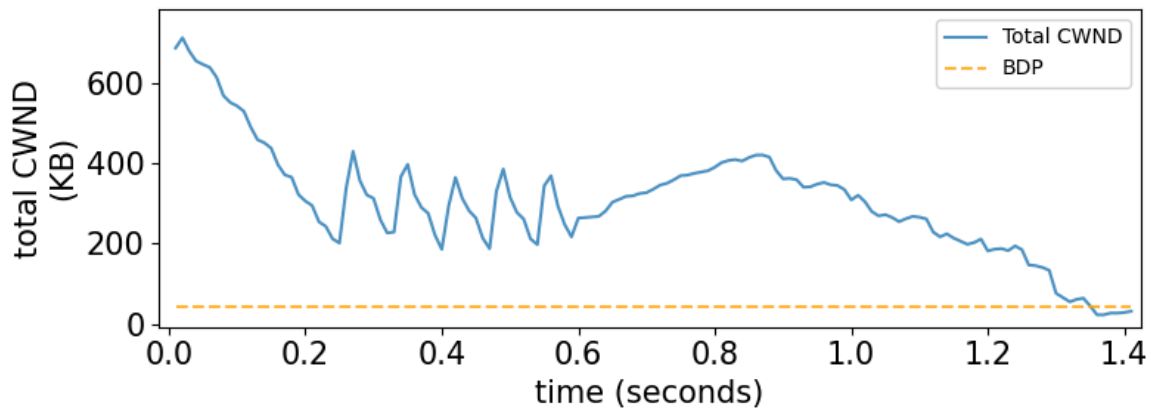
Figure



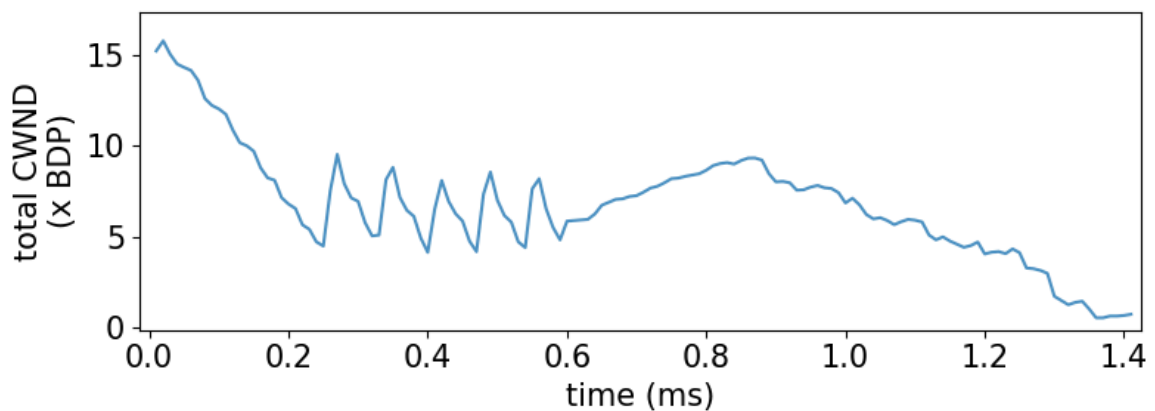
Figure



Figure



Figure



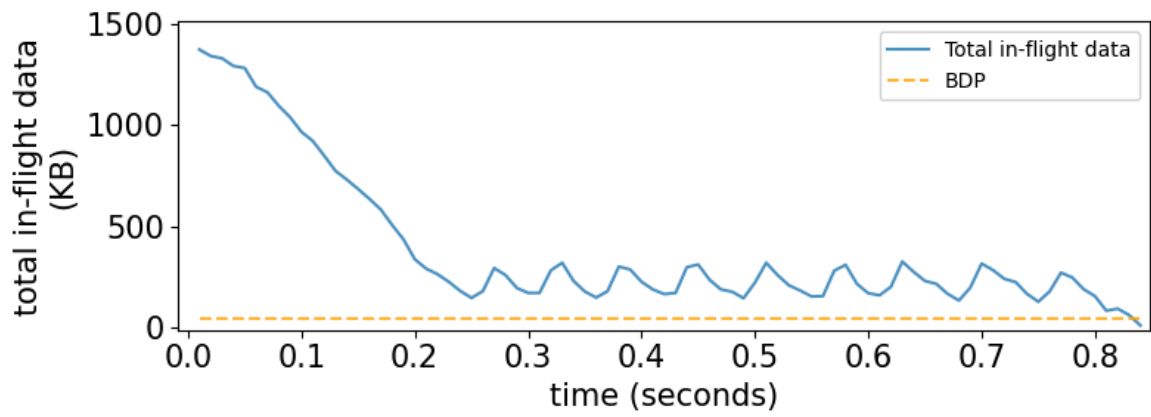
```
In [21]: def graph_total_inflight(
    inflight_metrics_by_burst,
    num_bursts,
    bdp_bytes,
    graph_dir,
    prefix,
):
    graph_total_cwnd(
        inflight_metrics_by_burst,
        num_bursts,
        bdp_bytes,
        graph_dir,
        prefix,
        ylabel="in-flight data",
        fln="inflight",
    )

    if RUN:
        graph_total_inflight(
            INFLIGHT_METRICS_BY_BURST,
            NUM_BURSTS,
            BDP_BYTES,
            GRAPH_DIR,
```

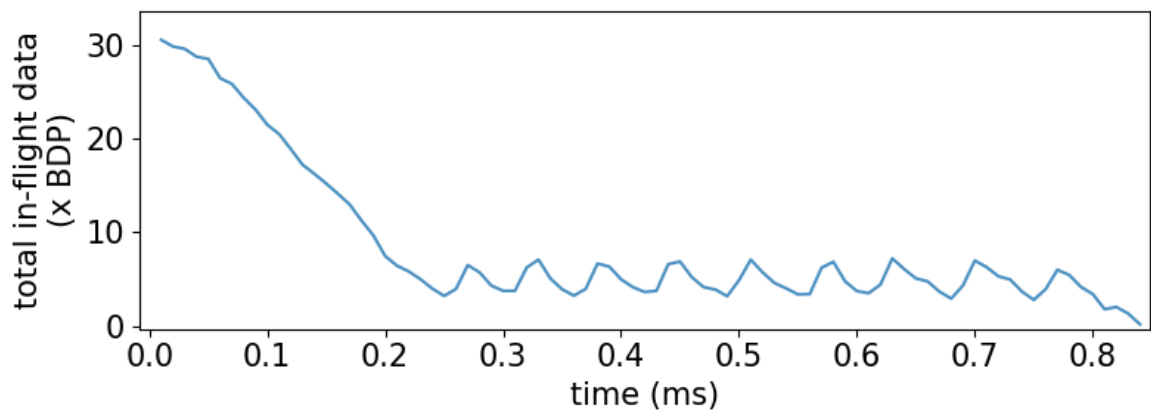
EXP,

)

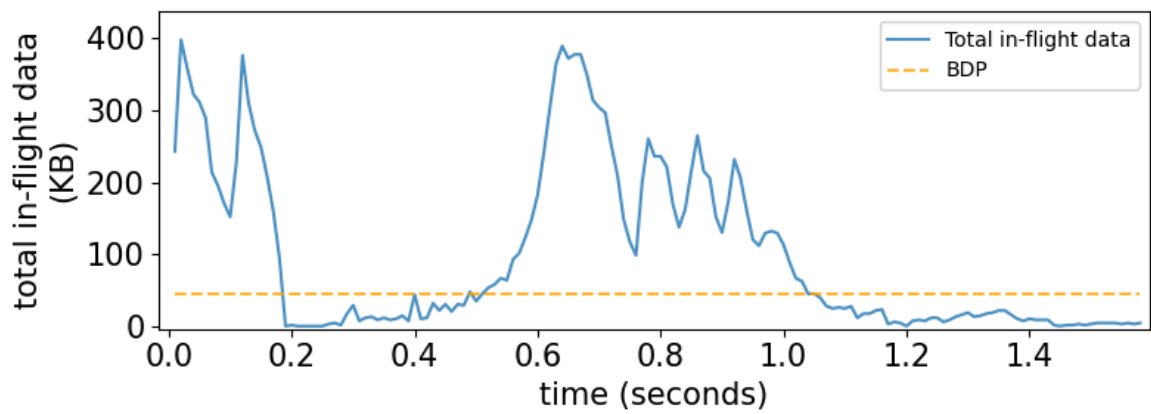
Figure



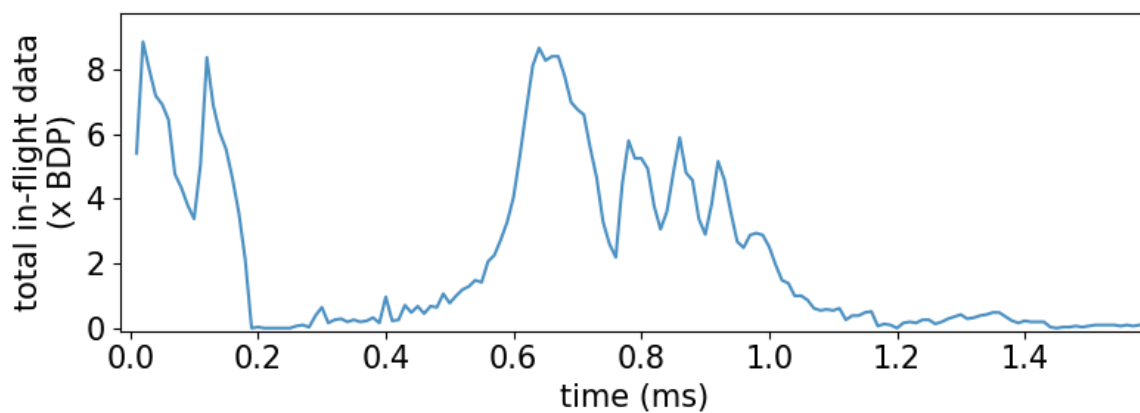
Figure



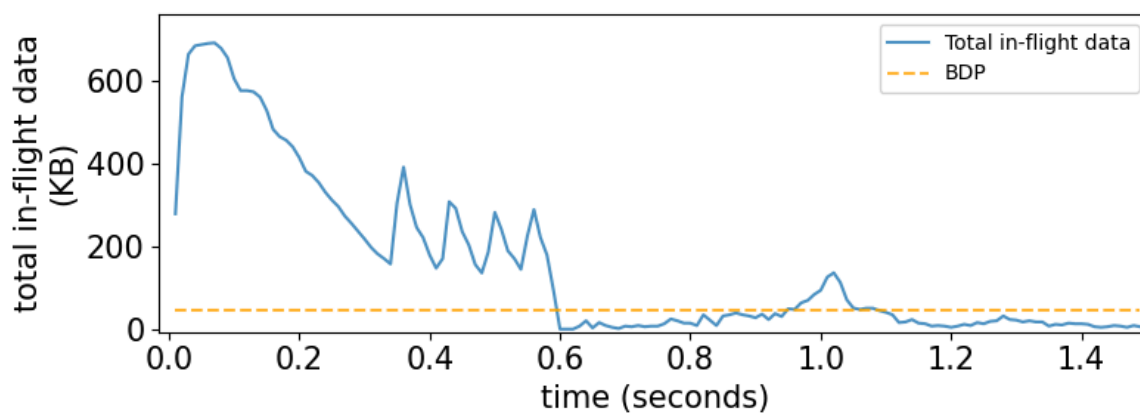
Figure



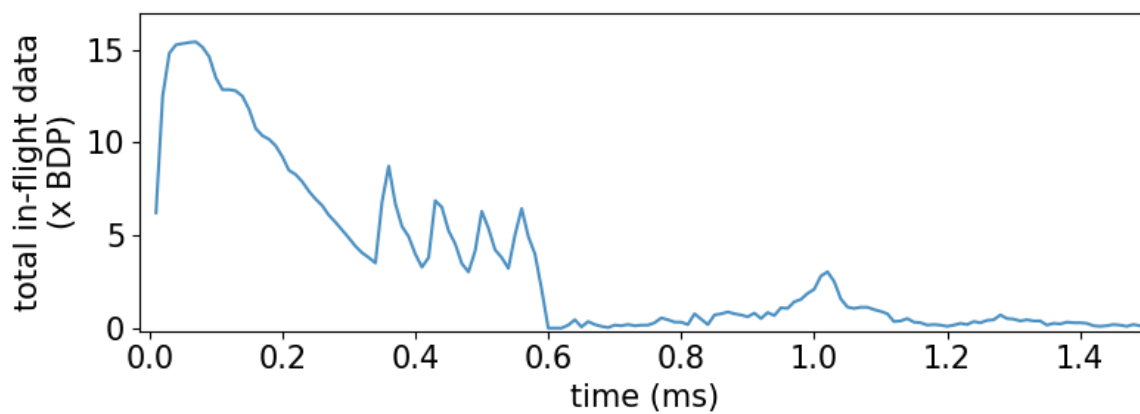
Figure



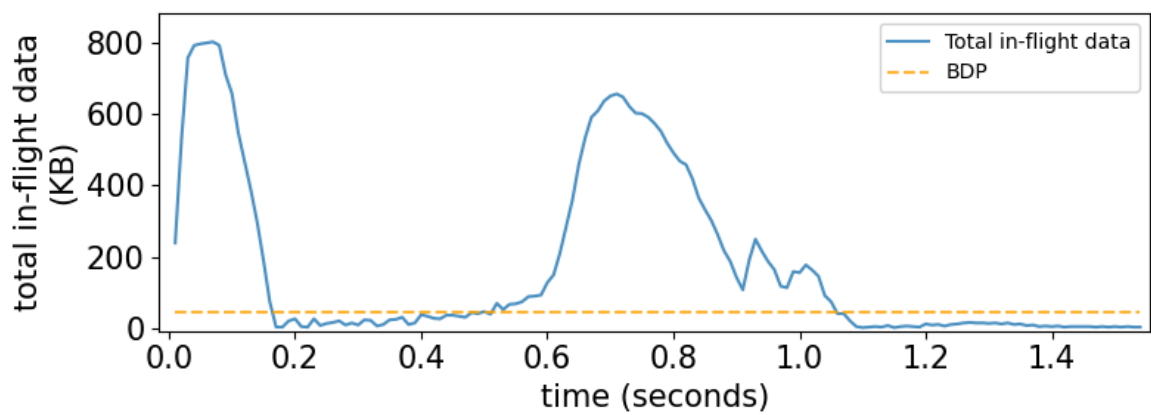
Figure



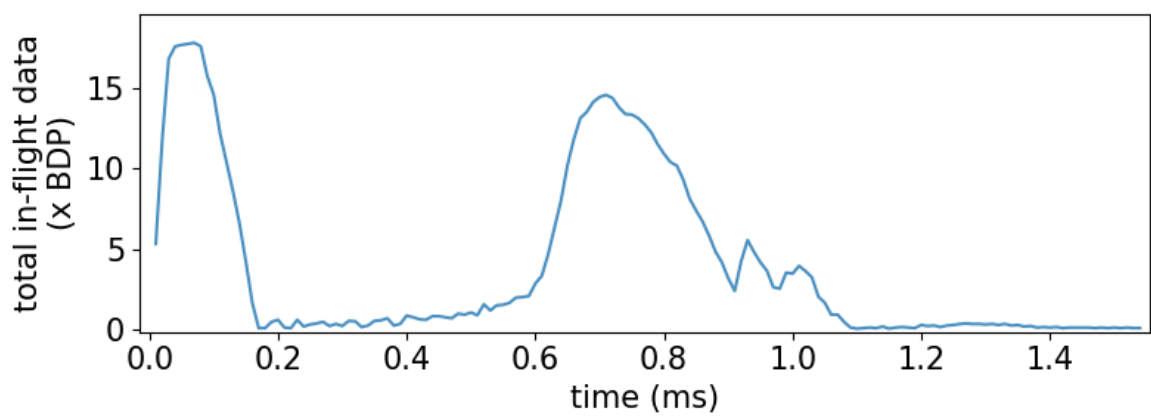
Figure



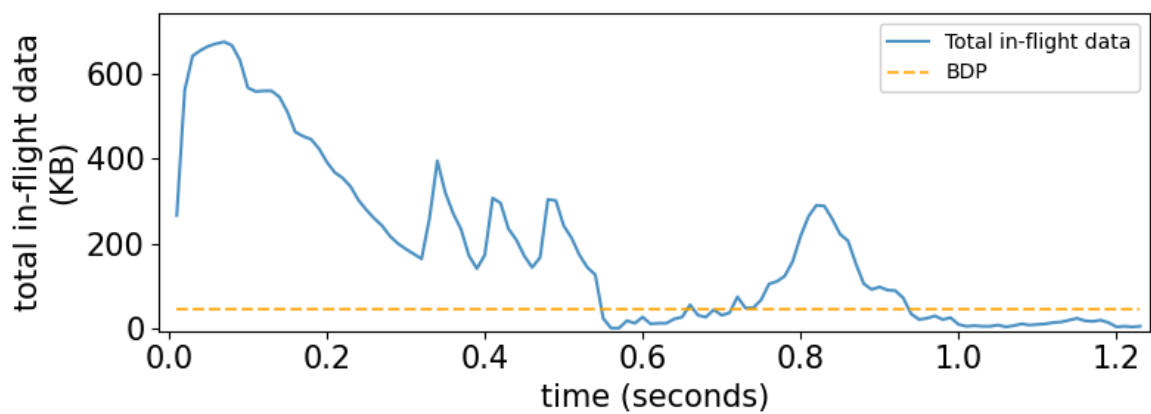
Figure



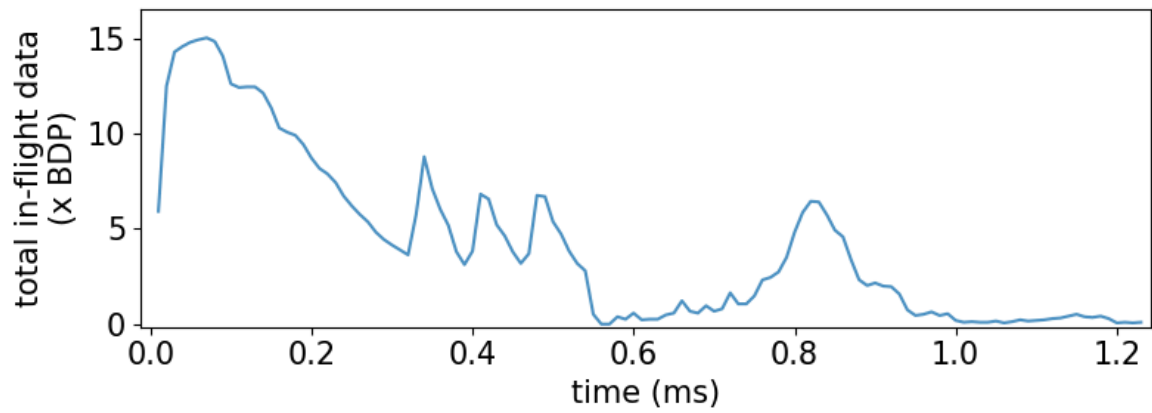
Figure



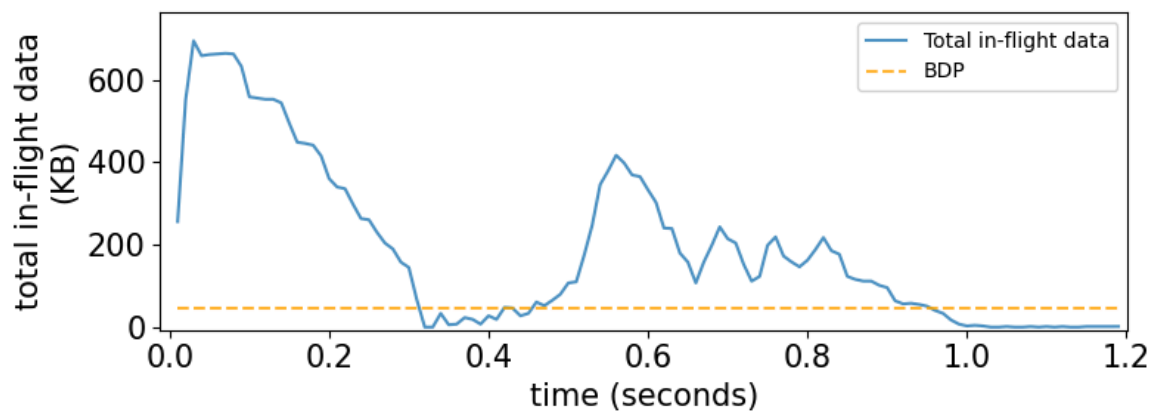
Figure



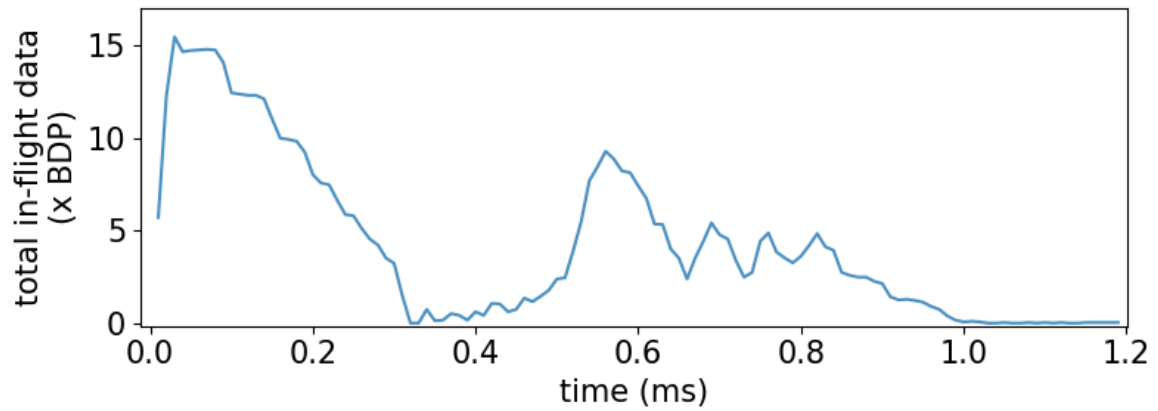
Figure



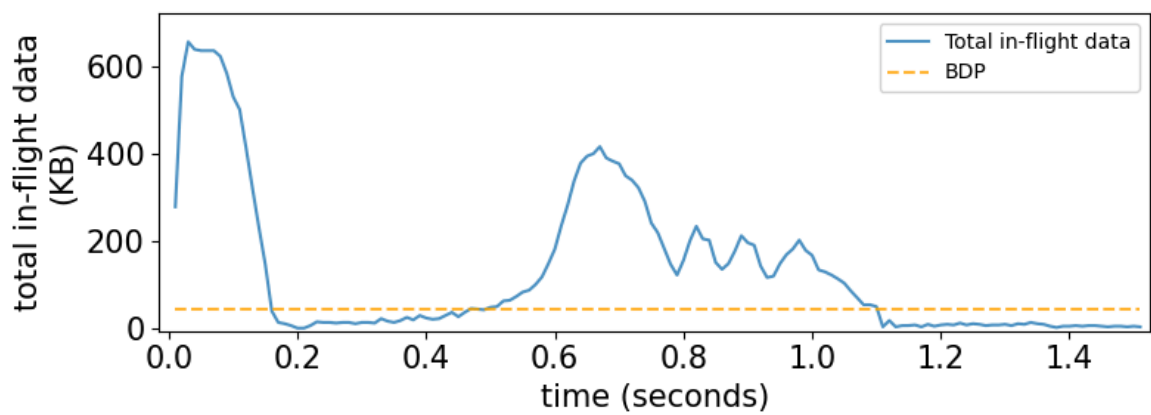
Figure



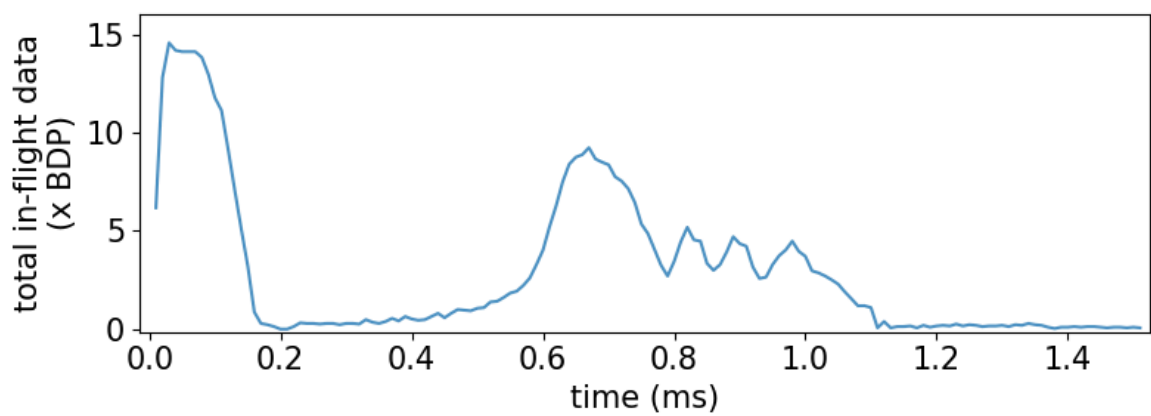
Figure



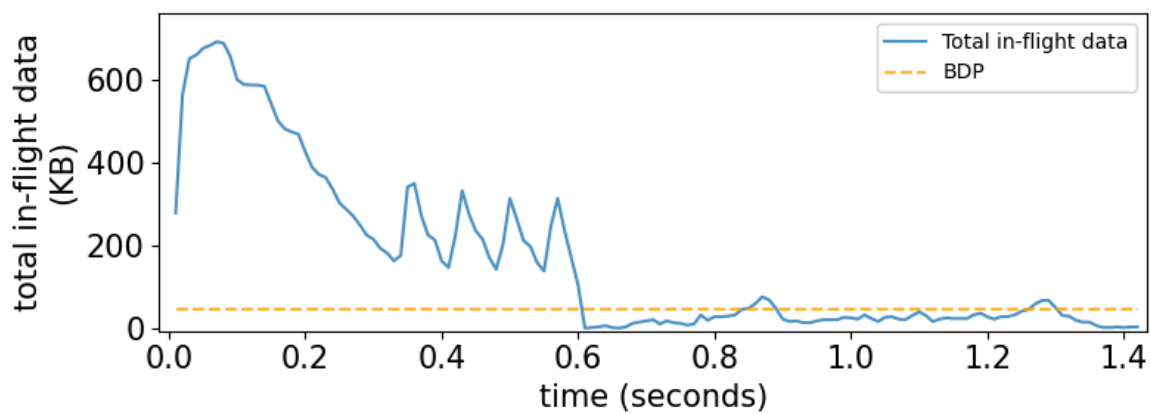
Figure



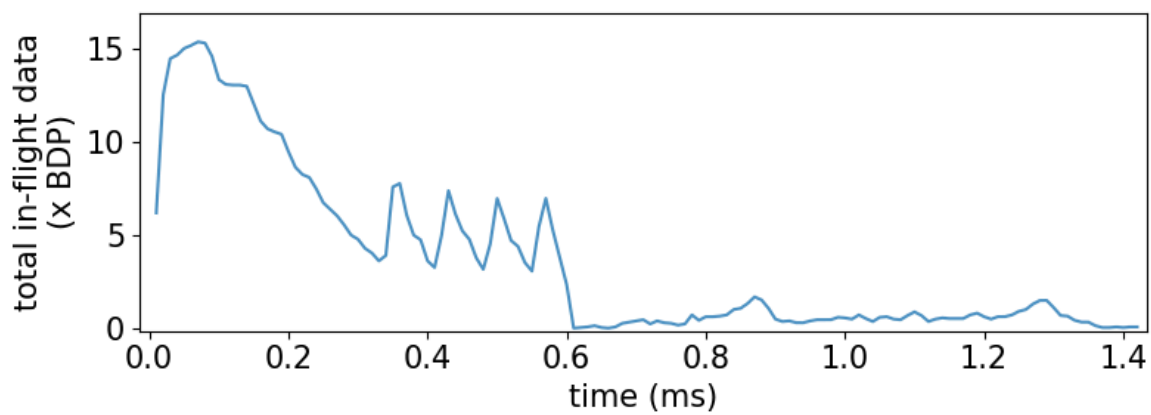
Figure



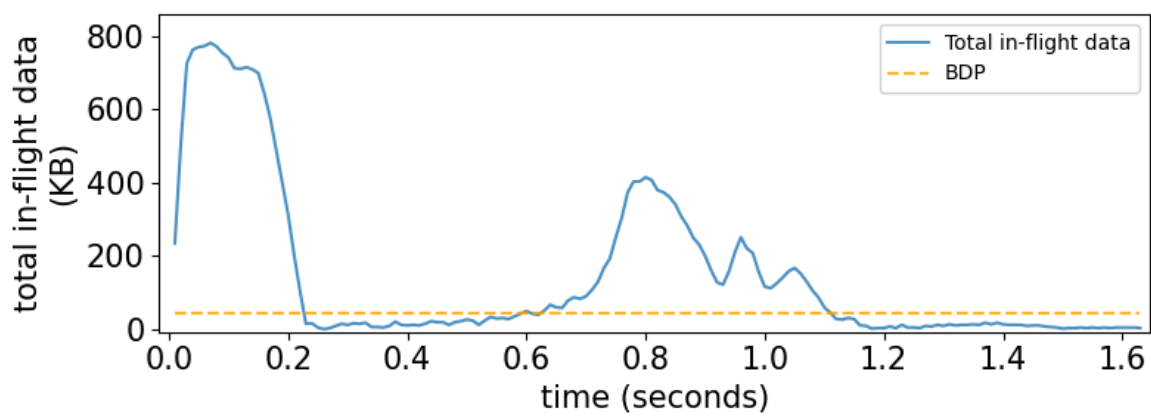
Figure



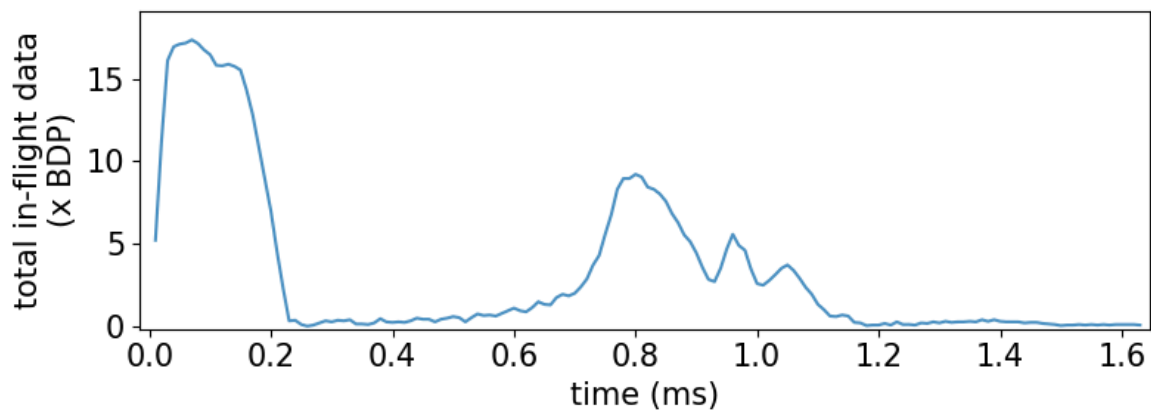
Figure



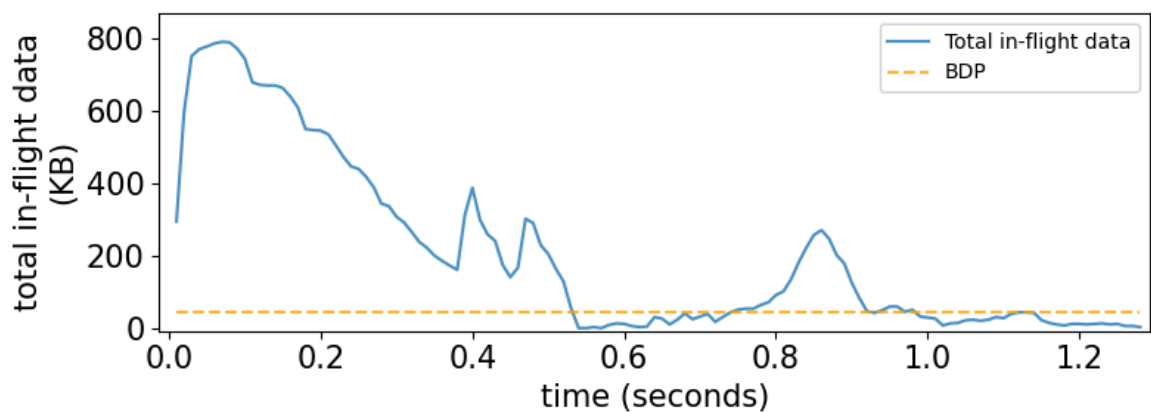
Figure



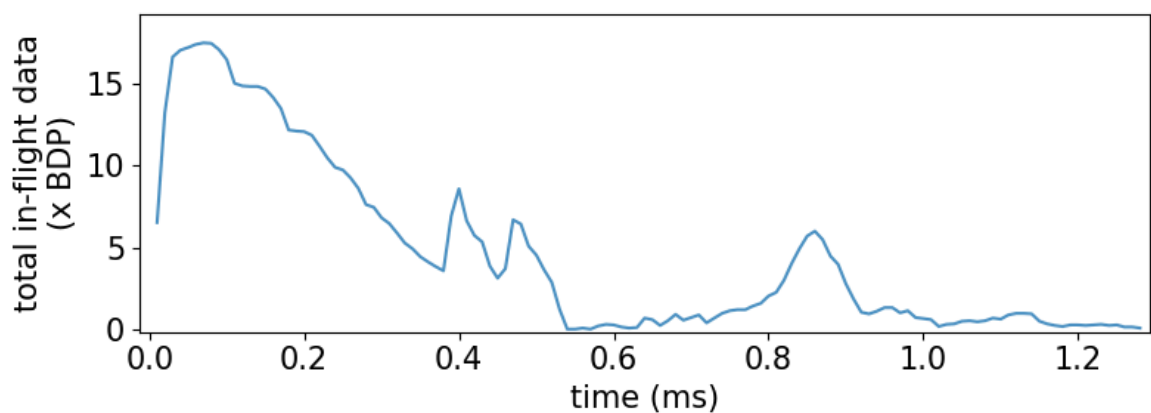
Figure



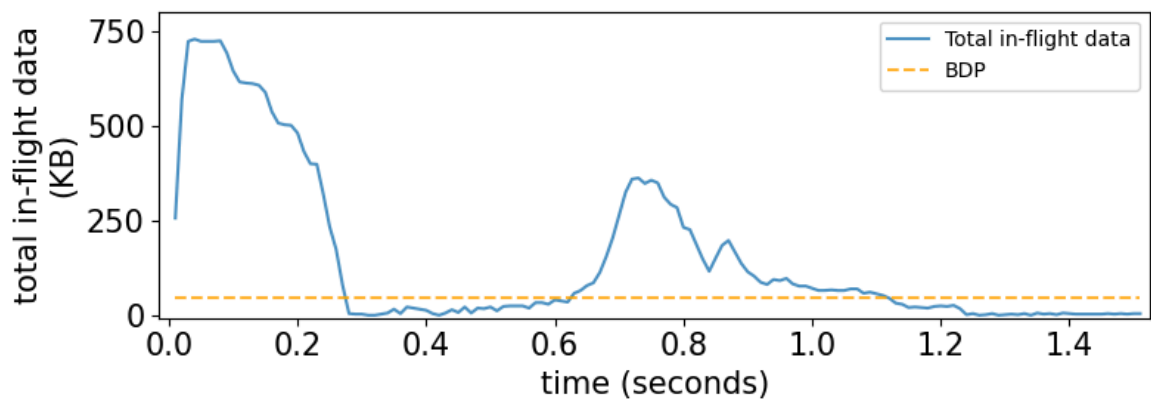
Figure



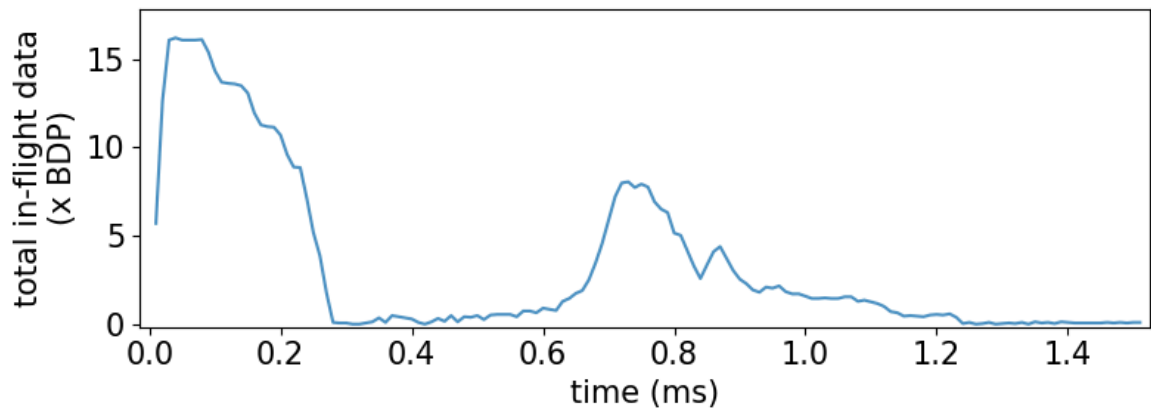
Figure



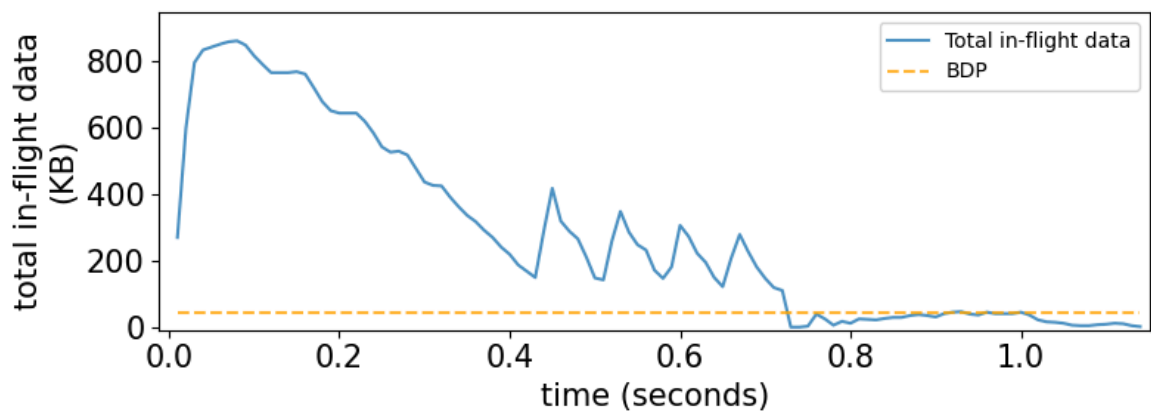
Figure



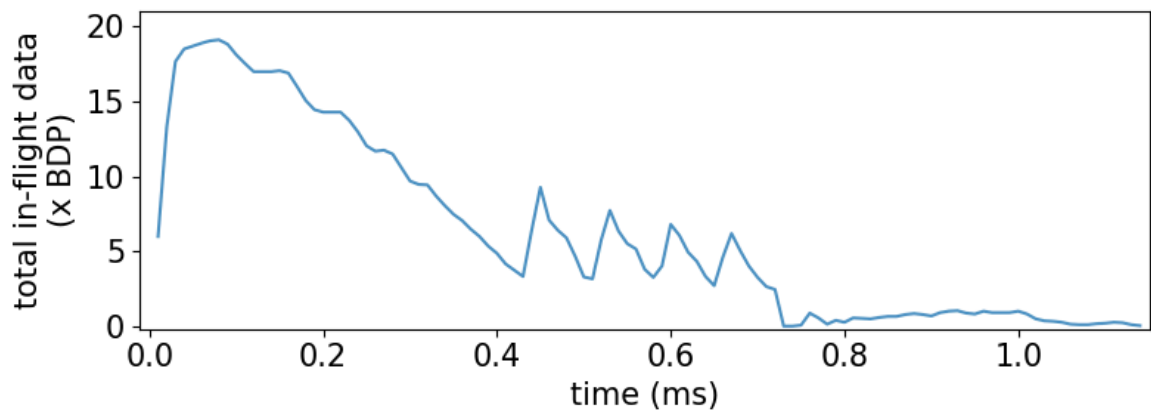
Figure



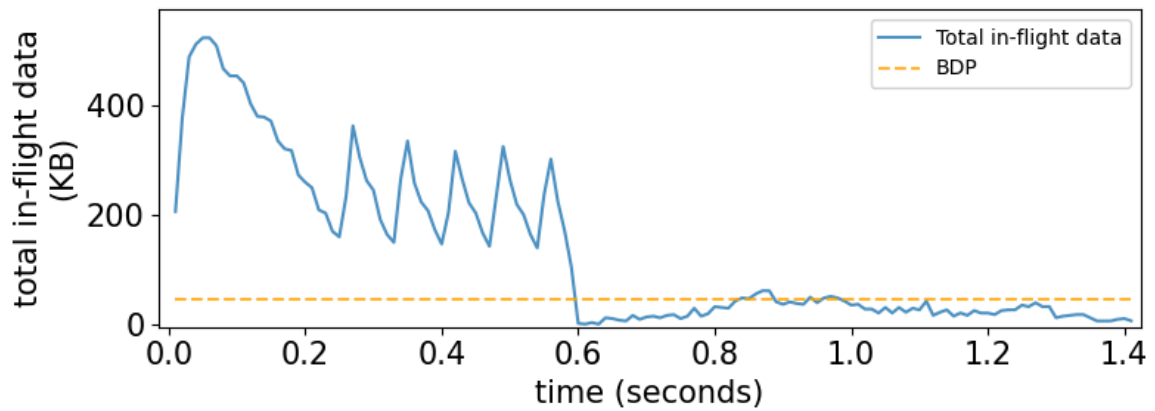
Figure



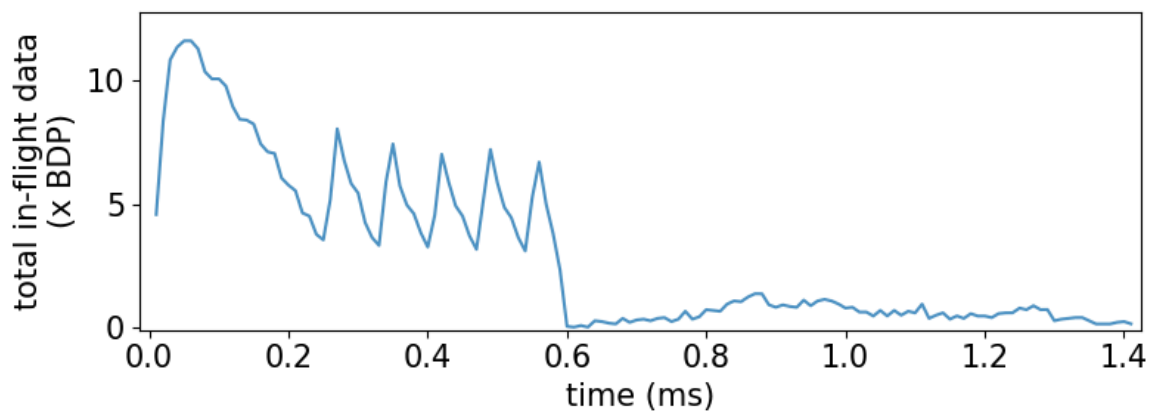
Figure



Figure



Figure



```
In [22]: def graph_cwnd_change_cdf(sender_to_cwnds_by_burst, num_bursts, graph_dir, p
    for burst_idx in range(num_bursts):
        fig, axes = get_axes(width=5)
        ax = axes[0]

        cwnd_down = []
        cwnd_up = []
        for sender_cwnds in sender_to_cwnds_by_burst.values():
            if len(sender_cwnds[burst_idx]) < 2:
                continue
            _, sender_cwnds_burst = zip(*sender_cwnds[burst_idx])
            # Compute percent difference
            cwnd_changes = np.diff(sender_cwnds_burst) / sender_cwnds_burst[0]
            # Filter based on whether increase or decrease
            cwnd_down.extend(abs(x) for x in cwnd_changes if x < 0)
            cwnd_up.extend(x for x in cwnd_changes if x > 0)

        # Remove inf values
        cwnd_down = np.asarray(cwnd_down)
        cwnd_down = cwnd_down[np.logical_not(np.isinf(cwnd_down))]
        cwnd_up = np.asarray(cwnd_up)
        cwnd_up = cwnd_up[np.logical_not(np.isinf(cwnd_up))]

        # Plot Cwnd decreases
        count, bins_count = np.histogram(cwnd_down, bins=len(cwnd_down))
```

```

ax.plot(
    bins_count[1:],
    np.cumsum(count / sum(count)),
    alpha=0.8,
    label="CWND decrease",
    color="red",
)

# Plot CWND increases
count, bins_count = np.histogram(cwnd_up, bins=len(cwnd_up))
ax.plot(
    bins_count[1:],
    np.cumsum(count / sum(count)),
    alpha=0.8,
    linestyle="dashed",
    label="CWND increase",
    color="green",
)

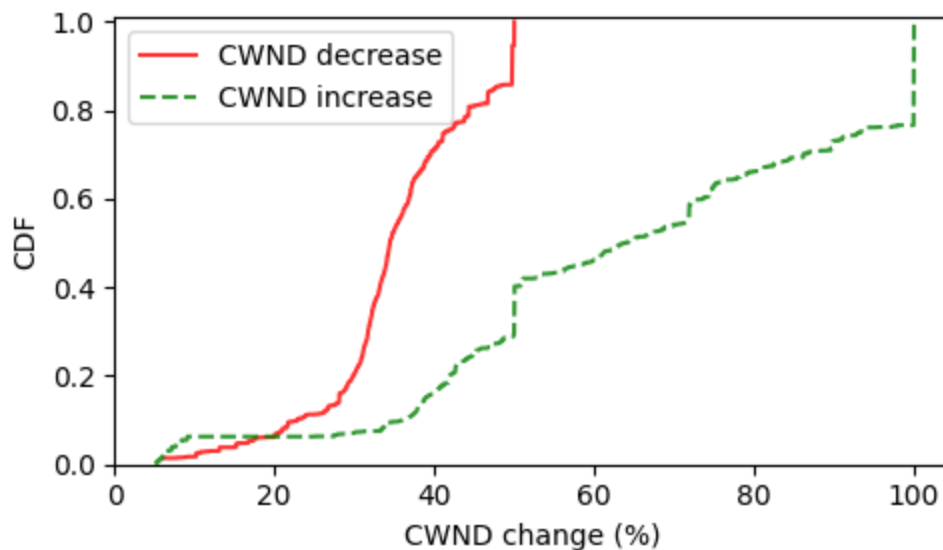
# ax.set_title(f"CDF of CWND change (%): Burst {burst_idx + 1} of {r
ax.set_xlabel("CWND change (%)")
ax.set_ylabel("CDF")
ax.set_xlim(left=0)
ax.set_ylim(bottom=0, top=1.01)
ax.legend()

show(fig)
save(graph_dir, prefix, suffix=f"cwnd_change_cdf_{burst_idx}")

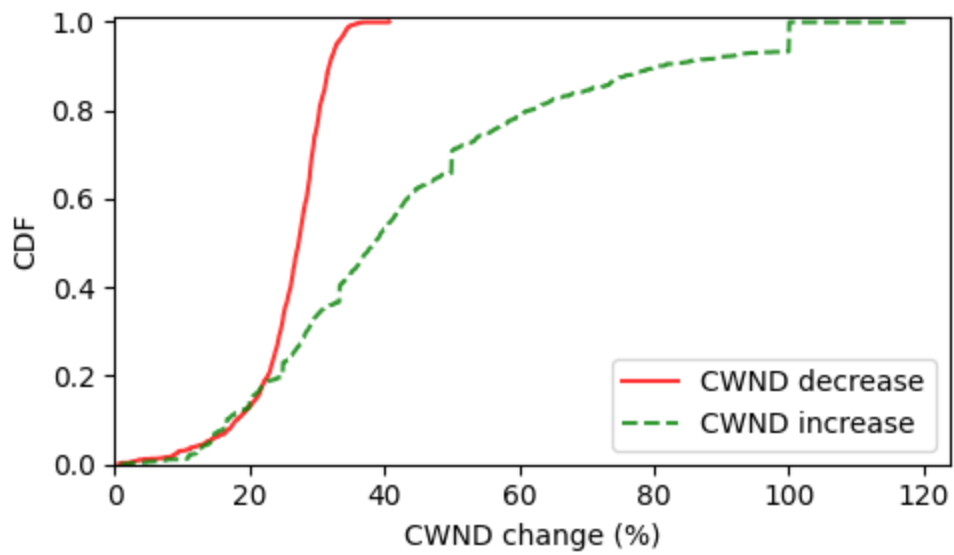
if RUN:
    graph_cwnd_change_cdf(SENDER_TO_CWNS_BY_BURST, NUM_BURSTS, GRAPH_DIR, E

```

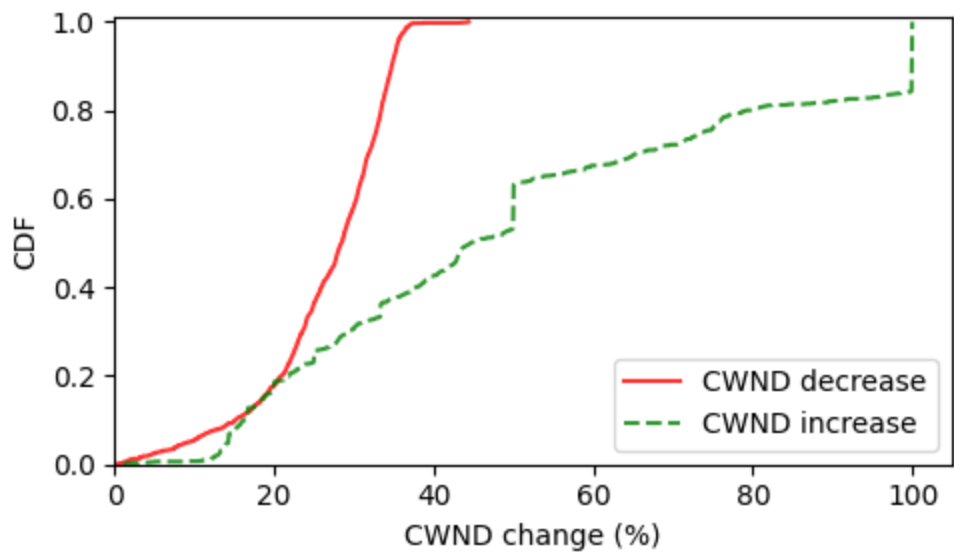
Figure



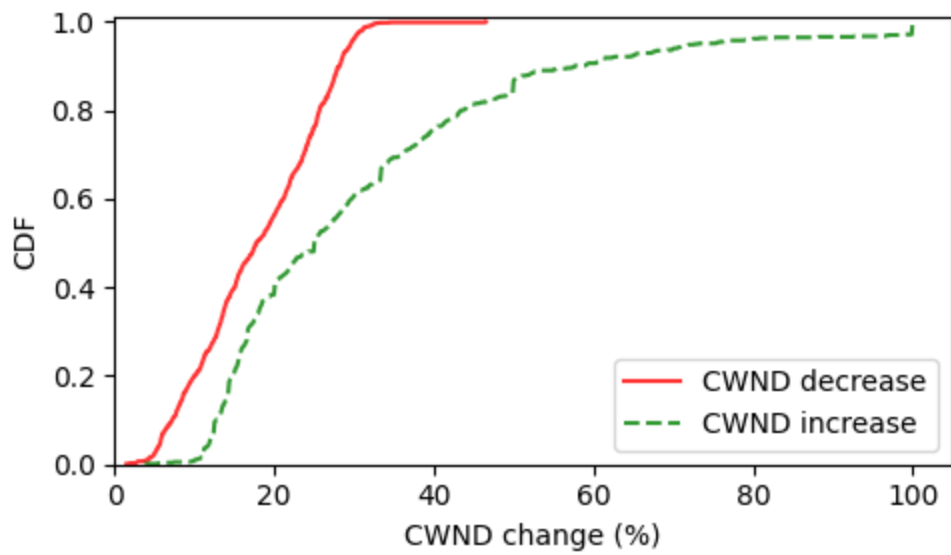
Figure



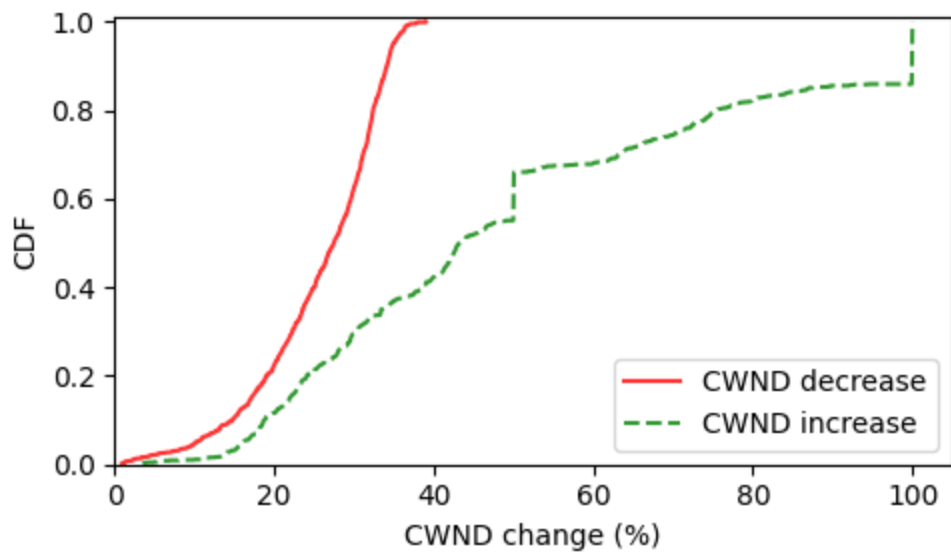
Figure



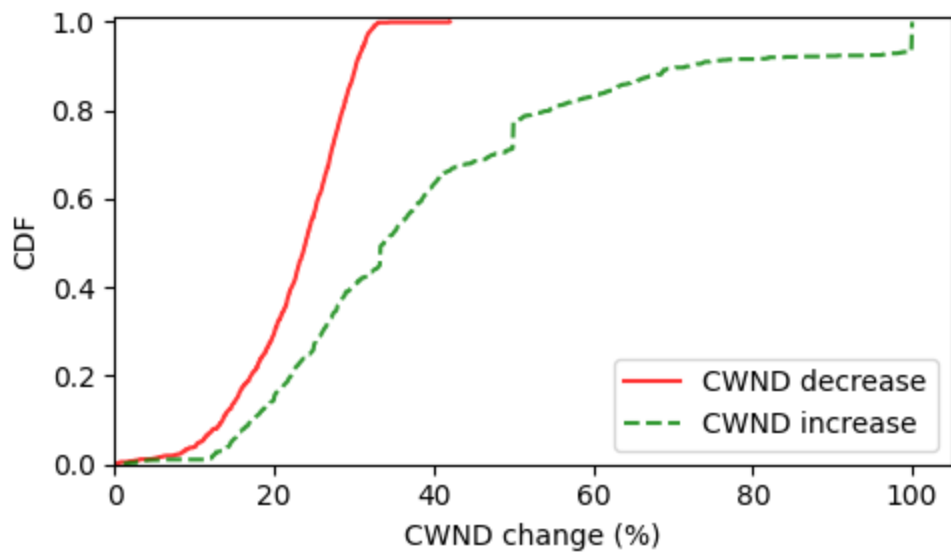
Figure



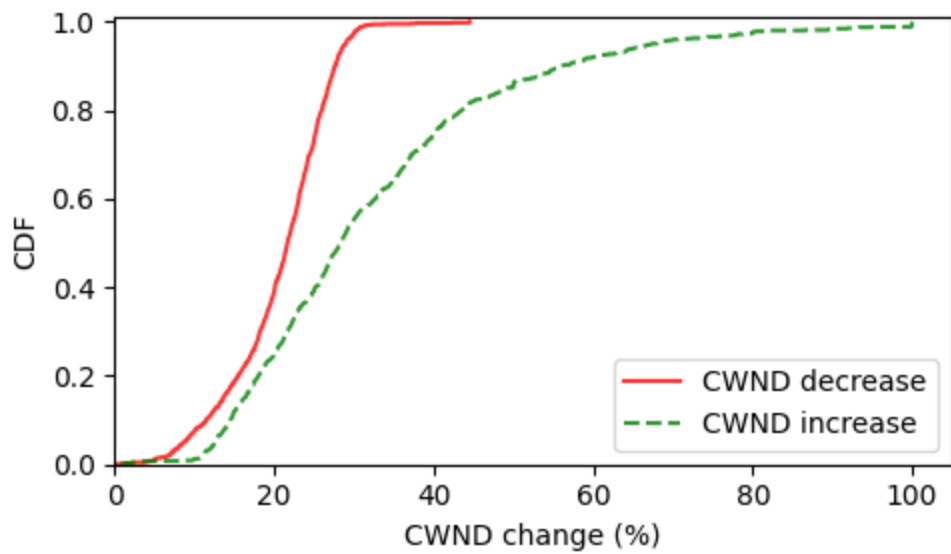
Figure



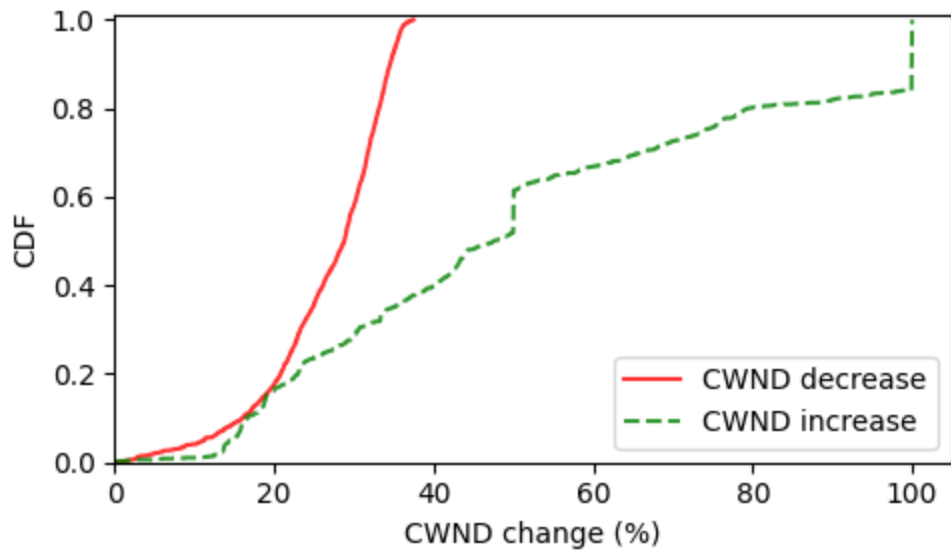
Figure



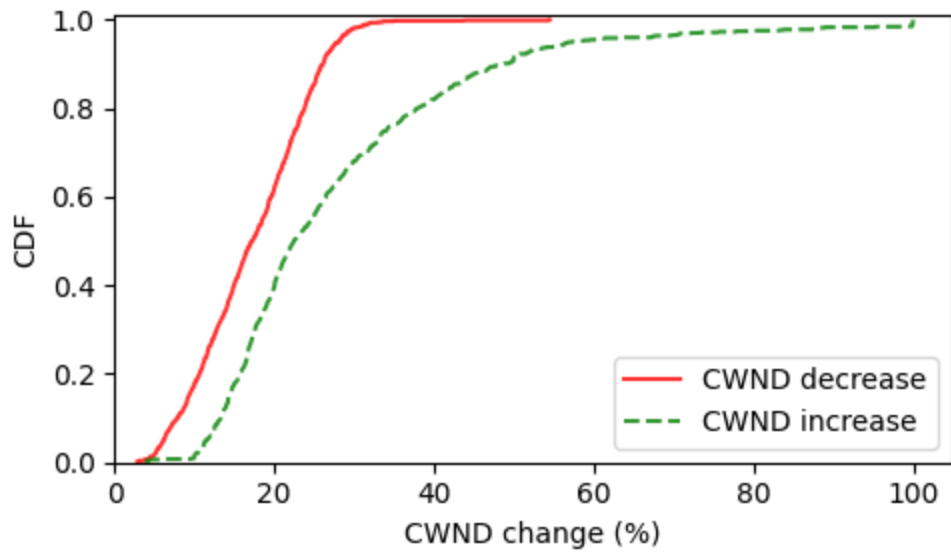
Figure



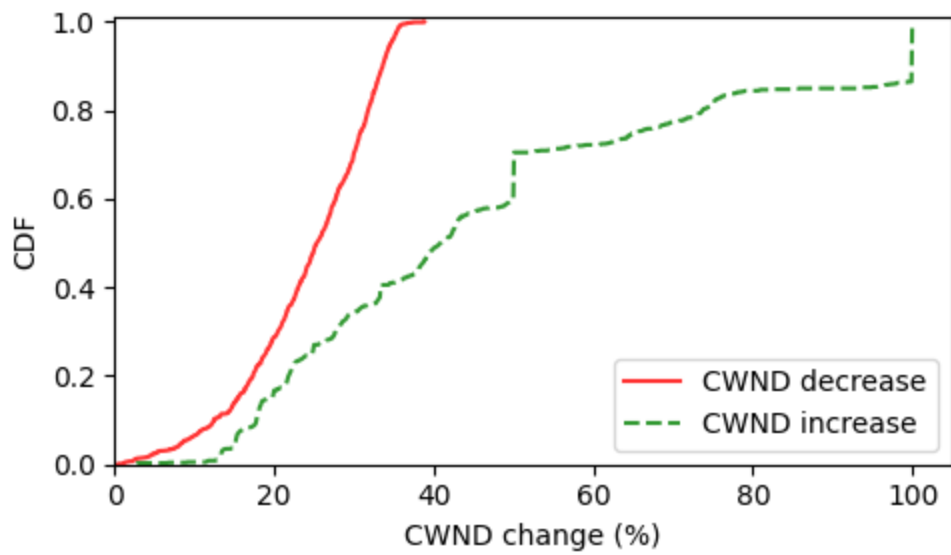
Figure



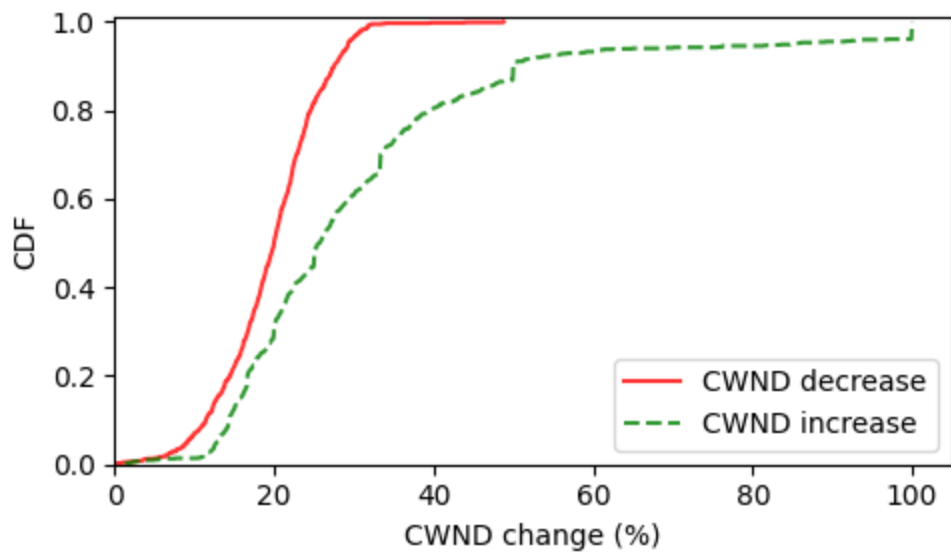
Figure



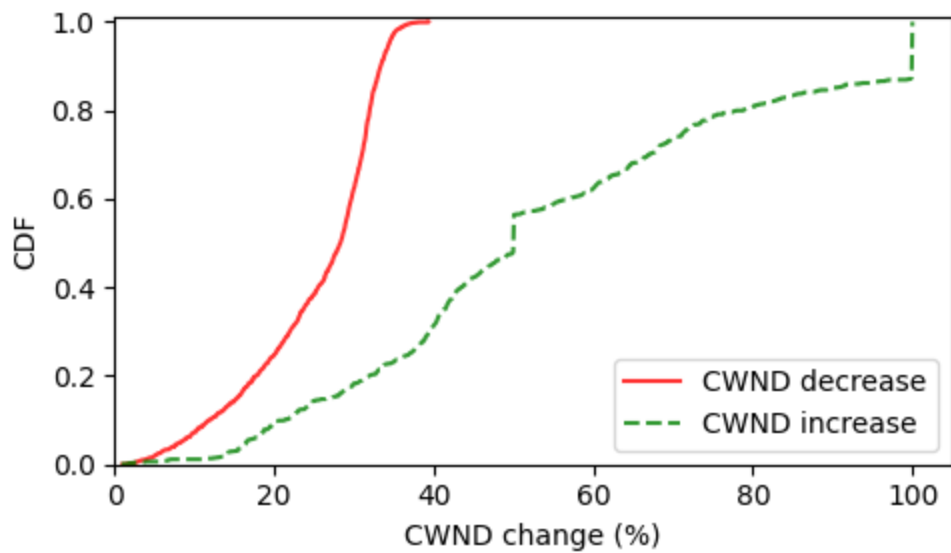
Figure



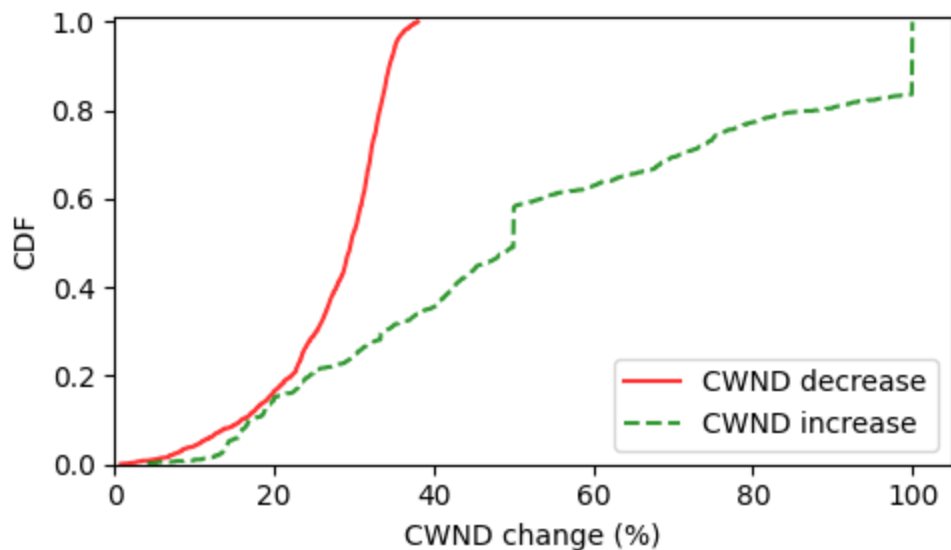
Figure



Figure



Figure



```
In [23]: def parse_congest_line(line):
    parts = line.strip().split(" ")
    assert len(parts) == 4
    time_sec, acked_ecn_bytes, total_acked_bytes, alpha = parts
    time_sec = float(time_sec)
    acked_ecn_bytes = int(acked_ecn_bytes)
    total_acked_bytes = int(total_acked_bytes)
    alpha = float(alpha)
    return time_sec, acked_ecn_bytes, total_acked_bytes, alpha

def parse_congest(flp):
    with open(flp, "r", encoding="utf-8") as fil:
        return [parse_congest_line(line) for line in fil if line.strip()[0]
```

```

def get_sender_to_congest_by_burst(exp_dir, burst_times, sender_to_flow_time):
    return {
        parse_sender(flp): separate_samples_into_bursts(
            # Read all congestion estimate samples for this sender
            parse_congest(flp),
            burst_times,
            # Look up the start and end time for this sender
            sender_to_flow_times_by_burst[parse_sender(flp)],
            filter_on_flow_times=True,
            bookend=True,
        )
        # Look up all congestion estimate log files.
    }
    for flp in [
        path.join(exp_dir, "logs", fln)
        for fln in os.listdir(path.join(exp_dir, "logs"))
        if fln.startswith("sender") and fln.endswith("_congest.log")
    ]
}

def graph_sender_dctcp_alpha(sender_to_congest_by_burst, num_bursts, graph_dir):
    for burst_idx in range(num_bursts):
        fig, axes = get_axes()
        ax = axes[0]

        # ax.set_title(
        #     f"Alpha of active flows: Burst {burst_idx + 1} of {num_bursts}"
        # )
        ax.set_xlabel("time (seconds)")
        ax.set_ylabel("alpha")

        for sender, bursts in sender_to_congest_by_burst.items():
            if not bursts[burst_idx]:
                continue
            xs, _, _, ys = zip(*bursts[burst_idx])
            ax.plot(
                xs,
                ys,
                "o",
                markersize=1.5,
                label=sender,
                alpha=0.8,
            )

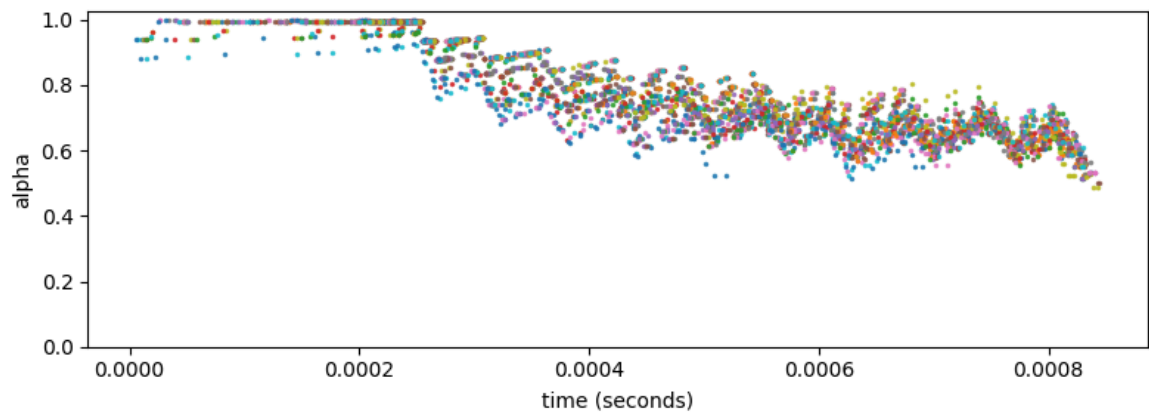
        ax.set_ylim(bottom=0)

        show(fig)
        save(graph_dir, prefix, suffix=f"dctcp_alpha_{burst_idx}")

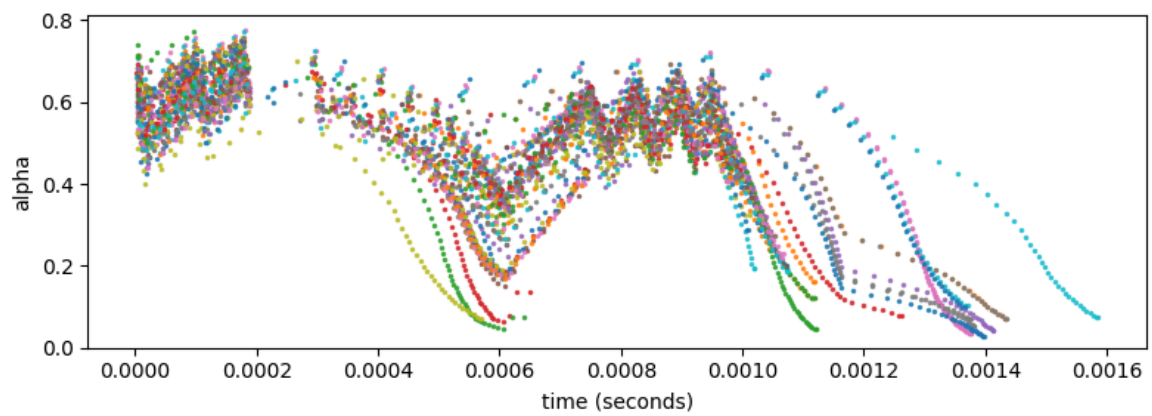
if RUN:
    SENDER_TO_CONGEST_BY_BURST = get_sender_to_congest_by_burst(
        EXP_DIR, BURST_TIMES, SENDER_TO_FLOW_TIMES_BY_BURST
    )
    graph_sender_dctcp_alpha(SENDER_TO_CONGEST_BY_BURST, NUM_BURSTS, GRAPH_DIR)

```

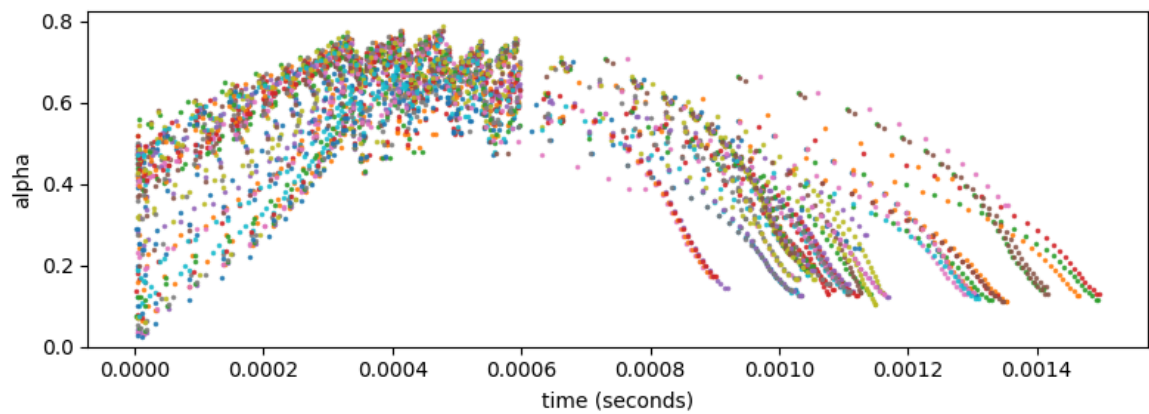
Figure



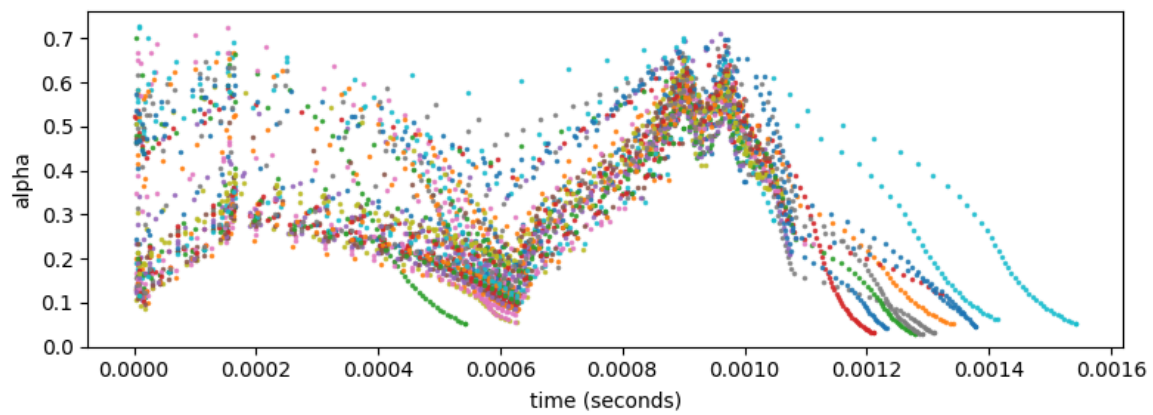
Figure



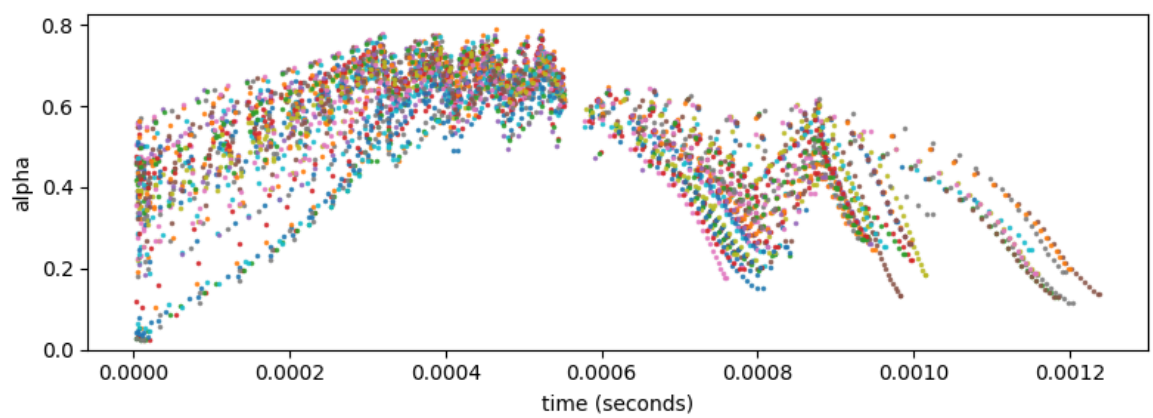
Figure



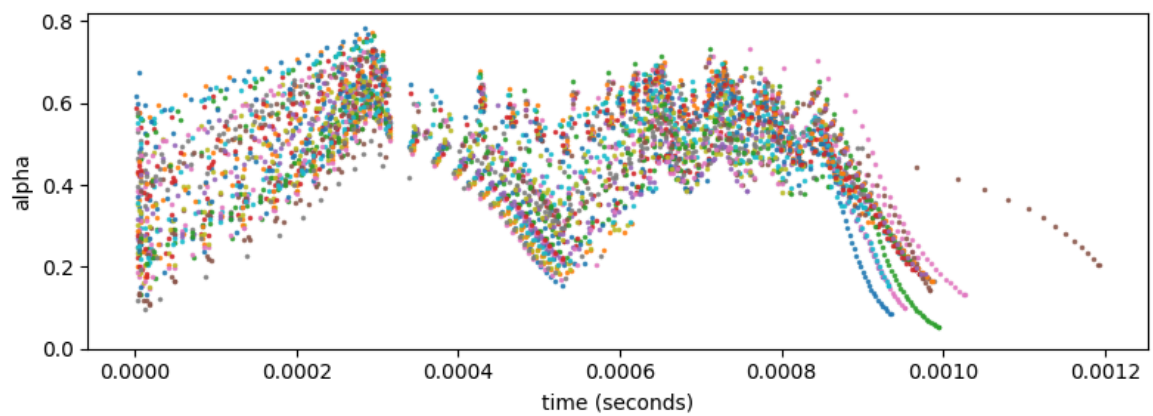
Figure



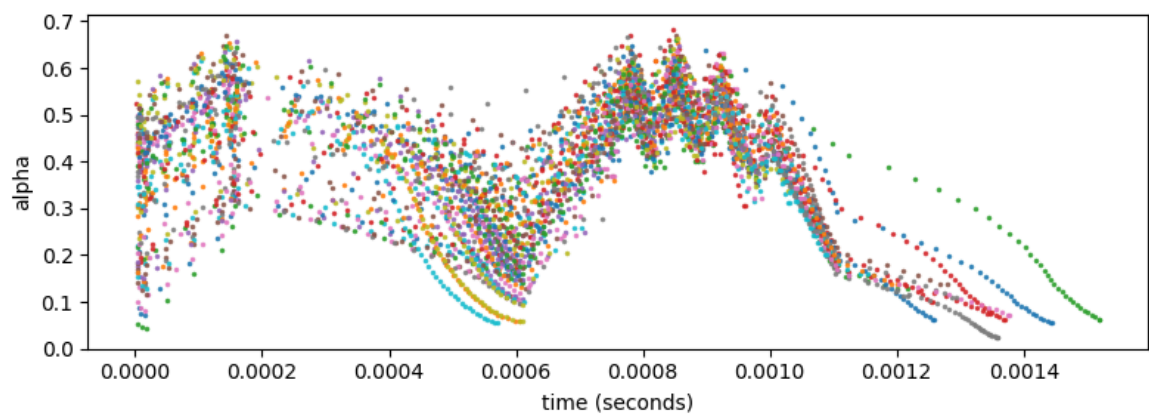
Figure



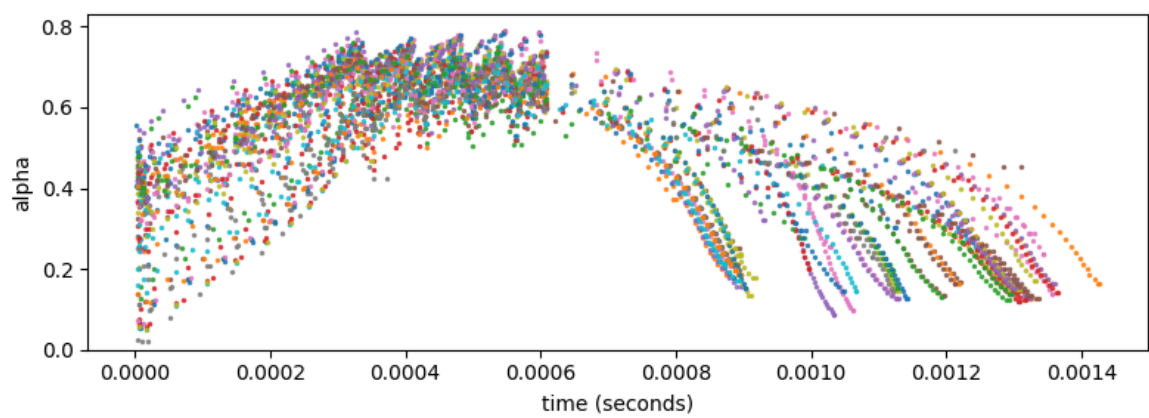
Figure



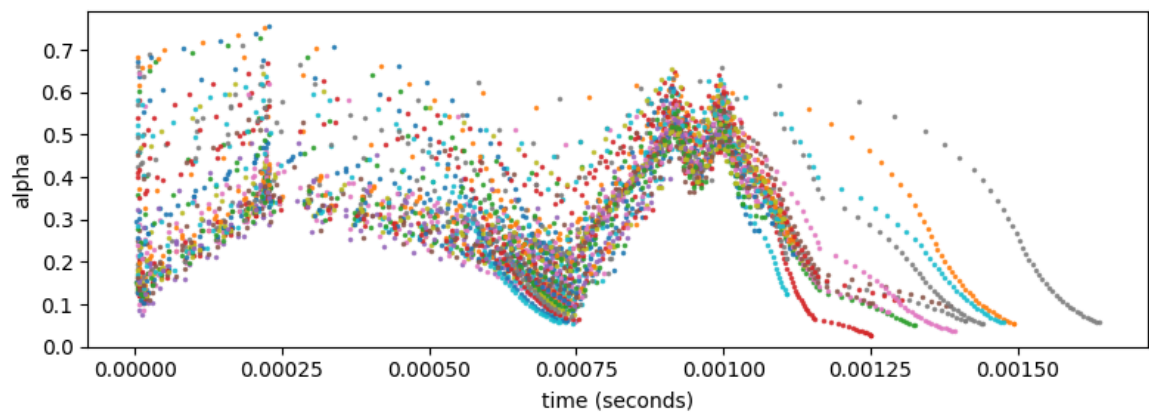
Figure



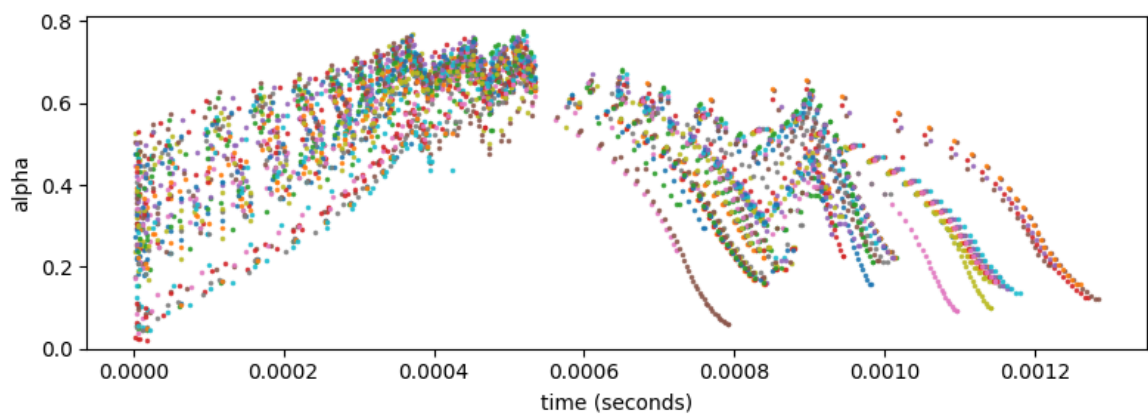
Figure



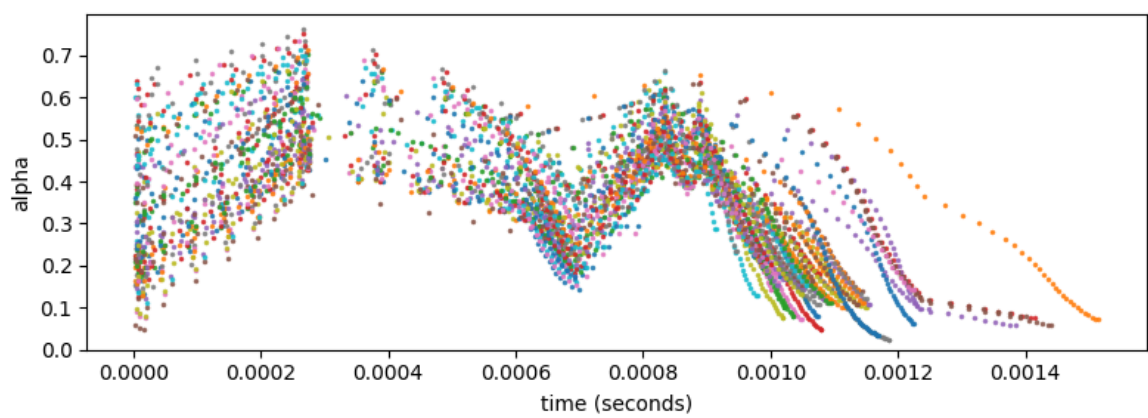
Figure



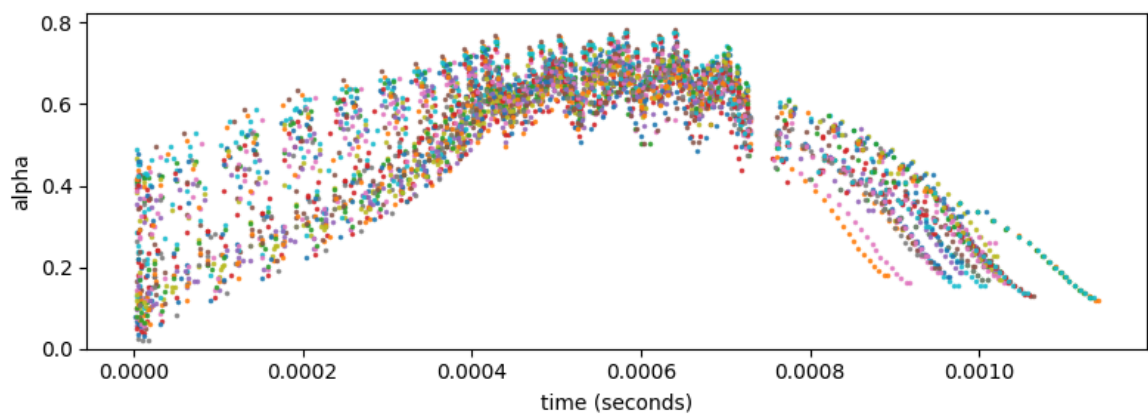
Figure



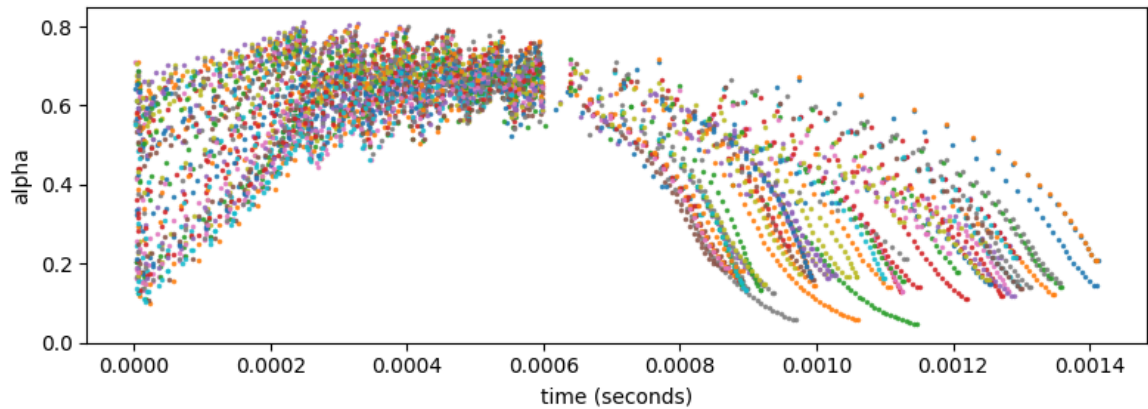
Figure



Figure



Figure



```
In [24]: def parse_rtt_line(line):
    parts = line.strip().split(" ")
    assert len(parts) == 2
    time_sec, rtt_us = parts
    time_sec = float(time_sec)
    rtt_us = float(rtt_us)
    return time_sec, rtt_us

def parse_rtts(flp):
    with open(flp, "r", encoding="utf-8") as fil:
        return [parse_rtt_line(line) for line in fil if line.strip()[0] != " "]

def get_sender_to_rtts_by_burst(exp_dir, burst_times, sender_to_flow_times_by_burst):
    return {
        parse_sender(flp): separate_samples_into_bursts(
            # Read all congestion estimate samples for this sender
            parse_rtts(flp),
            burst_times,
            # Look up the start and end time for this sender
            sender_to_flow_times_by_burst[parse_sender(flp)],
            filter_on_flow_times=True,
            bookend=True,
        )
        # Look up all congestion estimate log files.
        for flp in [
            path.join(exp_dir, "logs", fln)
            for fln in os.listdir(path.join(exp_dir, "logs"))
            if fln.startswith("sender") and fln.endswith("_rtt.log")
        ]
    }

def graph_sender_rtt(sender_to_rtts_by_burst, num_bursts, graph_dir, prefix):
    for burst_idx in range(num_bursts):
        fig, axes = get_axes()
        ax = axes[0]

        # ax.set_title(
```

```

# "Sender-measured RTT of active flows: "
# f"Burst {burst_idx + 1} of {num_bursts}"
# )
ax.set_xlabel("time (seconds)")
ax.set_ylabel("Sender-measured RTT (us)")

for sender, bursts in sender_to_rtts_by_burst.items():
    if not bursts[burst_idx]:
        continue
    xs, ys = zip(*bursts[burst_idx])
    ax.plot(xs, ys, "o", markersize=1.5, label=sender, alpha=0.8)

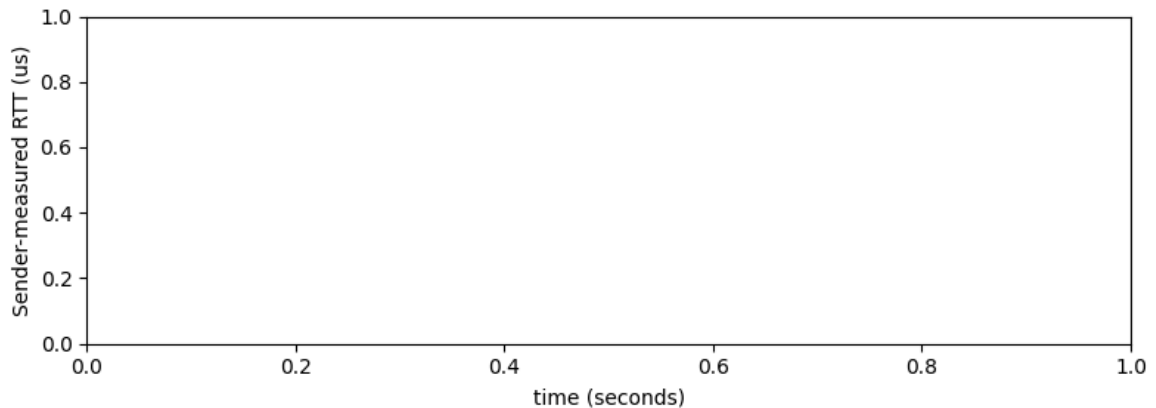
ax.set_ylim(bottom=0)

show(fig)
save(graph_dir, prefix, suffix=f"rtt_{burst_idx}")

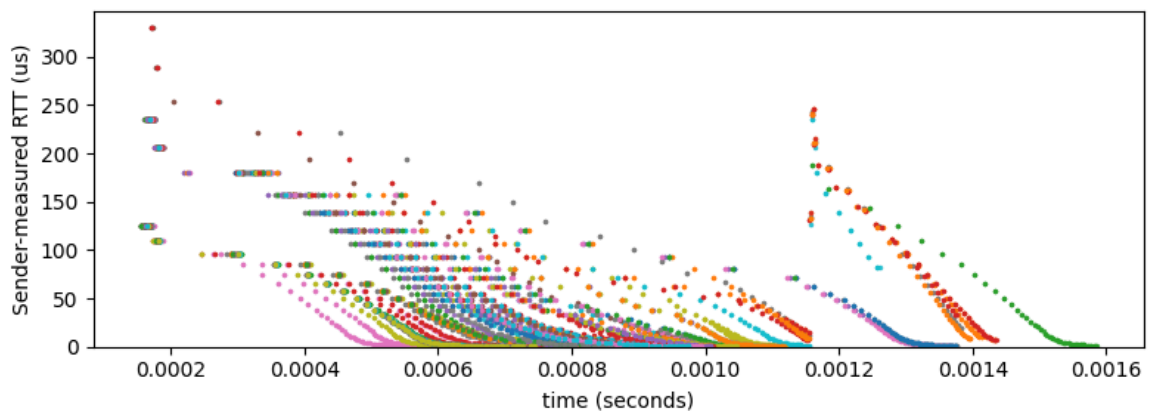
if RUN:
    SENDER_TO_RTTS_BY_BURST = get_sender_to_rtts_by_burst(
        EXP_DIR, BURST_TIMES, SENDER_TO_FLOW_TIMES_BY_BURST
    )
    graph_sender_rtt(SENDER_TO_RTTS_BY_BURST, NUM_BURSTS, GRAPH_DIR, EXP)

```

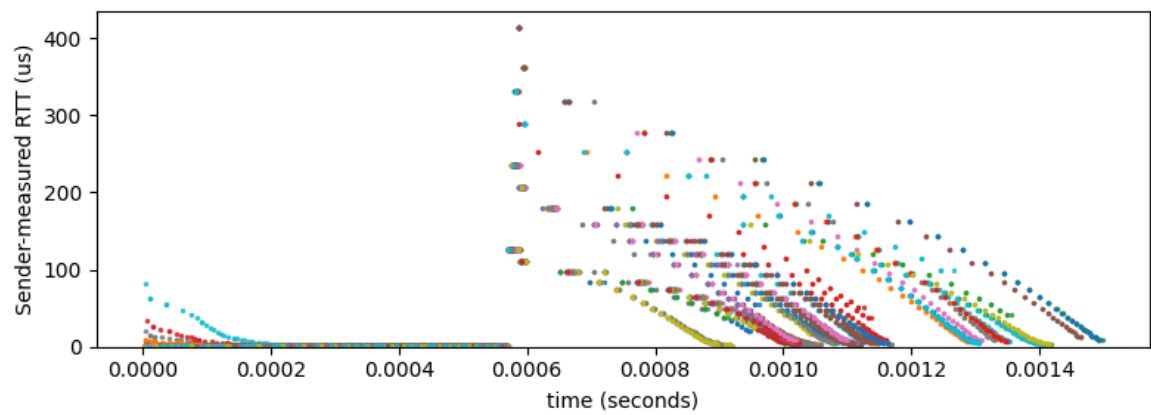
Figure



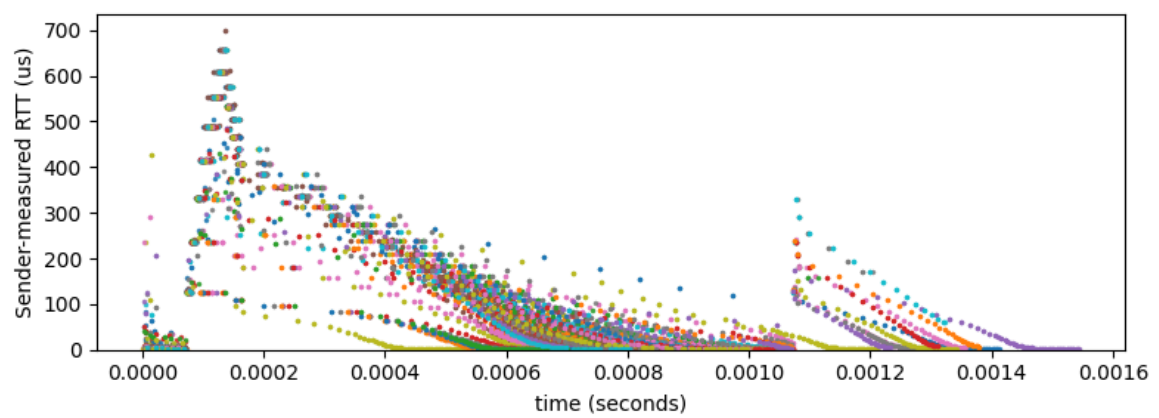
Figure



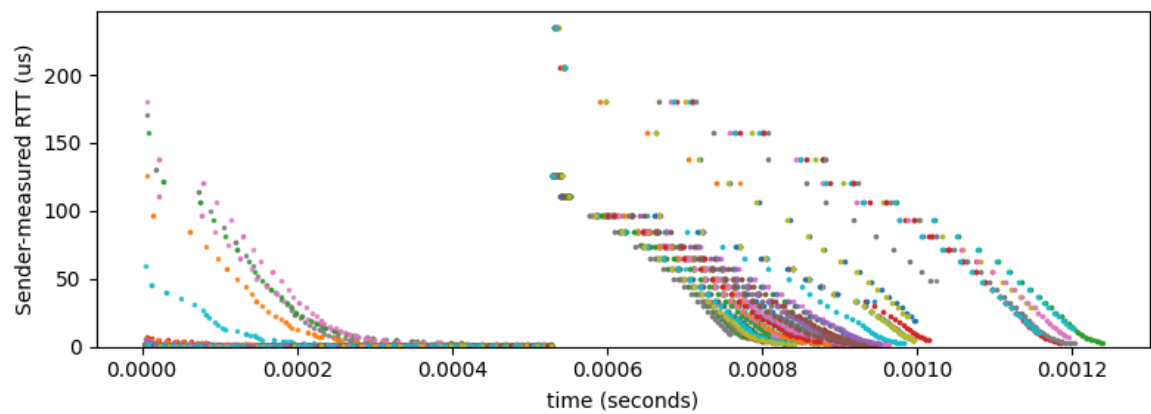
Figure



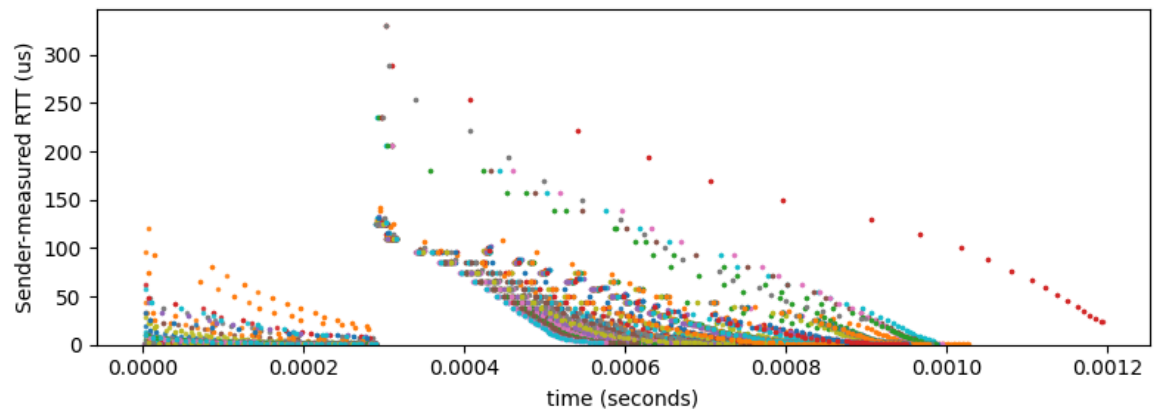
Figure



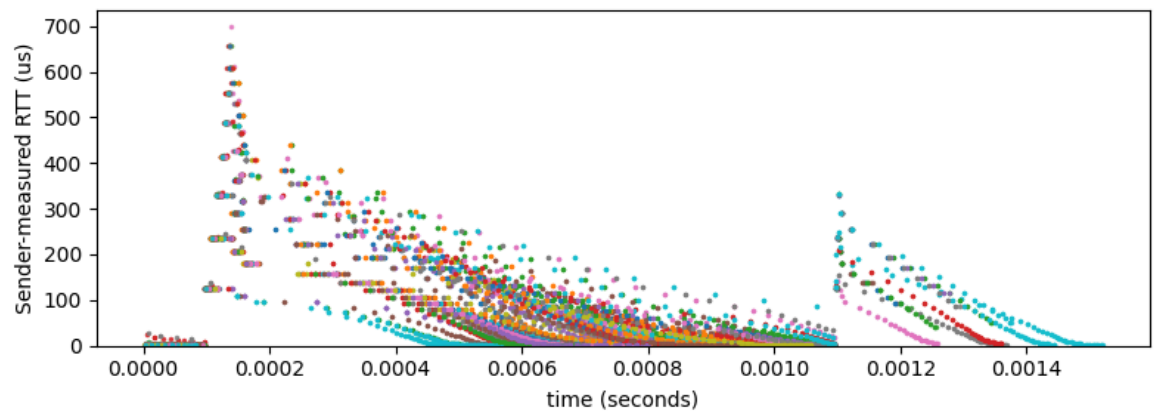
Figure



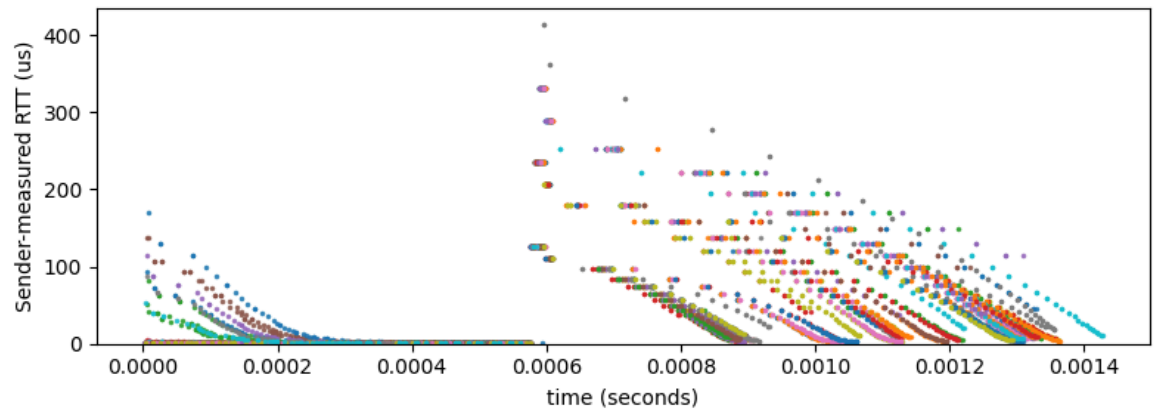
Figure



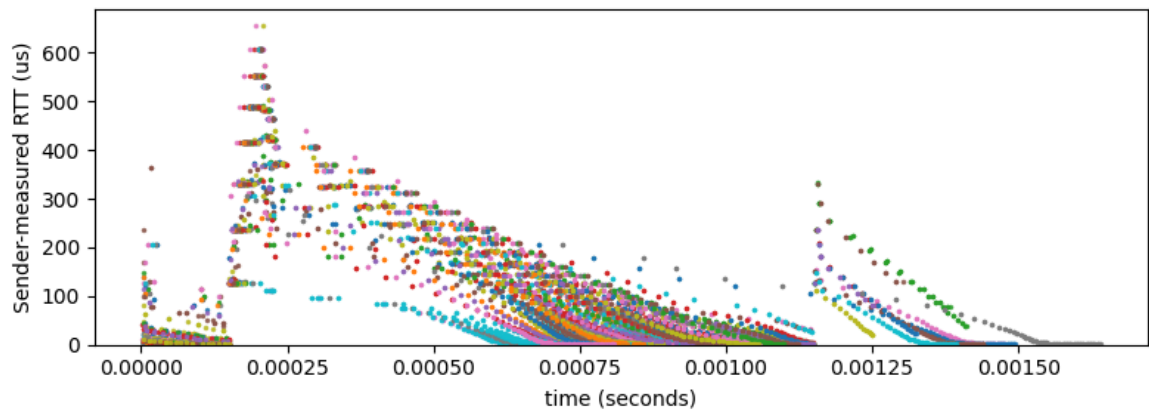
Figure



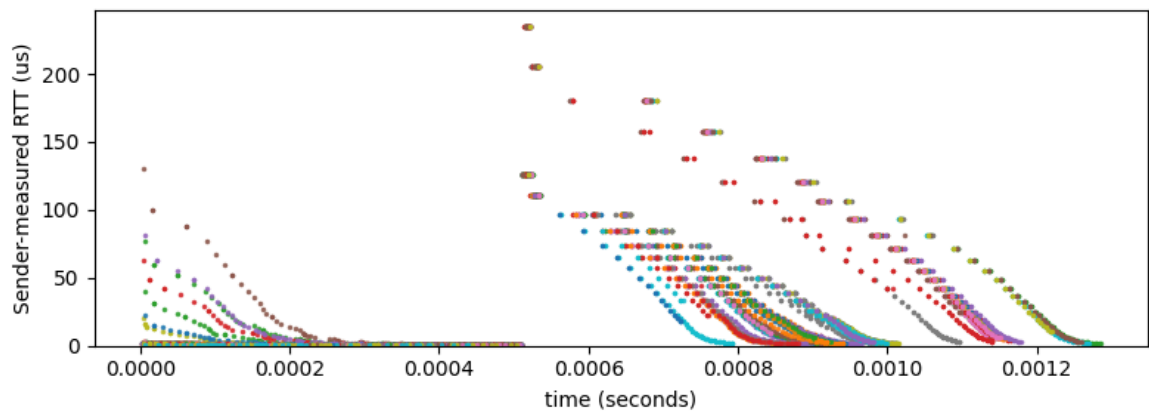
Figure



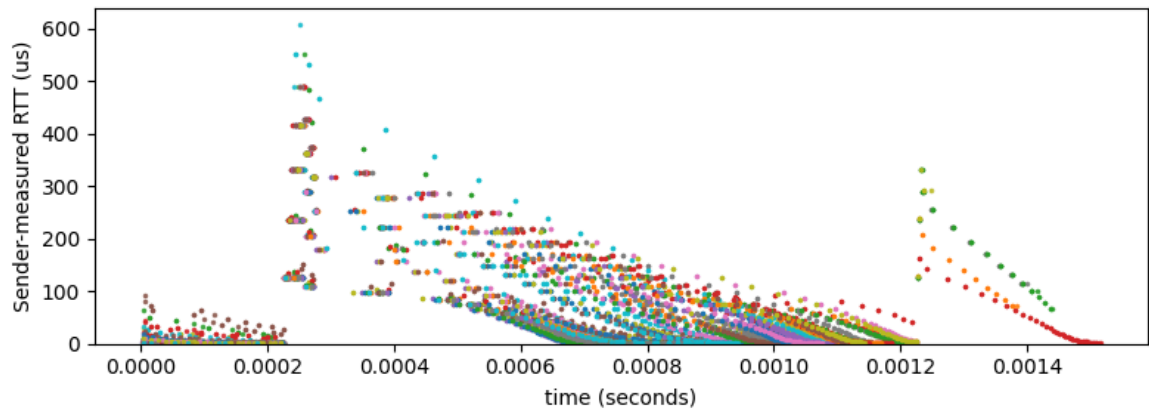
Figure



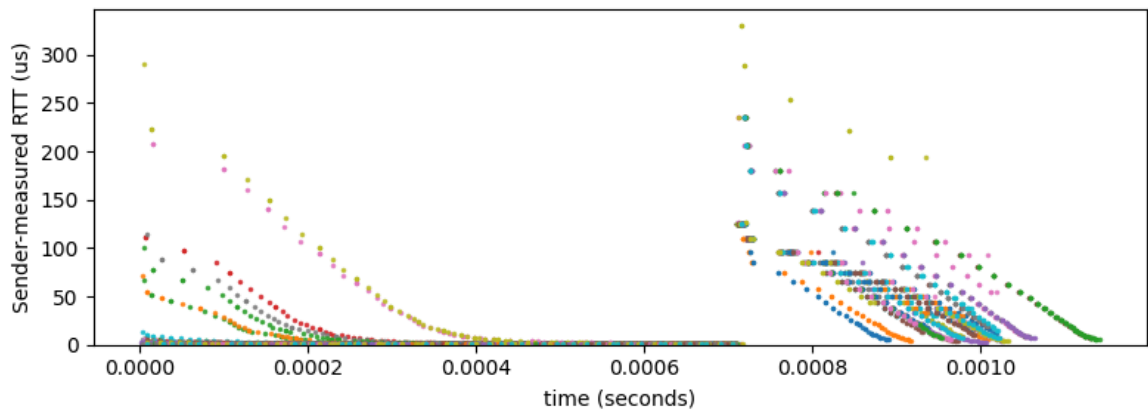
Figure



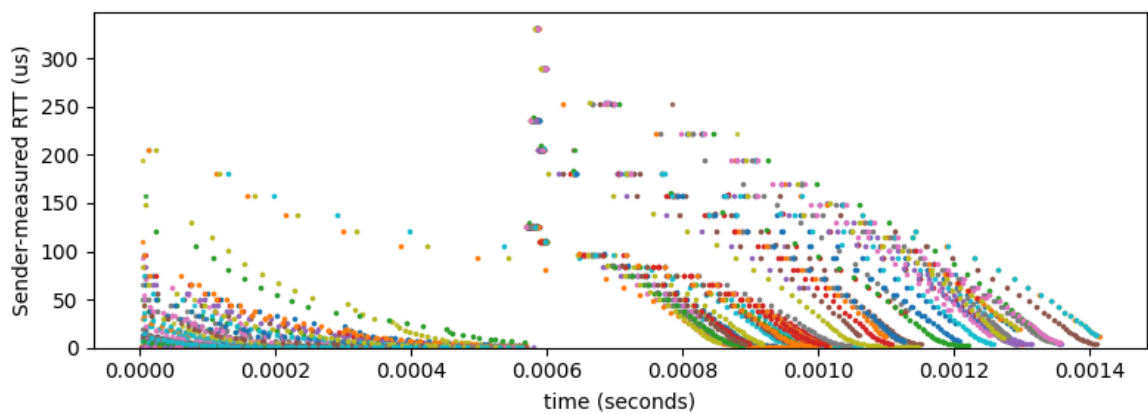
Figure



Figure



Figure



```
In [25]: def graph_rtt_cdf(sender_to_rtt_by_burst, num_bursts, graph_dir, prefix):
    for burst_idx in range(num_bursts):
        fig, axes = get_axes(width=5)
        ax = axes[0]

        rtt_us = [
            x[1]
            for sender_rtt in sender_to_rtt_by_burst.values()
            for x in sender_rtt[burst_idx]
        ]

        # Plot CDF of ACK size across all senders
        count, bins_count = np.histogram(rtt_us, bins=len(rtt_us))
        ax.plot(
            bins_count[1:],
            np.cumsum(count / sum(count)),
            alpha=0.8,
            label="ACKed MSS",
        )

        # ax.set_title(f"CDF of RTT (us): Burst {burst_idx + 1} of {num_bursts}")
        ax.set_xlabel("RTT (us)")
        ax.set_ylabel("CDF")
        ax.set_xlim(left=0)
        ax.set_ylim(bottom=0, top=1.01)
```

```
show(fig)
save(graph_dir, prefix, suffix=f"rtt_cdf_{burst_idx}")

if RUN:
    graph_rtt_cdf(SENDER_TO_RTTS_BY_BURST, NUM_BURSTS, GRAPH_DIR, EXP)
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[25], line 32
     28         save(graph_dir, prefix, suffix=f"rtt_cdf_{burst_idx}")
     31 if RUN:
--> 32     graph_rtt_cdf(SENDER_TO_RTTS_BY_BURST, NUM_BURSTS, GRAPH_DIR, EX
P)

Cell In[25], line 13, in graph_rtt_cdf(sender_to_rtts_by_burst, num_bursts,
graph_dir, prefix)
     6 rtt_us = [
     7     x[1]
     8     for sender_rtts in sender_to_rtts_by_burst.values()
     9     for x in sender_rtts[burst_idx]
    10 ]
    12 # Plot CDF of ACK size across all senders
--> 13 count, bins_count = np.histogram(rtt_us, bins=len(rtt_us))
    14 ax.plot(
    15     bins_count[1:],
    16     np.cumsum(count / sum(count)),
    17     alpha=0.8,
    18     label="ACKed MSS",
    19 )
    21 # ax.set_title(f"CDF of RTT (us): Burst {burst_idx + 1} of {num_burs
ts}")

File ~/miniconda3/envs/incast/lib/python3.11/site-packages/numpy/lib/_histog
rams_impl.py:792, in histogram(a, bins, range, density, weights)
    687 r"""
    688 Compute the histogram of a dataset.
    689
    (...)      788
    789 """
    790 a, weights = _ravel_and_check_weights(a, weights)
--> 792 bin_edges, uniform_bins = _get_bin_edges(a, bins, range, weights)
    794 # Histogram is an integer or a float array depending on the weights.
    795 if weights is None:

File ~/miniconda3/envs/incast/lib/python3.11/site-packages/numpy/lib/_histog
rams_impl.py:423, in _get_bin_edges(a, bins, range, weights)
    420         raise TypeError(
    421             '`bins` must be an integer, a string, or an array') from
e
    422     if n_equal_bins < 1:
--> 423         raise ValueError("`bins` must be positive, when an integer')
    425     first_edge, last_edge = _get_outer_edges(a, range)
    427 elif np.ndim(bins) == 1:

ValueError: `bins` must be positive, when an integer

```

```

In [ ]: def graph_acks_per_congest_cdf(
        sender_to_congest_by_burst, num_bursts, mss, graph_dir, prefix
    ):
        for burst_idx in range(num_bursts):
            fig, axes = get_axes(width=5)
            ax = axes[0]

```

```

acks_per_congest_bytes = [
    b[2] / mss
    for sender_congests in sender_to_congest_by_burst.values()
    for b in sender_congests[burst_idx]
]

# Plot CDF of ACKs per congest across all senders
count, bins_count = np.histogram(
    acks_per_congest_bytes, bins=len(acks_per_congest_bytes)
)
ax.plot(
    bins_count[1:],
    np.cumsum(count / sum(count)),
    alpha=0.8,
    label="ACKs per congestion estimate",
)

# ax.set_title(
#     f"CDF of ACKs per congestion estimate: Burst {burst_idx + 1} c
# )
ax.set_xlabel("ACKs per congestion estimate")
ax.set_ylabel("CDF")
ax.set_xlim(left=0)
ax.set_ylim(bottom=0, top=1.01)

show(fig)
save(graph_dir, prefix, suffix=f"acks_per_congest_cdf_{burst_idx}")

if RUN:
    MSS = 1448
    graph_acks_per_congest_cdf(
        SENDER_TO_CONGEST_BY_BURST, NUM_BURSTS, MSS, GRAPH_DIR, EXP
    )

```

```

In [ ]: def parse_bytes_in_ack_line(line):
    parts = line.strip().split(" ")
    assert len(parts) == 6
    (
        time_sec,
        sender_ip,
        sender_port,
        aggregator_ip,
        aggregator_port,
        bytes_in_ack,
    ) = parts
    time_sec = float(time_sec)
    sender_port = int(sender_port)
    aggregator_port = int(aggregator_port)
    bytes_in_ack = int(bytes_in_ack)
    return (
        time_sec,
        sender_ip,
        sender_port,
        aggregator_ip,

```

```

        aggregator_port,
        bytes_in_ack,
    )

def parse_bytes_in_ack(flp):
    with open(flp, "r", encoding="utf-8") as fil:
        return [parse_bytes_in_ack_line(line) for line in fil if line.strip()]

def get_sender_to_bytes_in_ack_by_burst(exp_dir, burst_times):
    bytes_in_ack_flp = [
        path.join(exp_dir, "logs", fln)
        for fln in os.listdir(path.join(exp_dir, "logs"))
        if fln.startswith("aggregator") and fln.endswith("_bytes_in_ack.log")
    ]
    assert len(bytes_in_ack_flp) == 1
    bytes_in_ack_flp = bytes_in_ack_flp[0]

    # Parse from disk. All sender stored together.
    bytes_in_ack = parse_bytes_in_ack(bytes_in_ack_flp)

    # Separate out the senders.
    sender_to_bytes_in_ack = collections.defaultdict(list)
    for record in bytes_in_ack:
        sender_to_bytes_in_ack[record[1]].append((record[0], record[5]))

    return {
        sender: separate_samples_into_bursts(
            # Read all congestion estimate samples for this sender
            bytes_in_ack,
            burst_times,
            None,
            filter_on_flow_times=False,
            bookend=False,
        )
        for sender, bytes_in_ack in sender_to_bytes_in_ack.items()
    }

def graph_ack_size_cdf(
    sender_to_bytes_in_ack_by_burst, num_bursts, mss, graph_dir, prefix
):
    for burst_idx in range(num_bursts):
        fig, axes = get_axes(width=5)
        ax = axes[0]

        bytes_in_ack = [
            b[1] / mss
            for sender_bytes_in_ack in sender_to_bytes_in_ack_by_burst.values()
            for b in sender_bytes_in_ack[burst_idx]
        ]

        # Plot CDF of ACKs per congest across all senders
        count, bins_count = np.histogram(bytes_in_ack, bins=len(bytes_in_ack))
        ax.plot(

```

```

        bins_count[1:],
        np.cumsum(count / sum(count)),
        alpha=0.8,
        label="ACK size",
    )

    # ax.set_title(f"CDF of ACK size: Burst {burst_idx + 1} of {num_bursts}")
    ax.set_xlabel("ACK size (MSS)")
    ax.set_ylabel("CDF")
    ax.set_xlim(left=0)
    ax.set_ylim(bottom=0, top=1.01)

    show(fig)
    save(graph_dir, prefix, suffix=f"ack_size_cdf_{burst_idx}")

if RUN:
    SENDER_TO_BYTES_IN_ACK_BY_BURST = get_sender_to_bytes_in_ack_by_burst(
        EXP_DIR, BURST_TIMES
    )
    graph_ack_size_cdf(SENDER_TO_BYTES_IN_ACK_BY_BURST, NUM_BURSTS, MSS, GRA

```

```

In [ ]: # TODO: Graph throughput over time

# Graph throughput at the aggregator

# TODO: Add JSON log file mapping sender ID to sender IP

# Log file will be similar to the

# Strategy:
#   Create buckets, sum up the number of bytes arriving in each bucket.

def parse_data_bytes_line(line):
    parts = line.strip().split(" ")
    assert len(parts) == 6
    (
        time_sec,
        sender_ip,
        sender_port,
        aggregator_ip,
        aggregator_port,
        frame_bytes,
    ) = parts
    time_sec = float(time_sec)
    sender_port = int(sender_port)
    aggregator_port = int(aggregator_port)
    frame_bytes = int(frame_bytes)
    return (
        time_sec,
        sender_ip,
        sender_port,
        aggregator_ip,
        aggregator_port,

```

```

        frame_bytes,
    )

def parse_data_bytes(flp):
    print(flp)
    with open(flp, "r", encoding="utf-8") as fil:
        return [parse_data_bytes_line(line) for line in fil if line.strip()]

def get_sender_to_data_bytes_by_burst(exp_dir, burst_times):
    data_bytes_flp = [
        path.join(exp_dir, "logs", fln)
        for fln in os.listdir(path.join(exp_dir, "logs"))
        if fln.startswith("aggregator") and fln.endswith("_bytes_received.log")
    ]
    assert len(data_bytes_flp) == 1
    data_bytes_flp = data_bytes_flp[0]

    # Parse from disk. All sender stored together.
    data_bytes = parse_data_bytes(data_bytes_flp)

    # Separate out the senders.
    sender_to_data_bytes = collections.defaultdict(list)
    for record in data_bytes:
        sender_to_data_bytes[record[1]].append((record[0], record[5]))

    return {
        sender: separate_samples_into_bursts(
            # Read all congestion estimate samples for this sender
            data_bytes,
            burst_times,
            None,
            filter_on_flow_times=False,
            bookend=False,
        )
        for sender, data_bytes in sender_to_data_bytes.items()
    }

def data_bytes_to_throughput(data_bytes, bucket_sec, rate_bps):
    rate_bytes_per_sec = rate_bps / 8

    times, byts = zip(*data_bytes)
    assert (np.diff(times) >= 0).all()
    start_sec = times[0]
    end_sec = times[-1]

    buckets = [
        # Bucket: [start time (inclusive), end time (exclusive), bytes]
        [start_sec + i * bucket_sec, start_sec + (i + 1) * bucket_sec, 0]
        # Start at -1 to add an extra bucket in case the first packet started
        # being transmitted before the start of the first bucket.
        for i in range(-1, math.ceil((end_sec - start_sec) / bucket_sec))
    ]

```

```

bucket_idx = 0
for transmit_end_sec, byts in data_bytes:
    # Find the bucket in which this transmission ended.
    while transmit_end_sec > buckets[bucket_idx][1]:
        bucket_idx += 1

    transmit_sec = byts / rate_bytes_per_sec
    transmit_start_sec = transmit_end_sec - transmit_sec

    bucket_start_sec, _, _ = buckets[bucket_idx]
    if transmit_start_sec < bucket_start_sec:
        # Make sure that this transmission does not span more than two buckets.
        assert transmit_start_sec >= buckets[bucket_idx - 1][0]

        # Split the bytes between previous and current buckets.
        prev_bucket_fraction_sec = (
            bucket_start_sec - transmit_start_sec
        ) / transmit_sec

        buckets[bucket_idx - 1][2] += prev_bucket_fraction_sec * byts
        buckets[bucket_idx][2] += (1 - prev_bucket_fraction_sec) * byts
    else:
        # Assign all bytes to the current bucket.
        buckets[bucket_idx][2] += byts

# print(buckets[:10], sep="\n")
return [
    # Pick the midpoint of the bucket as the x-coordinate.
    ((end_sec + start_sec) / 2, byts * 8 / bucket_sec)
    for start_sec, end_sec, byts in buckets
]

def data_bytes_to_throughput_gradient(data_bytes, rate_bps):
    bytes_per_sec = rate_bps / 8
    sec_per_byte = 1 / bytes_per_sec

    times, byts = zip(*data_bytes)
    assert (np.diff(times) >= 0).all()

    # Calculate the arrival time of each byte in each packet.
    one_byte_at_a_time = []
    for transmit_end_sec, byts in data_bytes:
        transmit_sec = byts / bytes_per_sec
        transmit_start_sec = transmit_end_sec - transmit_sec

        # Calculate the arrival time of each byte in this packet.
        for i in range(1, byts + 1):
            one_byte_at_a_time.append((transmit_start_sec + i * sec_per_byte, byts))

    # The x-axis is the byte arrival time. The y-axis is the total bytes
    # received until that time, which is, in effect, a measure of distance or
    # position. We have a measure of position over time, so we can take the
    # derivative to calculate the rate or throughput. If we take the second
    # derivative, then we get the acceleration, which in this case is the rate
    # of change in throughput, which can be interpreted as the congestion

```

```

# control algorithm's reactivity.
individual_byte_times, individual_bytes = zip(*one_byte_at_a_time)
cumulative_bytes = np.cumsum(individual_bytes)
rates = np.gradient(cumulative_bytes, individual_byte_times)
# Return: (time, position, rate, acceleration)
return zip(
    individual_byte_times,
    cumulative_bytes,
    rates * 8,
    np.gradient(rates, individual_byte_times) * 8,
)

def graph_throughput(
    sender_to_data_bytes_by_burst,
    rate_bps,
    num_bursts,
    graph_dir,
    prefix,
    bucket_sec,
    merge_senders=False, # Whether to graph the throughput of individual se
):
    for burst_idx in range(num_bursts):
        fig, axes = get_axes()
        ax = axes[0]

        # ax.set_title(
        #     "Throughput at aggregator: "
        #     f"Burst {burst_idx + 1} of {num_bursts}"
        # )
        ax.set_xlabel("time (seconds)")
        ax.set_ylabel("throughput (bps)")

        if merge_senders:
            merged = []
            for sender, bursts in sender_to_data_bytes_by_burst.items():
                if not bursts[burst_idx]:
                    continue

                merged.extend(bursts[burst_idx])
            merged = sorted(merged, key=lambda x: x[0])

            times, rates = zip(*data_bytes_to_throughput(merged, bucket_sec,
            ax.plot(times, rates, alpha=0.8)
        else:
            for sender, bursts in sender_to_data_bytes_by_burst.items():
                if not bursts[burst_idx]:
                    continue
                # times, _, rates, _ = zip(*data_bytes_to_throughput_gradier
                times, rates = zip(
                    *data_bytes_to_throughput(bursts[burst_idx], bucket_sec,
                )
                ax.plot(times, rates, label=sender, alpha=0.8)

        ax.set_ylim(bottom=0)

```

```

        # Draw a line at the downlink bandwidth
        # ax.plot(
        #     [xs[0], xs[-1]],
        #     [marking_threshold_packets] * 2,
        #     label="Marking threshold",
        #     color="orange",
        #     linestyle="dashed",
        #     alpha=0.8,
        # )

        show(fig)
        save(graph_dir, prefix, suffix=f"tput_{burst_idx}")

if RUN:
    # Set bucket granularity to 10 us. For reference, it takes about 1 us to
    TPUT_BUCKET_SEC = 1e-5

    SENDER_TO_DATA_BYTES_BY_BURST = get_sender_to_data_bytes_by_burst(
        EXP_DIR, BURST_TIMES
    )

    graph_throughput(
        SENDER_TO_DATA_BYTES_BY_BURST,
        CONFIG["smallLinkBandwidthMbps"] * 1e6,
        NUM_BURSTS,
        GRAPH_DIR,
        EXP,
        TPUT_BUCKET_SEC,
        merge_senders=False,
    )

```

```

In [ ]: if RUN:
        graph_throughput(
            SENDER_TO_DATA_BYTES_BY_BURST,
            CONFIG["smallLinkBandwidthMbps"] * 1e6,
            NUM_BURSTS,
            GRAPH_DIR,
            EXP,
            TPUT_BUCKET_SEC,
            merge_senders=True,
        )

```

```

In [ ]: def get_queue_length_across_bursts(
        lengths_by_burst, num_bursts, interp_delta, percentiles
    ):
        # We always ignore the first burst, since it is different than the other
        # due to slow start.
        if num_bursts == 1:
            print(
                "Error: No results because we ignore the frst burst, but there i
            )
            return

        # Interp each burst

```

```

lengths_by_burst_interp = []
for burst_idx in range(1, num_bursts):
    assert len(lengths_by_burst[burst_idx]) > 0
    new_xs = get_aligned_xs(
        lengths_by_burst[burst_idx][0][0],
        lengths_by_burst[burst_idx][-1][0],
        interp_delta,
    )
    assert len(new_xs) > 0
    new_ys = step_interp(*zip(*lengths_by_burst[burst_idx]), new_xs)
    new_xs -= new_xs[0]
    lengths_by_burst_interp.append(list(zip(new_xs, new_ys)))

# Create a new xs array that covers the longest burst
# for points in lengths_by_burst_interp:
#     print(points[0][0], points[-1][0], len(points))
# end_x = max(points[-1][0] for points in lengths_by_burst_interp)
# print("end_x", end_x)
# print("interp_delta", interp_delta)
# print("end_x * interp_delta", end_x * interp_delta)
# print("range end", math.floor(end_x * interp_delta) + 1)
# Add an extra +1 to account for floating point error
# xs = np.array(
#     [x / interp_delta for x in range(math.floor(end_x * interp_delta)
# )
# )
xs = get_aligned_xs(
    0, max(points[-1][0] for points in lengths_by_burst_interp), interp_
)

# Calculate across bursts.
avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, sum_ys = tolerant_metric(
    xs, lengths_by_burst_interp, percentiles
)
assert len(xs) == len(avg_ys)
assert len(xs) == len(stdev_ys)
assert len(xs) == len(min_ys)
assert len(xs) == len(max_ys)
assert len(xs) == percentiles_ys.shape[1]
assert len(xs) == len(sum_ys)
return xs, avg_ys, stdev_ys, min_ys, max_ys, percentiles_ys, sum_ys

def graph_queue_across_bursts(
    queue_name,
    lengths_across_bursts,
    marking_threshold_packets,
    capacity_packets,
    graph_dir,
    prefix,
):
    fig, axes = get_axes()
    ax = axes[0]

    # Plot lengths
    xs, avg_ys, _, _, _, _ = lengths_across_bursts
    xs = xs * 1e3

```

```

max_y = max(avg_ys)
ax.plot(
    xs,
    avg_ys,
    label="queue length",
    drawstyle="steps-post",
    linewidth=LINESIZE,
    alpha=0.8,
)
# Draw a line at the marking threshold
ax.plot(
    [xs[0], xs[-1]],
    [marking_threshold_packets] * 2,
    label="ECN threshold",
    color="orange",
    linestyle="dashed",
    linewidth=LINESIZE,
    alpha=0.8,
)
# For readability, only draw a line at the capacity if the max y is at 1
if max_y > capacity_packets / 2:
    # Draw a line at the queue capacity
    ax.plot(
        [xs[0], xs[-1]],
        [capacity_packets] * 2,
        label="queue capacity",
        color="red",
        linestyle="dotted",
        linewidth=LINESIZE,
        alpha=0.8,
    )
    max_y = capacity_packets
else:
    max_y = capacity_packets / 2

ax.set_xlabel("time (ms)", fontsize=FONTSIZE)
ax.set_ylabel("packets", fontsize=FONTSIZE)
ax.tick_params(axis="x", labelsz=FONTSIZE)
ax.tick_params(axis="y", labelsz=FONTSIZE)
ax.set_xlim(left=-0.01 * xs[-1], right=1.01 * xs[-1])
ax.set_ylim(bottom=-0.01 * max_y, top=1.1 * max_y)
ax.legend(fontsize=FONTSIZE, loc="upper right", ncols=1)

show(fig)
save(
    graph_dir,
    prefix,
    suffix="_".join(queue_name.split(" ")).lower() + "_length_across_bursts",
)

if RUN:
    INCAST_Q_length_ACROSS_BURSTS = get_queue_length_across_bursts(
        INCAST_Q_METRICS["lengths"], NUM_BURSTS, INTERP_DELTA, PERCENTILES
    )
    graph_queue_across_bursts(

```

```

    "Incast Queue",
    INCAST_Q_length_ACROSS_BURSTS,
    MARKING_THRESHOLD,
    QUEUE_CAPACITY,
    GRAPH_DIR,
    EXP,
)

```

```

In [ ]: def get_all_metrics_for_exp(
    exp_dir,
    interp_delta=1e5,
    percentiles=[0, 25, 50, 75, 95, 100],
    bytes_per_packet=1500,
    filt=None,
    desired=None,
):
    print(f>Loading: {exp_dir}")
    config = get_config_json(exp_dir)

    # If this experiment does not match the filter, then skip it.
    if filt is not None and not filt(config):
        return None

    num_bursts = config["numBursts"]
    burst_times = get_burst_times(exp_dir)
    sender_to_flow_times_by_burst = get_sender_to_flow_times_by_burst(exp_dir,
    sender_to_cwnds_by_burst = get_sender_to_cwnds_by_burst(
        exp_dir, burst_times, sender_to_flow_times_by_burst
    )
    sender_to_cwnds_by_burst_interp = get_sender_to_x_by_burst_interp(
        sender_to_cwnds_by_burst, num_bursts, interp_delta
    )
    sender_to_inflight_by_burst = get_sender_to_inflight_by_burst(
        exp_dir, burst_times, sender_to_flow_times_by_burst
    )
    sender_to_inflight_by_burst_interp = get_sender_to_x_by_burst_interp(
        sender_to_inflight_by_burst, num_bursts, interp_delta
    )
    sender_to_rtts_by_burst = get_sender_to_rtts_by_burst(
        exp_dir, burst_times, sender_to_flow_times_by_burst
    )
    incast_q_metrics = get_queue_metrics_by_burst(exp_dir, "Incast Queue", b
    uplink_q_metrics = get_queue_metrics_by_burst(exp_dir, "Uplink Queue", b
    return {
        "exp_dir": exp_dir,
        "config": config,
        "burst_times": burst_times,
        "avg_tput_by_burst_bps": [
            config["bytesPerBurstSender"]
            * 8
            * config["maxIncastScale"]
            / (end - start)
            for start, end in burst_times
        ],
        "sender_to_flow_times_by_burst": sender_to_flow_times_by_burst,
        "active_flows_by_burst": None
    }

```

```

if (desired is not None and "active_flows_by_burst" not in desired)
else get_active_flows_by_burst(sender_to_flow_times_by_burst, num_bu
"ideal_sec": (
    # config["bytesPerSender"]
    # * config["numSenders"]
    config["bytesPerBurstSender"]
    * config["maxIncastScale"]
    / (config["smallLinkBandwidthMbps"] * 1e6 / 8)
    + (6 * config["delayPerLinkUs"] / 1e6)
),
# lengths, drops, marks -> [burst 1, burst 2, ..]
"incast_queue_by_burst": None
if (desired is not None and "incast_queue_by_burst" not in desired)
else incast_q_metrics,
"uplink_queue_by_burst": None
if (desired is not None and "uplink_queue_by_burst" not in desired)
else uplink_q_metrics,
"incast_queue_across_bursts": None
if (desired is not None and "incast_queue_across_bursts" not in desi
else get_queue_length_across_bursts(
    incast_q_metrics["lengths"], num_bursts, interp_delta, percentil
),
"uplink_queue_across_bursts": None
if (desired is not None and "uplink_queue_by_burst" not in desired)
else get_queue_length_across_bursts(
    uplink_q_metrics["lengths"], num_bursts, interp_delta, percentil
),
"sender_to_cwnds_by_burst": None
if (desired is not None and "sender_to_cwnds_by_burst" not in desire
else sender_to_cwnds_by_burst,
"sender_to_cwnds_by_burst_interp": None
if (desired is not None and "sender_to_cwnds_by_burst_interp" not in
else sender_to_cwnds_by_burst_interp,
"cwnd_metrics_by_burst": None
if (desired is not None and "cwnd_metrics_by_burst" not in desired)
else get_metrics_by_burst(
    sender_to_cwnds_by_burst_interp, num_bursts, interp_delta, perce
),
"cwnd_metrics_across_bursts": None
if (desired is not None and "cwnd_metrics_across_bursts" not in desi
else get_cwnd_metrics_across_bursts(
    sender_to_cwnds_by_burst_interp, num_bursts, interp_delta, perce
),
"sender_to_inflight_by_burst": None
if (desired is not None and "sender_to_inflight_by_burst" not in des
else sender_to_inflight_by_burst,
"sender_to_inflight_by_burst_interp": None
if (desired is not None and "sender_to_inflight_by_burst_interp" not
else sender_to_inflight_by_burst_interp,
"inflight_metrics_by_burst": None
if (desired is not None and "inflight_metrics_by_burst" not in desir
else get_metrics_by_burst(
    sender_to_inflight_by_burst_interp, num_bursts, interp_delta, pe
),
"inflight_metrics_across_bursts": None
if (desired is not None and "inflight_metrics_across_bursts" not in

```

```

else get_inflight_metrics_across_bursts(
    sender_to_inflight_by_burst_interp, num_bursts, interp_delta, pe
),
"sender_to_congest_by_burst": None
if (desired is not None and "sender_to_congest_by_burst" not in desi
else get_sender_to_congest_by_burst(
    exp_dir, burst_times, sender_to_flow_times_by_burst
),
"sender_to_bytes_in_ack_by_burst": None
if (desired is not None and "sender_to_bytes_in_ack_by_burst" not in
else get_sender_to_bytes_in_ack_by_burst(exp_dir, burst_times),
"sender_to_rtts_by_burst": None
if (desired is not None and "sender_to_rtts_by_burst" not in desired
else sender_to_rtts_by_burst,
# "sender_to_rtts_by_burst_interp": get_sender_to_x_by_burst_interp(
#     sender_to_rtts_by_burst, num_bursts, interp_delta
# ),
"incast_q_above_empty": calculate_time_at_or_above_threshold(
    incast_q_metrics["lengths"],
    burst_times,
    1,
),
"incast_q_above_mark": calculate_time_at_or_above_threshold(
    incast_q_metrics["lengths"],
    burst_times,
    config["smallQueueMinThresholdPackets"],
),
"incast_q_above_90": calculate_time_at_or_above_threshold(
    incast_q_metrics["lengths"],
    burst_times,
    config["smallQueueSizePackets"] * 0.9,
),
"incast_q_avg_length_by_burst": calculate_average_queue_length(
    incast_q_metrics["lengths"],
    interp_delta,
    config["smallLinkBandwidthMbps"] * 1e6,
    bytes_per_packet,
),
"uplink_q_avg_length_by_burst": calculate_average_queue_length(
    uplink_q_metrics["lengths"],
    interp_delta,
    # config["largeLinkBandwidthMbps"] * 1e6,
    config["largeBurstLinkBandwidthMbps"] * 1e6,
    bytes_per_packet,
),
}

```

In []: